Punto 1.1 — Arquitectura monolítica vs. microservicios

Módulo 1: Introducción a los microservicios

Objetivos de aprendizaje:

- Comprender qué es una arquitectura monolítica y qué características la definen.
- Comprender qué es una arquitectura de microservicios y qué características la definen.
- Identificar las ventajas y limitaciones de cada estilo arquitectónico.
- Reconocer los criterios técnicos y organizativos que justifican la adopción de microservicios.
- Crear la estructura inicial del proyecto creciente AceriaApp con una solución, un proyecto de API monolítico de referencia, un proyecto de tests y la documentación de la decisión arquitectónica.

Del diseño del código a la frontera de despliegue

Antes de comparar estilos, conviene seguir una misma operación de negocio en ambos. Crearemos la referencia de Órdenes de AceriaApp como una aplicación que atiende HTTP, aplica reglas y persiste datos. Sobre ese recorrido observaremos qué ocurre cuando todo se publica junto y qué nuevas obligaciones aparecen si se separan procesos. El propósito no es asociar una arquitectura con el éxito y otra con el fracaso, sino entender su efecto sobre cambios, operación y consistencia.

Definición de arquitectura monolítica

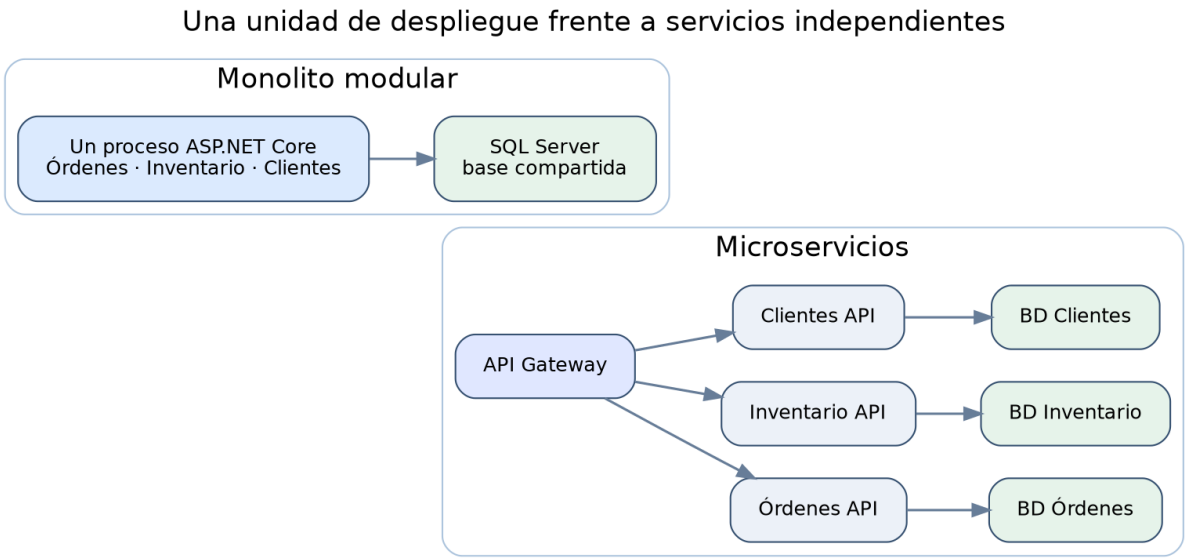


Figura. Comparación de límites de despliegue entre monolito modular y microservicios.

El rasgo que define a una aplicación monolítica no es la cantidad de carpetas ni de proyectos de Visual Studio, sino su **unidad de despliegue**. Podemos separar presentación, casos de uso, dominio y persistencia en varias bibliotecas y aun así publicar una única aplicación ASP.NET Core: cuando se modifica una capacidad de negocio, se genera y distribuye de nuevo ese conjunto. Este modelo ha sustentado numerosas aplicaciones empresariales porque facilita la comunicación interna, la puesta en marcha y el diagnóstico de los primeros errores. El riesgo aparece si confundimos esa comodidad

operativa con permiso para mezclar responsabilidades. En AceriaApp interesa comenzar por una unidad ejecutable que permita seguir una orden desde la petición HTTP hasta SQL Server, pero cuyos componentes se organicen de manera que sus dependencias puedan inspeccionarse y evolucionar. La modularidad es una propiedad del diseño; la distribución entre procesos es una decisión posterior.

La estructura interna de un monolito suele organizarse por capas: presentación, aplicación, dominio e infraestructura. Cada capa depende de la inferior y expone una interfaz estable al resto del sistema. Esta organización por capas permite separar responsabilidades dentro del proceso, aunque todas ellas compartan el mismo ciclo de vida de compilación y despliegue. El código fuente completo reside en un único repositorio y, normalmente, en una única solución de Visual Studio. Los equipos trabajan sobre el mismo conjunto de archivos, lo que facilita la reutilización de utilidades comunes y evita duplicaciones accidentales de lógica. La transacción de base de datos se convierte en el mecanismo natural para garantizar la consistencia, porque todas las operaciones ocurren sobre el mismo contexto de persistencia. La clase Orden pertenece al mismo proceso que las rutas HTTP, aunque residan en carpetas distintas. Si una acción de producción empieza a invocar directamente utilidades de inventario desde cada endpoint, la estructura por carpetas no bastará para preservar la autonomía lógica.

```
AceriaApp.Monolito/
├── AceriaApp.Monolito.Api/          → capa de presentación (endpoints)
├── AceriaApp.Monolito.Aplicacion/   → casos de uso y servicios
├── AceriaApp.Monolito.Dominio/      → entidades y reglas de negocio
└── AceriaApp.Monolito.Infraestructura/ → EF Core, repositorios, migraciones
```

El árbol distingue cuatro responsabilidades que conviene reconocer antes de escribir código. `Api` recibe peticiones; `Aplicacion` organiza los casos de uso; `Dominio` concentra las reglas que deberían seguir siendo válidas aunque cambiase el transporte; `Infraestructura` se ocupa de EF Core y los recursos externos. La estructura no representa cuatro microservicios, sino cuatro proyectos que pueden publicarse juntos. Su utilidad es que, al revisar una operación de órdenes, el alumno pueda identificar dónde debería tomarse cada decisión.

Esta separación lógica de proyectos dentro de un mismo despliegue prepara el terreno para entender por qué los microservicios aplican el mismo principio de separación, pero con un límite físico distinto. La organización por capas dentro de un monolito no impide que el sistema crezca en complejidad interna. Cuando varias funcionalidades comparten el mismo proceso, cualquier cambio en una de ellas obliga a recompilar y redesplegar el conjunto completo. La transacción única garantiza consistencia fuerte, pero también acopla el ciclo de vida de todas las funcionalidades. Este acoplamiento es aceptable en dominios pequeños y estables, y se vuelve problemático cuando el dominio crece y los equipos se multiplican. Esto no impide diseñar módulos con contratos internos bien definidos; la modularidad no exige distribuir procesos. El primer control de calidad consiste en localizar quién conoce a quién: presentación puede invocar aplicación, y persistencia puede implementar necesidades definidas en el núcleo.

En AceriaApp, la condición monolítica se verifica observando el límite de despliegue: el host de ASP.NET Core registra el DbContext, configura los endpoints y publica una aplicación. La separación entre capas debe comprobarse por la dirección de las dependencias y por la facilidad para modificar una regla sin alterar el transporte. Una solución con cuatro bibliotecas que se despliegan juntas sigue siendo monolítica.

La estructura de un monolito no se determina por el número de proyectos de Visual Studio. Una solución puede contener bibliotecas de dominio, aplicación e infraestructura, compilar varios ensamblados y seguir desplegándose como una sola aplicación. Para identificar la arquitectura hay que seguir una petición desde Kestrel hasta su almacenamiento: si los endpoints invocan componentes dentro de un proceso y todas las capacidades se actualizan como una unidad, el límite operativo es monolítico. Separar el código en bibliotecas ayuda a ordenar las dependencias; no crea,

por sí mismo, independencia de despliegue. Este matiz explica por qué el primer estado de AceriaApp utiliza una API ejecutable, aunque ya se distingan carpetas de dominio e infraestructura.

Un módulo lógico adecuado tiene una interfaz pequeña y un interior que puede cambiar sin afectar a los demás módulos. En el dominio de una acería, Órdenes debería responsabilizarse de las reglas de creación y estado de órdenes; Inventario debería administrar existencias y movimientos, y Clientes, la información comercial. Aunque todos compartan la misma base de datos al comienzo, conviene evitar que un endpoint de Órdenes modifique directamente la representación interna de Inventario. La modularidad se deteriora cuando una capacidad conoce las tablas, validaciones y estructuras privadas de todas las demás.

Elemento observado	Monolito modular	Microservicios independientes
Cambio en una regla	Se publica el host común	Se publica el servicio propietario
Llamada entre capacidades	Puede resolverse en memoria	Atraviesa una interfaz remota
Compilación	Puede abarcar varios ensamblados	Puede ejecutarse por servicio
Diagnóstico	Logs centralizados del proceso	Correlación entre procesos
Persistencia	Es posible compartir una transacción local	Las operaciones distribuidas requieren coordinación

La comparación muestra que modularizar no equivale a distribuir. Antes de separar procesos resulta más barato verificar el primer requisito: que una regla de negocio esté localizada. En el punto 1.1 puede localizarse `Orden` en `Dominio` y `AceriaDbContext` en `Infraestructura`; la revisión consiste en seguir esas referencias, observar qué cambios requiere cada funcionalidad y comprobar que el endpoint no es el único lugar donde se toman decisiones. El proyecto didáctico inicial es deliberadamente sencillo: prepara una referencia observable, no un ejemplo terminado de arquitectura hexagonal.

Características operativas del monolito

Imaginemos que tenemos que publicar una corrección en la validación de órdenes. En un monolito típico, el equipo ejecuta el pipeline común, despliega una versión de la aplicación y comprueba que el mismo proceso puede atender el resto de las funcionalidades. No necesita coordinar las versiones de varios servicios remotos; en cambio, debe someter el conjunto a una regresión suficiente para detectar efectos colaterales. El mismo criterio se aprecia al escalar: si se duplican las instancias del host, se duplica su consumo de memoria y de recursos, incluso cuando la carga procede de una parte pequeña del negocio. Una configuración centralizada, como `appsettings.json`, resulta fácil de localizar, pero exige saber qué valores pertenecen a cada entorno y cuáles nunca deben almacenarse en un repositorio. Estas características explican por qué el despliegue monolítico es sencillo sin que sea automáticamente barato a cualquier escala.

```
{
  "ConnectionStrings": {
    "AceriaDb": "Server=.\\SQLEXPRESS;Database=AceriaDb;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

La configuración muestra dos asuntos diferentes. `ConnectionStrings` establece qué almacén utilizará EF Core; `Logging:LogLevel` determina qué eventos se enviarán a los registros. Ambos pueden

cambiar entre el equipo de desarrollo y un despliegue, sin recompilar el código de negocio. La contraseña, cuando exista, no debería codificarse en este archivo versionado; para ese dato se utilizan secretos de desarrollo o configuración protegida del entorno.

El archivo de configuración único refleja la naturaleza centralizada del monolito: una sola fuente de verdad para todos los componentes. La observabilidad en un monolito se concentra en un único flujo de logs. Cuando ocurre un error, existe un solo archivo o una sola consola donde buscar la traza completa. Esta concentración acelera el diagnóstico inicial, pero también genera ruido: los logs de todas las funcionalidades se mezclan en el mismo flujo. El versionado del artefacto es lineal y cada versión contiene el estado de todas las funcionalidades en ese instante. No es posible desplegar una funcionalidad nueva sin arrastrar los cambios de las demás que se hayan fusionado en la misma rama.

La cadena de conexión del ejemplo utiliza autenticación integrada y un servidor SQL Server Express identificado mediante el nombre de instancia. Es una configuración didáctica de Windows, no una cadena recomendable para entregar en producción con credenciales incrustadas. La aplicación debe obtener los parámetros sensibles del entorno de ejecución y registrar mensajes que permitan diferenciar fallos de configuración de fallos de consulta. Al escalar réplicas del host, todas usarán la misma base de datos salvo que se diseñe otra distribución; por ello debe comprobarse que las operaciones no dependen de memoria local. Un contador guardado en una variable estática del proceso tendrá valores distintos en cada réplica. En cambio, una orden persistida en SQL Server es visible para las demás, respetando el aislamiento transaccional. La trazabilidad de peticiones se beneficia de identificadores de correlación aunque el sistema sea un único proceso.

Ventajas del monolito

La principal ventaja práctica de comenzar con un monolito bien organizado es que permite avanzar desde una necesidad del negocio hasta una respuesta observable con pocos componentes operativos. Una petición entra por Kestrel, invoca código del mismo proceso, utiliza un contexto de datos y devuelve un resultado. Este recorrido se puede depurar con un solo entorno de desarrollo y, cuando los cambios de varias entidades pertenecen al mismo ámbito de persistencia, puede cerrarse mediante una transacción local. La transacción es especialmente útil en una aplicación inicial como AceriaApp: si crear una orden y registrar un movimiento requieren confirmarse juntos, una transacción permite aceptar ambas escrituras o deshacerlas. La ventaja no proviene de que el monolito sea «mejor», sino de que evita introducir fronteras de red antes de necesitar sus beneficios. Conviene diferenciar este ejemplo conceptual de las operaciones que la primera etapa del curso implementa realmente.

```
using var scope = app.Services.CreateScope();
var db = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
await using var tx = await db.Database.BeginTransactionAsync();
db.Ordenes.Add(new Orden { Id = Guid.NewGuid(), Cliente = "ACX-001" });
db.Inventario.Add(new Movimiento { Id = Guid.NewGuid(), Material = "CHAPA-3MM" });
await db.SaveChangesAsync();
await tx.CommitAsync();
```

El ejemplo abre una transacción sobre `AceriaDbContext`, añade dos cambios y solo los confirma después de que `SaveChangesAsync` termine correctamente. Si la operación falla antes de `CommitAsync`, el ámbito transaccional permite revertir las escrituras pendientes. El fragmento es un esquema pedagógico: para ejecutarlo habría que disponer también de las entidades, propiedades y `DbSet` correspondientes a Inventario. Lo importante aquí es comprender qué garantía aporta una única base transaccional, no dar por implementado ese flujo en la primera práctica.

Esta transacción única es imposible de replicar de forma directa en microservicios, donde cada servicio posee su propia base de datos y la consistencia se gestiona con patrones como Saga. La tercera ventaja es la facilidad para refactorizar. Cuando todo el código reside en el mismo repositorio y

en el mismo proceso, renombrar una clase o mover una responsabilidad entre capas es una operación mecánica que el IDE resuelve sin coordinación entre equipos. La cuarta ventaja es la depuración: se puede ejecutar el sistema completo en la máquina del desarrollador con un único F5 y establecer puntos de interrupción en cualquier capa sin necesidad de levantar varios procesos.

La transacción del ejemplo agrupa varias operaciones, pero la mera presencia de `BeginTransactionAsync` no garantiza que todas las entidades y tablas pertenezcan al modelo. En el punto inicial solo se incluye la entidad Orden; la operación con Movimiento e Inventario es conceptual y requiere modelar esos tipos antes de compilarse. SaveChangesAsync ejecuta las escrituras pendientes; `CommitAsync` decide la confirmación definitiva de la transacción explícita. Si el guardado falla, disponer correctamente la transacción evita que los cambios queden confirmados. También debe distinguirse un test unitario de una comprobación transaccional: un test de propiedades de Orden no confirma el funcionamiento de SQL Server. La ventaja ACID se demuestra ejecutando una operación real, provocando deliberadamente un error entre dos escrituras y verificando que ninguna queda persistida. Esta evidencia fundamenta la comparación posterior con los costes de coordinación distribuida.

Una transacción de EF Core agrupa escrituras realizadas a través de la misma conexión y transacción de base de datos. La secuencia de crear una orden y registrar un movimiento parece simple, pero presenta dos condiciones distintas: el modelo debe contener ambas entidades y la ejecución debe llegar al mismo ámbito transaccional. En el primer estado de AceriaApp existe `Orden`, no la implementación completa de inventario. Por tanto, el ejemplo conjunto de este bloque describe la garantía que se quiere enseñar; no debe copiarse sin añadir antes las entidades y el mapeo faltantes. Diferenciar un fragmento conceptual de uno ejecutable evita que el alumno atribuya al proyecto capacidades que aún no ha construido.

La atomicidad se comprueba provocando un fallo entre dos operaciones de escritura y verificando el estado final de SQL Server Express. Si ambas escrituras se ejecutan dentro de una transacción y la segunda provoca una excepción antes de `CommitAsync`, ninguna debería quedar confirmada. En cambio, si se guarda y confirma la primera operación y después falla la segunda, la aplicación puede quedar en un estado incompleto. La propiedad ACID relevante aquí es la atomicidad; el aislamiento es otra característica, responsable de la visibilidad de cambios concurrentes. Conviene separar esas propiedades porque un caso de prueba que solo verifica que SaveChangesAsync no lanza excepciones no demuestra ninguna de ellas.

Para preparar una verificación reproducible deben anotarse el estado inicial de las tablas, los identificadores de las entidades afectadas y el resultado esperado tras el error. Un test con proveedor InMemory puede servir para reglas de dominio y consultas sencillas, pero no reproduce restricciones relacionales, aislamiento ni confirmación transaccional de SQL Server. Cuando se quiera verificar una propiedad de persistencia, el entorno adecuado será la base real y una comprobación posterior independiente. Esta distinción une el concepto teórico con la práctica: los tests de `Orden` introducen la automatización; la migración y el escenario HTTP/SQL acreditan que existe una persistencia ejecutable.

Limitaciones del monolito

Los inconvenientes de un monolito se manifiestan sobre todo cuando el **ciclo de vida compartido** deja de corresponderse con las necesidades del negocio. Si Órdenes tiene mucha más carga que el resto de capacidades, replicar el host completo consume recursos que esas otras funciones no demandan. Si distintos equipos necesitan publicar cambios diariamente, todos dependen de una misma cadena de integración y de una regresión común. Y si un componente monopoliza memoria o bloquea recursos críticos, la incidencia puede afectar a rutas que no tienen relación funcional con él. Ninguna de estas limitaciones implica que un monolito esté condenado a fracasar: separar módulos, limitar dependencias y medir cuellos de botella puede aplazar durante años una división en servicios.

La decisión de distribuir debe responder a dificultades observadas, no a un umbral arbitrario de líneas de código.

```
Monolito con 4 funcionalidades y escalado homogéneo:

[Instancia 1] Órdenes + Inventario + Clientes + Facturación
[Instancia 2] Órdenes + Inventario + Clientes + Facturación
[Instancia 3] Órdenes + Inventario + Clientes + Facturación

Si solo Órdenes necesita 3 instancias, las otras 3 funcionalidades
también se replican sin necesitarlo.
```

El esquema replica tres veces el mismo host. Todas las réplicas llevan Órdenes, Inventario, Clientes y Facturación, aunque el crecimiento de tráfico proceda solo de Órdenes. Esa es la diferencia entre escalar la unidad desplegable y escalar una capacidad concreta. Antes de concluir que la replicación es ineficiente, habría que medir cuánto consumen los componentes y si realmente existe un cuello de botella independiente.

La cuarta limitación es la organización de equipos. Cuando varias personas trabajan sobre el mismo artefacto, los conflictos de fusión se multiplican y las decisiones técnicas se centralizan. La quinta limitación es la adopción de nuevas tecnologías: migrar una parte del sistema a una versión superior del framework obliga a migrar todo el conjunto, porque el artefacto es indivisible. Estas limitaciones no invalidan el monolito como opción; simplemente definen el contexto en el que deja de ser la elección óptima.

El impacto del escalado homogéneo debe evaluarse sobre consumo real de CPU, memoria y concurrencia. Si Órdenes concentra las peticiones y Facturación casi no recibe tráfico, añadir réplicas del monolito aumenta la capacidad disponible para ambas funciones aunque solo una sea la causa. El coste adicional incluye memoria de arranque, conexiones potenciales a la base de datos y tiempo de inicialización del proceso. No implica necesariamente que todo el monolito sea ineficiente: un host ligero y una infraestructura sencilla pueden resultar más económicos que varios servicios mínimos. El dato útil no es el número de módulos, sino la relación entre carga y coste marginal de una instancia. Para justificar la extracción, conviene medir latencia del endpoint, porcentaje de CPU, uso de memoria y frecuencia de cambios; después se comprueba si la independencia de escalado compensa el coste de red y operación.

Definición de arquitectura de microservicios

La arquitectura de microservicios desplaza el límite de independencia desde los módulos de código hacia **procesos que se publican y operan por separado**. Una capacidad como Inventario puede tener su propia API, responsables, ciclo de despliegue y propiedad de datos, mientras Producción mantiene contratos explícitos para consultar existencias o notificar hechos relevantes. El cambio introduce una frontera real: donde antes existía una llamada de método, ahora intervienen protocolos, direcciones, tiempos de espera, errores parciales y evolución de contratos. La independencia tampoco exige que cada servicio tenga un servidor físico exclusivo; sí exige evitar que uno pueda escribir directamente el modelo privado de otro. Los nombres de los servicios del esquema son ilustrativos del diseño futuro de AceriaApp: no todos se materializan en el primer módulo. Es importante distinguir desde el principio entre una propuesta de descomposición y una solución distribuida ejecutable.

```
AceriaApp (microservicios)/
├── AceriaApp.Ordenes.Api/       → servicio de órdenes (puerto 5001)
├── AceriaApp.Inventario.Api/    → servicio de inventario (puerto 5002)
├── AceriaApp.Clientes.Api/      → servicio de clientes (puerto 5003)
└── AceriaApp.Gateway/           → API Gateway con YARP (puerto 5000)
```

Los directorios representan posibles límites de ejecución: cada API podría tener su propia configuración y ciclo de publicación, y YARP actuaría como entrada común. No conviene interpretar los

puertos del esquema como una topología ya creada ni pensar que un API Gateway convierte automáticamente una aplicación en microservicios. La separación solo sería efectiva si los servicios se comunicasen mediante contratos y respetasen la propiedad de sus datos.

Cada servicio de la lista anterior se compila, se despliega y se escala de forma independiente respecto a los demás. La autonomía de cada servicio implica que su equipo puede elegir el stack tecnológico, la versión del runtime y la cadencia de despliegue. Esta libertad acelera la innovación, pero también introduce heterogeneidad operativa. La organización de equipos alrededor de servicios, con equipos pequeños, autónomos y responsables de extremo a extremo, es uno de los principios fundacionales de este estilo arquitectónico. El límite del servicio debe coincidir con una capacidad de negocio, no con una capa técnica.

En el sistema distribuido propuesto, Órdenes e Inventario tienen responsabilidades y límites de almacenamiento diferentes. Es importante evitar la equivalencia entre “servicio pequeño” y “una sola entidad”: la capacidad de órdenes puede necesitar estados, validaciones, historiales y varias tablas sin perder cohesión. El Gateway representa una capa de entrada que enruta solicitudes; no debe contener las reglas de negocio de todos los servicios. Las direcciones y puertos del esquema son ejemplos de desarrollo, no contratos de negocio. Para comprobar la autonomía, debe ser posible cambiar la implementación de Inventario manteniendo la interfaz pública usada por Órdenes. Si ambos servicios requieren coordinar una actualización del esquema de la misma tabla, la frontera es nominal y no real. La descomposición se justifica cuando disminuye un acoplamiento problemático, no cuando simplemente multiplica aplicaciones ASP.NET Core.

Características operativas de los microservicios

La diferencia operativa aparece en las rutinas cotidianas del equipo. Cada servicio puede compilarse, probarse, configurarse y desplegarse con su propio artefacto, lo que permite ampliar una capacidad sin publicar obligatoriamente todas las demás. Sin embargo, esa autonomía exige contratos estables, gestión de configuración por entorno y mecanismos para observar una operación que atraviesa varios procesos. Cuando Producción consulta Inventario por HTTP, el éxito de ambos procesos por separado no garantiza que la llamada funcione: debe resolverse el destino, establecerse la conexión y respetarse el contrato acordado. Por eso los tests unitarios de cada servicio y los tests de integración entre ellos tienen propósitos diferentes. En los módulos posteriores la solución incorporará rutas HTTP, mensajería y un gateway; aquí basta con comprender que la independencia solo es valiosa cuando puede verificarse en despliegues y comportamientos reales.

```
// AceriaApp.Ordenes.Api/Program.cs
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<OrdenesDbContext>(opt =>
    opt.UseSqlServer(builder.Configuration.GetConnectionString("OrdenesDb")));
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
app.MapGet("/api/v1/ordenes", async (OrdenesDbContext db) =>
    await db.Ordenes.ToListAsync());
app.Run();
```

El fragmento reduce un servicio a su arranque ASP.NET Core, el registro de un contexto EF y una ruta GET. `UseSqlServer` vincula el contexto a una configuración que el proceso deberá recibir; `MapGet` establece la operación visible para el consumidor. Faltan las clases del dominio y otros elementos necesarios para ejecutarlo como aplicación autónoma: aquí interesa reconocer la forma que adoptaría un proceso de Órdenes independiente.

Este archivo Program.cs pertenece exclusivamente al servicio de órdenes: el servicio de inventario tendrá su propio Program.cs, su propio DbContext y su propia cadena de conexión. La observabilidad en microservicios requiere correlación de trazas entre servicios, porque una petición puede atravesar varios procesos. El versionado es independiente: cada servicio puede evolucionar a su ritmo siempre

que respete los contratos públicos. La consistencia se vuelve eventual y exige patrones como Saga, Outbox o compensación explícita. La comunicación entre servicios debe diseñarse con tolerancia a fallos desde el primer momento.

El ejemplo de `OrdenesDbContext` ilustra el aislamiento de datos, pero no significa que cada servicio necesite siempre un servidor SQL distinto. Puede disponer de una base de datos independiente en una misma instancia administrativa, con permisos que impidan acceder a los esquemas ajenos. Lo esencial es la propiedad exclusiva de su modelo y la ausencia de lecturas y escrituras directas desde otros servicios. Esta independencia obliga a expresar necesidades interdominio mediante una API o eventos. Si Inventario publica información de disponibilidad, Órdenes debe gestionar que la respuesta llegue tarde o cambie después de la consulta. También se debe distinguir el descubrimiento de endpoints de la propiedad de datos: el Gateway simplifica la entrada de clientes, pero no convierte automáticamente en autónomas las bases de datos. Los tests de un servicio deben poder ejecutarse sin modificar tablas propiedad de otro.

Separar Órdenes e Inventario en dos aplicaciones ASP.NET Core cambia las condiciones de error del sistema. En el monolito, una llamada local puede fallar por una excepción y un acceso a datos puede fallar por la base de datos. Cuando Órdenes consulta Inventario mediante HTTP aparecen nuevos estados: el nombre del servicio puede no resolverse, la conexión puede rechazarse, la petición puede expirar, el servidor puede devolver 503 o la respuesta puede tener una estructura incompatible. Todos esos resultados son posibles incluso cuando ambos servicios compilan correctamente. Por eso la independencia de despliegue debe acompañarse de contratos y tratamiento explícito del fallo remoto.

Consideremos una orden de fabricación que necesita verificar disponibilidad de chapa. Una petición para crear la orden llega correctamente al servicio de Órdenes, pero Inventario tarda más del límite configurado. Si el servicio repite la petición sin comprobar qué operación ha fallado, puede añadir carga al sistema precisamente cuando Inventario está degradado. Si, por el contrario, la operación exige confirmar una reserva y la reserva se realiza sin respuesta, Órdenes no puede deducir con certeza si la reserva tuvo éxito. Aparece una incertidumbre que no existe en una llamada local simple: el resultado de la operación remota y la respuesta recibida no siempre coinciden. Este hecho prepara el estudio posterior de idempotencia, mensajería y coordinación.

Una frontera distribuida correcta debe responder a tres comprobaciones: quién es propietario de los datos, qué contrato reconoce el consumidor y qué hace el consumidor cuando el productor no responde. No basta con probar el camino satisfactorio. Aunque los primeros proyectos AceriaApp aún se ejecuten como monolito modular, su documentación puede describir esos límites futuros sin añadir dependencias remotas prematuras. El alumno debe comprender que distribuir servicios aumenta la autonomía potencial y, al mismo tiempo, obliga a diseñar explícitamente los fallos que antes quedaban dentro de un mismo proceso.

Ventajas de los microservicios

La libertad para publicar Inventario sin recompilar Órdenes puede acortar el tiempo entre corregir una regla y ponerla a disposición del usuario. También ofrece una ventaja de escalado: si aumentan las consultas de stock, se pueden ampliar las instancias que atienden esa demanda, en vez de replicar indiscriminadamente todo el sistema. Para un equipo con varios responsables funcionales, esa separación puede disminuir conflictos y permitir ciclos de trabajo distintos. Pero ninguna de estas ventajas nace automáticamente al colocar varias APIs en carpetas diferentes. Hace falta comprobar que sus contratos no obligan a cambiar todos los servicios en cada despliegue, que los datos tienen propietarios claros y que la infraestructura puede atender las nuevas dependencias. Una arquitectura de microservicios que requiere publicar cinco procesos juntos para corregir una propiedad apenas ha ganado independencia.

```
Escalado selectivo:
```

```
[Órdenes]    3 instancias  ← recibe el 70% del tráfico
[Inventario] 1 instancia   ← recibe el 20% del tráfico
[Clientes]   1 instancia   ← recibe el 10% del tráfico

Cada servicio se escala según su carga real.
```

Las proporciones de tráfico son hipotéticas y facilitan comparar el despliegue de cada servicio. Con el 70 % de las peticiones dirigido a Órdenes, el ejemplo le asigna tres instancias y mantiene una para Inventario y Clientes. En un sistema real habría que medir utilización, latencia y coste por petición, no dimensionar procesos únicamente por porcentaje de tráfico. El beneficio arquitectónico es poder tomar esa decisión por capacidad.

La quinta ventaja es la alineación con equipos autónomos. Cada equipo posee un servicio completo, desde la base de datos hasta el endpoint, y puede tomar decisiones sin coordinación con otros equipos. Esta alineación reduce los cuellos de botella organizativos y acelera la entrega. La sexta ventaja es la posibilidad de adoptar nuevas tecnologías de forma incremental: un servicio puede migrar a una versión superior del runtime sin afectar a los demás. La séptima ventaja es la resiliencia: el aislamiento de fallos permite diseñar degradaciones controladas.

La independencia de despliegue solo es una ventaja efectiva si los contratos permiten actualizar el productor sin romper consumidores antiguos. Un servicio de inventario con API estable puede cambiar su estructura interna y publicarse varias veces mientras Órdenes permanece en la versión anterior. Sin embargo, el aislamiento de fallos no implica disponibilidad universal: si crear una orden requiere reservar material de forma síncrona, la caída de Inventario todavía puede bloquear esa operación. La resiliencia consiste en decidir qué funciones deben detenerse, cuáles pueden continuar y qué información se puede recuperar después. Los beneficios se comprueban mediante métricas separadas por servicio, incluyendo duración del despliegue, tasa de errores y coste de las réplicas. También debe medirse el trabajo operacional necesario para certificados, configuración y alertas, pues puede superar el ahorro aparente del escalado selectivo.

Desafíos y costes de los microservicios

La distribución convierte los fallos de comunicación en parte normal del diseño. Una petición remota puede tardar demasiado, llegar a su destino cuando el cliente ya ha desistido o devolver un error después de haber ejecutado parte del trabajo. El servidor necesita presupuestos temporales, los clientes deben interpretar correctamente los estados de respuesta y los reintentos no pueden añadirse indiscriminadamente. En una operación de lectura suele resultar razonable volver a consultar ante un fallo transitorio; repetir una creación sin una garantía de idempotencia puede generar duplicados. Tampoco hay una transacción local que abarque de manera natural bases pertenecientes a servicios distintos; coordinar cambios requiere contratos, mensajes, compensaciones o decisiones explícitas sobre consistencia. Por último, el diagnóstico deja de realizarse sobre un solo proceso: hay que correlacionar eventos, peticiones y tiempos. El coste operativo de estas medidas debe entrar en la decisión arquitectónica.

```
var policy = Policy
    .Handle<HttpRequestException>()
    .WaitAndRetryAsync(3, attempt => TimeSpan.FromMilliseconds(200 * attempt));
var response = await policy.ExecuteAsync(() =>
    httpClient.GetAsync("http://acería-inventario/api/v1/materiales"));
```

La política de Polly muestra una reacción ante `HttpRequestException`, pero no contempla por sí sola todas las respuestas HTTP de error. Además, la espera `200 * attempt` crece **linealmente**: 200, 400 y 600 milisegundos; no es un retroceso exponencial. Conviene leer el ejemplo como una introducción a los reintentos, no como una configuración apta para producción. Cuando se estudie resiliencia habrá que limitar la duración total y comprobar qué operaciones pueden repetirse sin duplicar efectos.

La política anterior es necesaria porque en microservicios toda comunicación remota es falible por definición. El cuarto desafío es el coste operativo. Cada servicio requiere su propio pipeline, su propia monitorización y su propia configuración. La infraestructura mínima (contenedores, orquestación, descubrimiento de servicios) añade complejidad que un monolito no necesita. El quinto desafío es la duplicación de código: sin disciplina, cada equipo reimplementa utilidades comunes, lo que fragmenta las convenciones y multiplica los errores. El sexto desafío es la gestión de versiones de los contratos públicos entre servicios.

Las políticas de reintento y de circuit breaker requieren evaluar la semántica de cada operación. Reintentar un GET suele ser menos arriesgado que repetir un POST que crea un recurso, y hasta un GET puede tener efectos observables si modifica contadores o dispara procesos. El ejemplo del reintento lineal `200 * attempt` debe identificarse como espera creciente lineal, no exponencial; un retroceso exponencial necesita una función como `200 * 2^(attempt - 1)`. Los fallos de HTTP no se reducen a `HttpRequestException`: un servidor puede devolver un estado de error sin que `GetAsync` lance esa excepción. Por ello el manejo debe contemplar códigos de respuesta, cancelación y límites de tiempo. Al diseñar las políticas, la duración total esperada de todos los intentos debe mantenerse dentro del presupuesto de latencia del cliente.

En una cadena de llamadas síncronas el tiempo total de una operación depende del servicio más lento, de las conexiones utilizadas y del trabajo que se ejecuta en cada salto. Si una petición del Gateway recorre Órdenes, Inventario y Clientes de forma secuencial, las esperas se acumulan. Una operación que parece barata vista desde cada servicio puede superar el presupuesto de respuesta del cliente cuando se ejecuta como flujo completo. Esta observación no significa que todas las llamadas deban hacerse en paralelo: algunas dependen de resultados previos y otras modifican el estado. Significa que el diseño necesita una vista del recorrido completo de la petición, no una optimización aislada de cada endpoint.

El patrón de reintento modifica además la presión sobre los servicios. Un cliente que intenta una petición cuatro veces puede multiplicar su carga. Si varios componentes de la cadena aplican a su vez cuatro intentos, el número potencial de solicitudes hacia el servicio más profundo crece rápidamente. Este fenómeno, denominado amplificación por reintentos, demuestra por qué no debe añadirse una política uniforme sin comprender la topología. Los límites de concurrencia, la espera entre intentos y el presupuesto temporal de la operación deben coordinarse. Una consulta GET sin efectos secundarios admite reintentos con menos riesgos que una orden POST que pueda duplicar una acción de negocio.

En AceriaApp, una verificación de resiliencia futura no debería limitarse a observar que Polly ha sido configurado. Debería simular una dependencia que responde tarde o con error, ejecutar una petición real, contar las llamadas efectuadas, medir la duración y comprobar el código HTTP final. También debe comprobar qué registra el sistema: sin un identificador de correlación, varias respuestas y reintentos pueden resultar indistinguibles en los logs. Estos criterios ofrecen al alumno una conexión precisa entre arquitectura, operación y evidencia verificable; en el módulo inicial se explican, pero no se presenta una política distribuida como si ya estuviera implementada.

Comparativa y criterios de decisión

La pregunta que debe resolver una elección de arquitectura no es «¿qué estilo es más moderno?», sino **qué problemas resuelve cada frontera y cuánto cuesta mantenerla**. Un equipo pequeño que desarrolla un dominio todavía cambiante puede beneficiarse de un monolito modular: las reglas se clarifican con rapidez y permanecen disponibles transacciones locales. En cambio, cuando una capacidad tiene ritmos de publicación o carga muy diferentes, extraerla puede reducir dependencias de despliegue. Para comparar alternativas es útil registrar frecuencia de cambios, tiempos de regresión, incidencias, coste de infraestructura, volumen de llamadas y requisitos de consistencia. Son indicadores que deben interpretarse conjuntamente. Una cifra aislada —por ejemplo, el número de equipos o de tablas— no determina por sí sola la arquitectura apropiada. La tabla del apartado condensa criterios iniciales, no una ley universal.

```
Criterio                  | Monolito            | Microservicios
--------------------------|---------------------|---------------------
Tamaño del dominio        | Pequeño/mediano     | Grande
Número de equipos         | 1-2                 | 3 o más
Frecuencia de despliegue  | Baja/media          | Alta
Consistencia requerida    | Fuerte (ACID)       | Eventual aceptable
Complejidad operativa     | Baja                | Alta
Coste de infraestructura  | Bajo                | Medio/alto
```

La comparación sintetiza condiciones frecuentes, pero las columnas no deben interpretarse como reglas excluyentes. Existen monolitos que atienden grandes dominios y microservicios que necesitan consistencia fuerte dentro de su propia base. La decisión depende de cómo se agrupan las transacciones, de quién controla los cambios y de cuánta complejidad operativa está preparada para asumir la organización.

La regla práctica es empezar con un monolito modular y extraer microservicios solo cuando un criterio objetivo lo justifique. Los criterios anteriores no son mutuamente excluyentes. Un sistema puede tener un monolito para el dominio administrativo y microservicios para el dominio transaccional. La arquitectura evoluciona con el negocio y con la organización, y la decisión debe revisarse periódicamente. La inversión en microservicios solo se recupera cuando la independencia de despliegue y el escalado selectivo aportan un valor medible.

Los umbrales comparativos de equipos, costes o tamaños de dominio no deben interpretarse como reglas universales. Dos equipos pueden necesitar desplegar con independencia si gestionan productos sometidos a obligaciones regulatorias distintas; cuatro equipos pueden colaborar eficientemente sobre un monolito modular si el dominio cambia de manera coordinada. La decisión necesita mediciones históricas: cuántos despliegues por semana se bloquean entre sí, qué porcentaje de recursos consume cada capacidad y cuánto tarda en localizarse un fallo. También interesa saber si una transacción atraviesa varias funciones de negocio, porque extraerlas puede sustituir garantías ACID por un proceso distribuido explícito. El cuadro es un instrumento inicial para formular hipótesis. Una buena decisión registra el contexto y las consecuencias y señala indicadores que, si cambian, justificarán revisar el estilo arquitectónico.

Los criterios de comparación son útiles cuando se traducen a señales que el equipo pueda verificar. El número de personas, por sí solo, no obliga a distribuir; tampoco lo hace el tamaño del repositorio. Un equipo pequeño puede operar una arquitectura distribuida por exigencias de aislamiento o escala, y uno grande puede mantener un monolito bien modularizado. La pregunta decisiva es qué problema se pretende resolver y qué coste nuevo introduce la solución. Para AceriaApp, el problema inicial consiste en construir y validar reglas de producción con persistencia relacional y una API comprensible. No existe todavía evidencia de que el despliegue único impida entregar cambios ni de que sea rentable sostener varios servicios independientes.

Señal observable	Evidencia que debe reunirse	Interpretación posible
Despliegues bloqueados	Cambios que esperan versiones de otras capacidades	Revisar acoplamiento del ciclo de vida
Carga desigual	Métricas por endpoint y consumo de recursos	Valorar escalado selectivo
Cambios coordinados	Pull requests que tocan varias capacidades siempre juntas	Revisar límites de dominio
Incidentes cruzados	Errores de una capacidad que detienen otras	Revisar aislamiento de fallos
Transacciones conjuntas	Operaciones que deben confirmarse de modo indivisible	Mantener un límite transaccional claro

Estos indicadores pueden sugerir decisiones diferentes. Por ejemplo, una carga desigual puede resolverse optimizando la consulta o añadiendo caché antes de separar una API. Una organización lenta puede mejorar mediante propiedad clara de módulos sin necesidad de multiplicar repositorios. Y una necesidad estricta de consistencia puede aconsejar mantener ciertas operaciones dentro de un mismo servicio incluso cuando otras se distribuyen. La arquitectura resultante no tiene por qué ser puramente monolítica o puramente distribuida.

La decisión de comenzar con el monolito modular debe expresarse en un ADR con contexto, alternativas, consecuencias y condiciones de revisión. Registrar solamente «se elige monolito porque el equipo es pequeño» sería insuficiente: también hay que explicar qué observaciones justificarían revisar la decisión. Ese razonamiento fundamenta el ADR-001 de la práctica y enseña al alumno a reconocer que una elección arquitectónica es una hipótesis técnica susceptible de comprobarse, no una preferencia permanente.

Evolución y contexto del proyecto del curso

El itinerario de AceriaApp está pensado para hacer visible el coste de cada decisión antes de introducir la siguiente. En el módulo inicial se puede reconocer un monolito modular, seguir una orden y comprobar su persistencia. Después se diseñan recursos HTTP, se separan casos de uso y adaptadores, se introducen patrones, y el modelo del dominio gana comportamiento. Solo cuando existe esa base se incorporan Producción e Inventario como procesos que necesitan comunicación y observabilidad, hasta situarlos detrás de YARP. El módulo práctico de Docker completa el recorrido desplegando juntos los componentes ejecutables. Esta secuencia permite atribuir a cada cambio sus consecuencias: quién conoce a quién, qué se compila, qué se publica y qué prueba demuestra el funcionamiento. La meta del curso no es obligar a utilizar microservicios, sino aprender a decidir cuándo su coste está justificado.

```
Evolución del proyecto:

M1: Monolito modular de referencia
M2: + API REST de órdenes (Minimal API + OpenAPI)
M3: + Arquitectura hexagonal (puertos y adaptadores)
M4: + Patrones (Repository, UoW, MediatR, Result)
M5: + DDD y CQRS (entidades, VOs, commands, queries)
M6: + Comunicación HTTP y eventos entre servicios
M7: + Observabilidad (Serilog, health checks) y Polly
M8: + API Gateway con YARP (enrutamiento y balanceo)
```

La secuencia del curso es acumulativa: la API inicial permite observar HTTP y SQL, la arquitectura hexagonal delimita responsabilidades, DDD y CQRS aclaran el comportamiento, y los módulos posteriores hacen visibles las consecuencias de comunicar procesos distintos. Los nombres del esquema describen esa evolución pedagógica; cada etapa debe contrastarse con su código y sus pruebas, sin atribuir al primer módulo funcionalidades que todavía no existen.

Cada módulo añade código nuevo sin sustituir el anterior, de modo que la solución final conserva todas las versiones y permite comparar la evolución. El resultado final es una plataforma de microservicios funcional, con persistencia en SQL Server Express, pruebas automatizadas, observabilidad y resiliencia. El alumno puede abrir cualquier carpeta de punto y ejecutarla de forma independiente, y puede comparar dos carpetas consecutivas con diff -r para ver exactamente qué se ha añadido. El proyecto es acumulativo y no opcional: cada punto hace referencia explícita a cómo lo aprendido se integra en el proyecto creciente.

El recorrido de los ocho módulos no debe confundirse con una obligación de convertir cada capacidad en un microservicio. El monolito inicial es una referencia ejecutable sobre la que se añaden decisiones sucesivas: primero contratos REST, después una organización de puertos y adaptadores, más tarde patrones y límites de dominio. En cada transición deben preservarse las conductas observables mediante tests y comprobarse qué dependencia se eliminó o se hizo explícita. Las soluciones

independientes por punto permiten comparar el resultado sin destruir la etapa previa. Al introducir comunicación de red en el módulo seis aparecerán nuevas causas de error y consistencia; en el siete se gestionarán mediante observabilidad y resiliencia. El Gateway final centraliza el acceso, pero no sustituye las responsabilidades de cada servicio. Esta evolución exige justificar los cambios a partir de necesidades, no de la disponibilidad de una biblioteca.

Caso de análisis — Seguir una orden desde HTTP hasta SQL Server

La arquitectura del punto 1.1 se entiende mejor siguiendo una petición completa que identificando carpetas por separado. El usuario de la API envía un POST a `/api/v1/ordenes` con cliente, material y cantidad. Kestrel recibe la petición y ASP.NET Core selecciona el endpoint declarado en `Program.cs`. El sistema de inyección de dependencias proporciona `AceriaDbContext`, configurado con la cadena `AceriaDb`. El endpoint completa los datos gestionados por el servidor, añade la entidad `Orden` al contexto y ejecuta SaveChangesAsync. Si la base está preparada y la escritura tiene éxito, la API responde con la ubicación del recurso creado. Cada eslabón depende de un artefacto distinto, pero todos participan en el mismo host ejecutable.

Este recorrido deja ver una diferencia fundamental entre una responsabilidad de aplicación y una de infraestructura. Decidir si una cantidad es admisible es una regla de negocio. Saber cómo convertir una entidad en una instrucción SQL es una responsabilidad de persistencia. El primer proyecto reúne ambas en una implementación mínima para que el recorrido sea visible; todavía no pretende demostrar el aislamiento completo de todas las reglas del dominio. En una evolución hacia arquitectura hexagonal, las decisiones de negocio se moverían a un núcleo con puertos definidos, manteniendo las dependencias técnicas en los adaptadores externos. Eso cambiaría la estructura interna del código, aunque no obligaría a dividir inmediatamente la API en varios procesos.

Punto de observación	Artefacto de AceriaApp	Evidencia esperada
Recepción de petición	`Program.cs`	Existe la ruta POST para órdenes
Modelo de datos	`Dominio/Orden.cs`	La entidad tiene las propiedades previstas
Conexión y mapeo	`Infraestructura/AceriaDbContext.cs`	El contexto expone `Ordenes`
Esquema físico	Migración EF Core	La tabla está creada en SQL Server Express
Persistencia	Petición POST real	La orden se inserta sin error
Recuperación	Petición GET real	El recurso creado vuelve a consultarse

El diagnóstico de un fallo puede seguir exactamente la misma ruta. Si el POST devuelve un error de conexión, primero se verifica el nombre de la instancia SQL y la disponibilidad del servidor. Si devuelve una excepción de tabla inexistente, se examina si se ejecutó la migración. Si la ruta no aparece en Swagger, se revisa la inicialización del host y el entorno de desarrollo. Son problemas distintos y conviene no atribuirlos genéricamente a «microservicios» o «arquitectura»: pertenecen a configuración, persistencia y exposición HTTP. La utilidad de la solución mínima es que esas fronteras pueden identificarse con claridad.

También debe separarse la evidencia automatizada. El test inicial `Orden_ConDatosValidos_SeCreaCorrectamente` verifica propiedades de un objeto en memoria; no necesita arrancar Kestrel ni consultar SQL. El escenario E2E sí necesita iniciar el host, preparar la base y ejecutar peticiones reales. Una prueba no sustituye a la otra porque protegen riesgos diferentes. Esta distinción, aprendida en el primer punto, se aplicará a métricas, límites de servicio, convenciones

y buenas prácticas: cada componente nuevo necesita un resultado observable apropiado a lo que afirma resolver.

Finalmente, el caso permite explicar por qué se parte del monolito. La ruta, el modelo y la persistencia son suficientes para recorrer una operación de negocio completa. Introducir otro proceso y una llamada HTTP interna no añadiría todavía una capacidad funcional; añadiría un nuevo límite operativo que necesitaría políticas de error, contratos y seguimiento. La futura distribución tendrá sentido cuando la separación resuelva un problema observado y cuando el equipo disponga de pruebas que demuestren que el comportamiento funcional sigue siendo correcto.

Síntesis para continuar

La primera decisión importante consiste en separar dos preguntas que suelen confundirse: **cómo organizamos el código** y **qué publicamos como proceso independiente**. Un monolito puede tener límites internos de calidad; unos microservicios pueden estar mal delimitados y funcionar como un monolito distribuido. El primer estilo simplifica las comunicaciones y las transacciones locales; el segundo puede aportar autonomía de publicación y escalado selectivo, a cambio de introducir latencia, fallos parciales y coordinación entre datos. En AceriaApp comenzamos por el modelo más sencillo que permite explicar y probar una operación de negocio. Los módulos siguientes modificarán esa frontera cuando la necesidad y la evidencia técnica lo justifiquen.

Punto 1.2 — Ventajas e inconvenientes

Módulo 1: Introducción a los microservicios

Objetivos de aprendizaje:

- Identificar las ventajas concretas de una arquitectura monolítica y de una arquitectura de microservicios.
- Identificar los inconvenientes concretos de cada estilo arquitectónico.
- Relacionar cada ventaja y cada inconveniente con criterios técnicos medibles.
- Reconocer los costes ocultos de la distribución y los costes ocultos de la centralización.
- Añadir al proyecto AceriaApp un documento de análisis comparativo en docs/ y una clase de cálculo de métricas de acoplamiento en el proyecto monolítico.

Comparar consecuencias, no etiquetas

La arquitectura se evalúa mejor cuando se toma una misma necesidad y se analiza qué exige cada alternativa. Seguiremos el ciclo de vida de una orden de AceriaApp y compararemos quién publica los cambios, cómo se mantienen sus datos y qué costes aparecen al aumentar el tráfico o la cantidad de equipos. Esta perspectiva permite distinguir ventajas reales de promesas que solo se cumplirían después de invertir en infraestructura y operación.

Marco de análisis de ventajas e inconvenientes

Para comparar arquitecturas necesitamos observar la misma operación desde varios ángulos. Pensemos en una orden de producción: alguien la crea, el sistema comprueba sus datos, guarda el resultado y, eventualmente, consulta el inventario. El diseño será adecuado si esa secuencia cumple sus requisitos de tiempo de respuesta, integridad y evolución, con un esfuerzo asumible para el equipo. Por eso conviene distinguir al menos cuatro dimensiones: cuánto cuesta publicar cambios, qué recursos requiere responder a una carga concreta, cuánta coordinación necesita el equipo y qué consistencia deben mantener los datos. Ninguna dimensión es suficiente por sí sola. Una opción puede simplificar una transacción y complicar el crecimiento; otra puede facilitar despliegues independientes y aumentar el tiempo de diagnóstico. En AceriaApp compararemos ambos enfoques sobre decisiones observables, no sobre etiquetas arquitectónicas.

```
Dimensiones del marco de análisis:

1. Despliegue    → tiempo de fusión a producción.
2. Escalado      → coste por unidad de carga.
3. Organización  → equipos que trabajan sin coordinación.
4. Consistencia  → garantía transaccional requerida.
```

El esquema enumera las cuatro dimensiones que guiarán la comparación. El tiempo entre fusión y despliegue no debe interpretarse sin saber cuántas pruebas exige el cambio; el coste por carga necesita mediciones; la independencia de equipos depende de contratos estables; y la consistencia requiere identificar exactamente qué operaciones han de confirmarse juntas. Se analizarán en conjunto porque una decisión favorable en una dimensión puede tener costes relevantes en otra.

Este marco se aplica a cada decisión arquitectónica del curso y permite justificar por qué el proyecto empieza como monolito modular y evoluciona hacia microservicios. La primera dimensión, el despliegue, es la que más peso tiene en equipos pequeños porque determina la frecuencia con la que se puede entregar valor al negocio. La segunda dimensión, el escalado, es la que más peso tiene en sistemas con cargas muy desiguales entre funcionalidades. El tiempo de entrega puede calcularse

desde la fusión hasta la disponibilidad en producción, separando espera en cola y ejecución del pipeline. La consistencia debe describirse mediante invariantes de negocio: una orden no puede reservar más unidades de material que las disponibles, por ejemplo.

La tercera dimensión, la organización, es la que más peso tiene en empresas con varios equipos trabajando en paralelo sobre el mismo producto. La cuarta dimensión, la consistencia, es la que más peso tiene en dominios financieros o regulados donde la integridad transaccional no es negociable. Ninguna dimensión es universalmente más importante que las demás: el peso relativo depende del contexto del negocio. Lo que sí es universal es la necesidad de medir cada dimensión con indicadores concretos antes de decidir. El marco se completa con una quinta consideración transversal: el coste operativo total, que agrega el esfuerzo de monitorización, configuración y soporte. El escalado puede expresarse como euros por mil peticiones completadas bajo una latencia objetivo, para evitar comparar máquinas con cargas diferentes. Sin definir estas unidades, los resultados de la comparación no son reproducibles.

El coste operativo es una dimensión transversal que incluye guardias, alertas, despliegues, administración de secretos y diagnóstico de incidencias, y no se deduce únicamente de la factura del proveedor de infraestructura.

Una comparación arquitectónica requiere establecer primero una unidad de observación. Para despliegue puede utilizarse el intervalo entre la aprobación de un cambio y su disponibilidad en el entorno de destino. Para escalado, el coste puede normalizarse por petición atendida o por carga sostenida. En organización interesa la proporción de cambios que requieren coordinación entre equipos, no únicamente la cantidad de desarrolladores. En consistencia deben enumerarse las operaciones de negocio que exigen una confirmación atómica. Si estas definiciones cambian entre alternativas, la comparación deja de ser válida: un monolito medido bajo carga real no debe contrastarse con microservicios medidos sin tráfico ni supervisión.

Conviene distinguir también indicadores directos y aproximaciones. La cantidad de proyectos .NET es fácil de contar, pero no mide por sí sola el acoplamiento entre capacidades. El número de endpoints tampoco determina automáticamente la complejidad. Un endpoint que coordina movimientos, reservas e incidencias puede presentar más riesgos que diez consultas independientes. Las métricas construidas en este módulo permiten practicar el cálculo de ratios y el filtrado de candidatos, pero deben entenderse como indicadores pedagógicos iniciales. Para usarlas en una decisión empresarial habría que vincular cada referencia a una dependencia real y cada tabla compartida a una titularidad de datos documentada.

Ejemplo de ficha de evaluación: una operación de Órdenes cambia dos tablas, se despliega junto al resto de la API y no realiza llamadas de red entre módulos. La ficha puede registrar ese comportamiento como línea base. Si se propone extraer Inventario, se volverá a medir la misma operación y se anotarán la latencia, los errores remotos y el procedimiento de recuperación. La nueva arquitectura no se considerará mejor solo porque aumente la independencia de publicación. Debe mejorar el indicador que motivó el cambio sin introducir un coste inasumible en las demás dimensiones. Este método transforma la matriz teórica en una herramienta de decisión reproducible.

Ventajas del monolito en la dimensión de despliegue

Publicar una sola aplicación ofrece una ventaja muy concreta a quien mantiene un sistema pequeño: existe una cadena de compilación, pruebas y publicación que entrega el conjunto como una unidad. El responsable del despliegue sabe qué versión está instalada y puede revertirla sin construir una matriz de compatibilidad entre servicios. En una acería con un equipo reducido, esa simplicidad permite dedicar más esfuerzo a entender las reglas de las órdenes que a sostener infraestructura. La contrapartida es que un cambio localizado recorre también pruebas y pasos de distribución compartidos por funcionalidades que no se han modificado. Esa relación entre rapidez inicial y coste de los cambios frecuentes es más importante que contar cuántos comandos contiene el pipeline. La

medición debe incluir el tiempo de revisión y recuperación de incidentes, no solo la ejecución automática.

```
# Pipeline único del monolito
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - dotnet build AceriaApp.sln

test:
  stage: test
  script:
    - dotnet test AceriaApp.sln

deploy:
  stage: deploy
  script:
    - dotnet publish src/AceriaApp.Monolito.Api -c Release
```

El YAML ilustra un pipeline único que compila, prueba y publica el monolito. `dotnet build` y `dotnet test` se ejecutan sobre la solución común, mientras que `dotnet publish` prepara el artefacto del host. Es un ejemplo didáctico de secuencia, no una configuración completa de un servidor CI: faltan elementos como artefactos, credenciales, ejecución por entorno y comprobaciones de despliegue.

La ventaja del pipeline único se materializa en equipos pequeños, donde una sola persona puede mantener el pipeline sin dedicación exclusiva. La desventaja aparece cuando el pipeline tarda más de lo aceptable: cada corrección de una línea paga el coste completo del pipeline. La dimensión de despliegue, por tanto, no es una ventaja absoluta del monolito sino una ventaja condicionada al tamaño del equipo y a la frecuencia de cambio.

Un pipeline único puede ejecutar comprobaciones parciales para acelerar la respuesta al desarrollador y mantener una validación global antes de desplegar. Por tanto, “recompilar todo” no significa necesariamente volver a compilar cada fichero si existen cachés incrementales; el límite que permanece compartido es el artefacto y su publicación. El ejemplo YAML describe conceptualmente las fases de construcción, pruebas y publicación, pero para ejecutarlo hay que adaptarlo al sistema de integración continua concreto. La reversión de un monolito depende además de las migraciones de base de datos: volver a la versión anterior del código puede no bastar si el esquema cambió de forma incompatible. Una estrategia segura requiere migraciones compatibles durante la transición, copias de seguridad y capacidad de restauración. El beneficio operativo se comprueba practicando un despliegue y una reversión en un entorno no productivo.

Ventajas del monolito en la dimensión de consistencia

Una orden puede implicar varias escrituras relacionadas. Imaginemos que su creación necesita guardar el encabezado y registrar un movimiento de material. Cuando ambas operaciones pertenecen al mismo ámbito de base de datos, una transacción local ofrece una garantía muy comprensible: si el conjunto falla antes de confirmarse, se revierte. Ese mecanismo permite razonar sobre invariantes del negocio sin introducir comunicación remota. Sin embargo, una transacción ACID no convierte automáticamente el modelo en correcto; únicamente protege determinadas propiedades técnicas de un conjunto de operaciones. Tampoco obliga a que toda la aplicación tenga una única base física: la ventaja depende de que las escrituras que han de ser atómicas compartan una frontera transaccional concreta. Separarlas entre servicios no hace imposible la consistencia, pero exige una estrategia distinta para coordinar sus estados.

```
await using var tx = await db.Database.BeginTransactionAsync();
try
{
    var orden = new Orden { Id = Guid.NewGuid(), Cliente = "ACX-001", Cantidad = 10 };
    db.Ordenes.Add(orden);
    var movimiento = new Movimiento { Id = Guid.NewGuid(), OrdenId = orden.Id, Material = "CHAPA-3MM" };
    db.Movimientos.Add(movimiento);
    await db.SaveChangesAsync();
    await tx.CommitAsync();
}
catch
{
    await tx.RollbackAsync();
    throw;
}
```

La transacción envuelve las escrituras de Orden y Movimiento. Al confirmar después de SaveChangesAsync, ambas quedan dentro del mismo compromiso; en el bloque `catch` se intenta revertir y se propaga el error para que el llamador pueda tratarlo. Para ejecutar exactamente este ejemplo hacen falta los tipos y `DbSet` indicados. El mecanismo interesa como garantía de atomicidad cuando ambas escrituras pertenecen al mismo proveedor y conexión.

Esta garantía transaccional es el activo más valioso del monolito en dominios regulados. Su coste es el acoplamiento: todas las funcionalidades comparten el mismo esquema y el mismo contexto de persistencia. Un cambio en el esquema afecta a todas las funcionalidades y obliga a coordinarlas en el mismo despliegue.

Una transacción ACID de SQL Server permite actualizar filas de órdenes y movimientos como una unidad siempre que ambos modelos formen parte de la misma transacción y de la misma infraestructura transaccional. No significa que cualquier cambio en memoria se revierta automáticamente: las propiedades de objetos ya modificados no se restauran por arte de magia, y los mensajes externos enviados antes del commit tampoco se deshacen. La implementación debe distinguir entre el estado de la base de datos y los efectos secundarios que ocurren fuera de ella. En AceriaApp, el ejemplo con Movimiento es ilustrativo hasta que la entidad y el DbSet correspondientes existan. La garantía se verifica con una prueba que provoque un fallo en una de las escrituras y compruebe la ausencia de cambios parciales. Este enfoque explica después la necesidad de Outbox al integrar eventos con transacciones locales.

La consistencia relacional del monolito simplifica la coordinación cuando una misma operación necesita modificar órdenes y movimientos de inventario. Una transacción local puede asegurar que ambos cambios se confirmen o reviertan. Sin embargo, esa ventaja exige definir con precisión qué recursos participan. No todas las operaciones de negocio necesitan una transacción única. La consulta del catálogo de materiales puede tolerar datos ligeramente desactualizados, mientras que la confirmación de una reserva que afecte a existencias disponibles puede necesitar garantías más estrictas. Aplicar la misma exigencia a todos los flujos produce sobrecoste innecesario y dificulta la futura división del sistema.

Una separación por servicios redefine el límite transaccional: Órdenes puede confirmar su cambio local y emitir un evento; Inventario recibe el mensaje y procesa su propia transacción. Entre ambos momentos, los datos pueden encontrarse en un estado intermedio permitido. Este comportamiento no debe presentarse como «inconsistencia sin control». Se define una máquina de estados de negocio: por ejemplo, *pendiente de reserva*, *reservada* y *rechazada*. Las operaciones posteriores deben reconocer esos estados, y el sistema debe disponer de una política de recuperación cuando un mensaje no llega o se procesa dos veces. Un patrón Outbox relaciona la actualización local con el registro del evento pendiente dentro de una misma transacción; no convierte dos bases diferentes en una transacción ACID global.

En el módulo 1 no se implementa ese mecanismo distribuido. Sí se prepara la capacidad de razonar sobre él. El indicador `IndiceComparticionDeDatos` del ejercicio pretende visibilizar cuánto dependen

unas capacidades de las tablas de otras, pero su implementación simplificada no identifica titularidad ni distingue tablas propias de compartidas. Por eso su valor debe acompañarse de una interpretación escrita: qué entidades pertenecen a Órdenes, cuáles a Inventario y qué consultas atraviesan el límite. El alumno aprende a diferenciar la propiedad transaccional comprobable en SQL Server de la independencia de datos que una futura arquitectura de microservicios debe diseñar.

Ventajas del monolito en las dimensiones de escalado y organización

Escalar un monolito significa normalmente replicar el proceso completo detrás de un balanceador. Es una solución fácil de entender: cada instancia dispone de las mismas rutas y el reparto se realiza en la entrada. Esta homogeneidad reduce la complejidad del despliegue cuando todas las capacidades utilizan recursos parecidos y el tráfico puede distribuirse uniformemente. También facilita que un equipo pequeño depure peticiones sin seguirlas por una red de servicios. Si crece solo una funcionalidad, en cambio, cada nueva réplica arrastra el resto de bibliotecas y sus consumos. El análisis real debe contemplar memoria, CPU, conexiones y coste por operación: un escalado aparentemente redundante puede seguir resultando rentable si evita una infraestructura distribuida más costosa. La simplicidad operativa también tiene valor económico.

```
Escalado del monolito:

[Balanceador]
      |
      +---> [Instancia 1] (artefacto completo)
      +---> [Instancia 2] (artefacto completo)
      +---> [Instancia 3] (artefacto completo)

Todas las instancias ejecutan el mismo artefacto.
```

El balanceador envía peticiones a tres copias del mismo host. La sencillez está en que todas ofrecen los mismos endpoints y tienen el mismo ciclo de publicación. Si una parte de la aplicación es especialmente costosa, cada réplica incluye también los componentes que quizá no reciban tráfico. La necesidad de escalar selectivamente debe demostrarse con medidas, no deducirse únicamente de este dibujo.

Las ventajas de simplicidad en escalado y organización son reales y medibles. Su límite aparece cuando el tamaño del equipo supera la capacidad de una única conversación técnica o cuando la carga entre funcionalidades es muy desigual. En ese momento, la simplicidad se convierte en rigidez y el monolito empieza a frenar la evolución del producto.

El escalado horizontal del monolito no impide ajustar recursos verticalmente ni colocar cachés delante de endpoints con alta frecuencia de lectura. Antes de fragmentar la aplicación conviene probar mejoras que no añadan distribución: optimización de consultas, índices en SQL Server, paginación y reducción de bloqueo de recursos. La coordinación organizativa también depende del diseño del código y de las reglas de revisión. Un monolito modular con responsabilidades claras puede reducir conflictos de fusión aunque varios desarrolladores trabajen en el mismo repositorio. La decisión de escalar o dividir debe responder a los cuellos de botella medidos. Si tres réplicas reducen la latencia sin elevar demasiado el coste, la separación puede no aportar valor inmediato. Las métricas relevantes incluyen uso sostenido de CPU, presión de memoria, tiempos de respuesta por ruta y cambios simultáneos en las mismas zonas del código.

Inconvenientes del monolito

El principal riesgo de una aplicación monolítica no es que todos sus módulos residan en una carpeta, sino que sus **cambios dejen de estar localizados**. Si una regla de Inventario obliga a modificar código de Órdenes, Facturación y varias pruebas de extremo a extremo, la organización interna ha

perdido autonomía. A eso se añade que los equipos comparten pipeline, planificación de publicación y, a veces, decisiones de persistencia. El efecto puede ser notable en una aplicación grande sometida a cambios frecuentes; en un proyecto pequeño apenas se percibe. Antes de dividir procesos conviene atacar las causas internas: extraer reglas de controladores, introducir interfaces entre módulos, reducir referencias y aislar pruebas. Repartir un código acoplado entre contenedores no elimina ese acoplamiento, sino que puede convertirlo en llamadas de red todavía más difíciles de modificar.

```
Coste por cambio en el monolito:

Cambio de 1 línea → recompilar todo → ejecutar todos los tests → desplegar todo

Tiempo total = tiempo del pipeline completo.
```

La secuencia representa el coste de publicar un cambio pequeño dentro de un artefacto compartido. No significa que todas las pruebas deban ejecutarse siempre en cada máquina ni que una compilación larga sea inevitable; muestra que el riesgo de integración y las obligaciones de publicación son comunes. La pregunta operativa es cuánto tarda realmente el flujo y qué fallos bloquean la entrega.

El quinto inconveniente es la adopción de nuevas tecnologías: migrar una parte del sistema obliga a migrar todo el conjunto. El sexto inconveniente es la escalabilidad organizativa: a partir de cierto número de equipos, el monolito se convierte en un cuello de botella de coordinación. Estos inconvenientes no invalidan el monolito en contextos pequeños; simplemente marcan el umbral a partir del cual su coste supera su beneficio.

Los inconvenientes deben expresarse como riesgos condicionados, no como defectos inevitables. Compartir un ciclo de publicación se vuelve problemático cuando unos cambios urgentes quedan retenidos por pruebas de funcionalidades ajenas o por permisos organizativos. La barrera tecnológica puede mitigarse con modularización y actualización gradual de dependencias; no todas las bibliotecas del monolito necesitan renovarse a la vez. Los conflictos de fusión suelen reflejar responsabilidades mal repartidas, archivos demasiado centrales o ramas prolongadas. En AceriaApp, un indicador útil es cuánto código de Orden cambia cuando se modifica una regla exclusiva de Inventario. Si el cambio obliga a tocar Program.cs, consultas de varios módulos y scripts de despliegue simultáneamente, existe acoplamiento que conviene registrar. La lista de inconvenientes prepara preguntas para las métricas, no una orden automática de extraer servicios.

El coste de cambiar una línea no equivale siempre al tiempo completo del pipeline: existen cachés de compilación, pruebas selectivas, despliegues sin interrupción y distintos modos de publicación. El inconveniente propio de un monolito es la dependencia del ciclo de vida común: incluso si se optimiza la compilación, la versión publicada incluye un conjunto de funcionalidades. En un sistema de gran tamaño, la revisión de un cambio puede verse ralentizada por incertidumbre sobre sus efectos secundarios. La medida útil combina el tiempo de entrega, la tasa de despliegues fallidos y el tiempo de recuperación. Un pipeline rápido que rompe otras capacidades con frecuencia no representa una mejora real.

Para estimar la coordinación deben analizarse cambios concretos del repositorio: cuántas carpetas se modifican, cuántos responsables deben aprobarlos y cuántas incidencias se deben a interferencias entre módulos. Una carpeta compartida por varios equipos no garantiza por sí misma acoplamiento: el código puede estar organizado mediante contratos estables. Del mismo modo, varios repositorios no eliminan dependencias si todos deben publicar una nueva versión de su API en conjunto. El verdadero problema aparece cuando un cambio de Orden obliga a ajustar el modelo de Inventario, los tests de Clientes y el esquema de Facturación sin que exista una necesidad de negocio que justifique esa propagación.

En AceriaApp, la comparación de carpetas consecutivas ofrece un primer instrumento didáctico para visualizar el cambio. El alumno puede identificar los archivos nuevos de `Metricas`, `Diseno` u `0rganizacion` y comprobar qué se conserva del punto anterior. Pero la ausencia de archivos modificados no es una prueba definitiva de buen diseño: el nuevo código podría depender

directamente de clases internas antiguas. Para valorar la evolución se necesita examinar direcciones de referencia y responsabilidades además del diff. El coste oculto de un monolito crece por interacción entre responsabilidades mal delimitadas, no simplemente por compartir un proceso. Esta distinción evita concluir que cualquier separación física resolverá problemas que nacieron en el diseño lógico.

Ventajas de los microservicios en la dimensión de despliegue

La independencia de publicación de un servicio aparece cuando su equipo puede cambiarlo, ejecutarle sus pruebas y distribuirlo sin reconstruir el resto del sistema. Esa posibilidad resulta atractiva para Inventario si su frecuencia de cambio o sus cargas difieren mucho de las de Producción. Un pipeline específico puede acortar la comprobación de una modificación, siempre que los contratos que utilizan los otros servicios sigan siendo compatibles. La ventaja se pierde cuando el despliegue exige coordinar de todos modos una nueva versión de cada consumidor. Por ello no basta con crear distintos YAML o soluciones de Visual Studio: hay que registrar dependencias, conservar pruebas de contrato y observar qué publicaciones necesitan actuar de manera sincronizada. Los ejemplos de pipelines de este apartado representan dos estrategias, no un mecanismo listo para desplegar con independencia de la infraestructura elegida.

```
# Pipeline del servicio de órdenes
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - dotnet build src/AceriaApp.Ordenes.Api

test:
  stage: test
  script:
    - dotnet test tests/AceriaApp.Ordenes.Tests

deploy:
  stage: deploy
  script:
    - dotnet publish src/AceriaApp.Ordenes.Api -c Release
```

En el pipeline independiente, las órdenes de compilación, prueba y publicación se dirigen al servicio de Órdenes, no a toda la solución. Este aislamiento puede acelerar entregas si el servicio conserva contratos compatibles con sus consumidores. En una organización real aún habrá pruebas de integración y verificación de despliegue; que el YAML nombre un único proyecto no demuestra por sí solo independencia funcional.

La ventaja de la independencia de despliegue se materializa en equipos que necesitan liberar con frecuencia y sin coordinar con otros equipos. Su coste es la duplicación de configuración y la necesidad de gestionar versiones de contratos públicos entre servicios.

Un servicio con pipeline propio necesita controles que garanticen la compatibilidad de su API publicada. La independencia se pierde si Órdenes e Inventario deben desplegarse en una secuencia exacta para que las llamadas sigan funcionando. Por ello los cambios de contrato suelen introducir primero nuevos campos o rutas compatibles, permitir la migración de consumidores y retirar después lo obsoleto. El pipeline de ejemplo debe completarse con restauración de paquetes, pruebas del contrato, construcción del artefacto y validaciones de seguridad antes de producción. La aplicación no queda protegida únicamente porque el pipeline de cada servicio sea corto. En AceriaApp, el beneficio se mediría comparando la frecuencia real de publicación de Órdenes y el número de cambios de Inventario que requieren coordinar. Esa medida sirve para distinguir autonomía real de mera separación de repositorios.

Ventajas de los microservicios en las dimensiones de escalado y organización

El escalado selectivo es útil cuando las capacidades soportan demandas distintas. Una campaña puede multiplicar las consultas de material disponible mientras la creación de órdenes se mantiene estable; si cada función es un proceso autónomo, podemos asignar más recursos a Inventario sin replicar obligatoriamente Producción. También pueden asignarse responsables que comprendan sus invariantes y publiquen cambios con ritmos propios. Ahora bien, esas ventajas se compran con sistemas de monitorización, autenticación, configuración y recuperación para varios procesos. La eficiencia no se deduce únicamente de tener menos réplicas: hay que comparar el coste de esas réplicas con el coste fijo de coordinar servicios. En el laboratorio de AceriaApp se estudiará el reparto real de solicitudes, pero no se afirmará que cada caso de negocio necesite un microservicio.

```
Escalado selectivo:

[Órdenes]    3 instancias  ← alta demanda
[Inventario] 1 instancia   ← demanda media
[Clientes]   1 instancia   ← demanda baja

Cada servicio se replica según su propia carga.
```

El ejemplo asigna más réplicas a Órdenes porque se supone una demanda mayor. El número real debe deducirse de métricas de carga, latencia y utilización, no de una proporción fija. En paralelo, Inventario y Clientes conservan recursos propios. El esquema permite visualizar la decisión de escalar por capacidad, pero no estima el gasto que añade mantener esas capacidades comunicadas.

La selectividad y la autonomía son ventajas medibles cuando el número de equipos y la desigualdad de carga lo justifican. Su coste aparece en la operación: cada servicio requiere su propia monitorización, su propia configuración y su propio soporte. El coste operativo agregado puede superar el ahorro de infraestructura si no se invierte en plataformas comunes de observabilidad y despliegue.

La selectividad del escalado reduce recursos infrautilizados solo si el coste fijo de cada servicio no supera el ahorro obtenido. Incluso una instancia de una API de bajo tráfico necesita memoria base, configuración, monitorización y mantenimiento. La autonomía organizativa no exige necesariamente un repositorio por equipo: un monorepo puede contener servicios con políticas de revisión y pipelines separados. La evaluación debe evitar comparar un monolito sin observabilidad con microservicios dotados de una plataforma madura, porque la diferencia observada sería atribuible a la inversión operativa. Para AceriaApp conviene calcular el coste de una réplica completa frente al coste de tres réplicas del servicio de Órdenes más una de Inventario. También debe comprobarse la carga adicional sobre SQL Server y el balanceador, ya que un cambio en el número de instancias no incrementa automáticamente la capacidad de la base de datos.

La posibilidad de replicar Órdenes sin replicar Inventario es una capacidad técnica de los microservicios, pero su rentabilidad depende del perfil de recursos de cada uno. Imaginemos una carga en la que Órdenes necesita tres réplicas y los otros dos servicios solo una. En un monolito, tres réplicas del conjunto podrían ser suficientes; en una arquitectura distribuida se mantendrían cinco procesos separados. La segunda opción solo supone un ahorro claro si las réplicas adicionales del monolito consumen recursos significativamente mayores que los procesos del sistema distribuido. También hay que incluir balanceadores, observabilidad, configuración, copias de seguridad y capacidad de recuperación.

Coste a examinar	Pregunta útil
Memoria residente	¿Cuánto consume cada host en reposo y con carga?

Coste a examinar	Pregunta útil
CPU por operación	¿Qué capacidad recibe realmente el tráfico?
Red interna	¿Cuántas llamadas adicionales genera una orden?
Gestión operativa	¿Quién actualiza, monitoriza y recupera cada servicio?
Disponibilidad	¿Qué redundancia necesita cada capacidad crítica?

Un cálculo didáctico puede comparar escenarios con los mismos requisitos de disponibilidad y el mismo volumen de operaciones. No deben mezclarse réplicas de alta disponibilidad con réplicas de escalado, pues responden a problemas diferentes. Tampoco se deben comparar solo los costes de las máquinas: las horas destinadas a mantener pipelines, certificados y alertas forman parte del coste total de propiedad. En equipos pequeños, ese componente humano puede ser el dominante.

La autonomía organizativa es igualmente condicional. Varios equipos pueden publicar servicios por separado, pero una nueva regla transversal de negocio todavía exigirá acordar semántica y compatibilidad. Si cada equipo decide nombres de campos y códigos de error sin convenciones comunes, se reduce el acoplamiento de repositorio a cambio de aumentar el acoplamiento de integración. El diseño debe equilibrar ambos costes mediante contratos públicos explícitos, políticas de versión y una plataforma común mínima. Esa es la razón por la que AceriaApp primero formaliza criterios y convenciones antes de introducir una topología distribuida.

Inconvenientes de los microservicios

Una arquitectura distribuida introduce posibilidades de fallo que no existen en una llamada local. Inventario puede estar funcionando y, sin embargo, ser inaccesible desde Producción por DNS, red, certificados o saturación de conexiones. Una respuesta tardía puede llegar cuando el cliente ya ha agotado su tiempo de espera. Si una operación abarca varias bases, cada servicio confirma o revierte su propia transacción y la coordinación del negocio no se resuelve automáticamente. Para evaluar el coste hace falta contemplar políticas de timeout, reintento, idempotencia, observabilidad, versiones de contrato y resolución de incidentes. La ventaja de disponer de procesos independientes sigue siendo real, pero sus consumidores deben aceptar explícitamente esa incertidumbre. Un test unitario de la clase que construye una URL no demuestra la disponibilidad de un servicio remoto.

```
try
{
    var response = await httpClient.GetAsync("http://acería-inventario/api/v1/materiales");
    response.EnsureSuccessStatusCode();
    var materiales = await response.Content.ReadFromJsonAsync<List<Material>>();
    return materiales;
}
catch (HttpRequestException ex)
{
    logger.LogError(ex, "Fallo al consultar el servicio de inventario");
    throw;
}
```

La llamada HTTP se ejecuta dentro de un `try` y `EnsureSuccessStatusCode` convierte una respuesta no satisfactoria en excepción. El `catch` registra la incidencia y vuelve a propagarla, de modo que no convierte automáticamente el fallo de Inventario en una consulta válida. El ejemplo no incorpora todavía cancelación, timeout ni una respuesta degradada: se utilizará para identificar precisamente las piezas que habría que añadir.

El cuarto inconveniente es el coste operativo: cada servicio requiere su propio pipeline, su propia monitorización y su propia configuración. La infraestructura mínima añade complejidad que un monolito no necesita. El quinto inconveniente es la duplicación de código: sin disciplina, cada equipo

reimplementa utilidades comunes. El sexto inconveniente es la gestión de versiones de contratos públicos entre servicios. El séptimo inconveniente es la latencia: una operación que en un monolito era una llamada en memoria se convierte en una cadena de llamadas de red.

La consistencia eventual no aparece obligatoriamente en todos los datos del sistema distribuido: cada servicio puede usar transacciones ACID dentro de su propia base de datos. El problema surge cuando una operación de negocio necesita actualizar estados propiedad de servicios distintos. Entonces debe describirse qué ocurre si Órdenes confirma un cambio y el envío a Inventario falla. Una Saga puede coordinar pasos y compensaciones, mientras que Outbox evita perder un evento generado dentro de una transacción local; ambos patrones resuelven riesgos diferentes. El fragmento HTTP mostrado captura errores de red, pero debe distinguirse un 404 funcional de un 503 transitorio antes de reintentar. La observabilidad debe unir las trazas de ambas peticiones mediante un identificador de correlación. Sin estas medidas, la autonomía del despliegue se compra a cambio de incertidumbre operativa.

Costes ocultos de cada estilo

Los costes más relevantes suelen aparecer en actividades que no se ven en una demostración de cinco minutos. Un monolito con muchos equipos puede necesitar una regresión larga, negociaciones continuas sobre interfaces y despliegues bloqueados por funcionalidades no relacionadas. En una solución distribuida, cada servicio introduce tareas recurrentes: certificados, cuotas, alertas, copias de seguridad, análisis de trazas y cambios de contrato. La infraestructura representa solo una parte del gasto. Cuando algo falla, el tiempo necesario para comprender qué ocurrió puede superar con mucho el ahorro de instancias conseguido mediante escalado selectivo. Para AceriaApp es preferible registrar estos costes en una ficha de decisión y revisarlos después de incorporar comunicación real. De lo contrario, se correría el riesgo de elegir la alternativa más vistosa en un diagrama y no la que resuelve mejor las restricciones del equipo.

```
Costes ocultos:

Monolito:
- Coordinación de equipos.
- Oportunidad tecnológica.
- Escalado homogéneo.

Microservicios:
- Complejidad accidental.
- Observabilidad distribuida.
- Duplicación de utilidades.
```

La lista contrapone tipos de esfuerzo que suelen quedar fuera de las estimaciones iniciales. En el monolito predominan coordinación y cambios compartidos; entre microservicios pesan más la infraestructura de comunicación, el seguimiento de incidencias y la duplicación de herramientas. El balance depende del tamaño y del modo de trabajo del equipo, no de atribuir un precio fijo a cada estilo.

La identificación explícita de estos costes ocultos permite comparar los dos estilos con criterios realistas. Sin este ejercicio, la decisión se basa en percepciones y no en datos. El coste oculto más subestimado en equipos pequeños es el de la complejidad accidental de los microservicios. El coste oculto más subestimado en equipos grandes es el de la coordinación del monolito.

Para valorar costes ocultos es necesario estimar el esfuerzo humano, no solo el número de máquinas. Un monolito grande puede requerir muchas reuniones de integración y pruebas de regresión costosas; varios microservicios exigen crear alertas, rotar certificados, versionar eventos y mantener contratos. En ambos casos, una parte del coste se hace visible solo durante incidentes: tiempo medio hasta detectar el problema, tiempo hasta restaurar el servicio y trabajo de conciliación posterior. En AceriaApp se puede construir una ficha por funcionalidad que registre dependencias, tablas utilizadas,

equipos responsables y frecuencia de cambio. Esa ficha ayuda a distinguir acoplamiento de código, acoplamiento de datos y acoplamiento organizativo. Una métrica agregada, como referencias divididas por proyectos, sirve para observar tendencias, pero no sustituye el análisis del tipo y gravedad de cada dependencia.

Criterios de decisión y aplicación al proyecto del curso

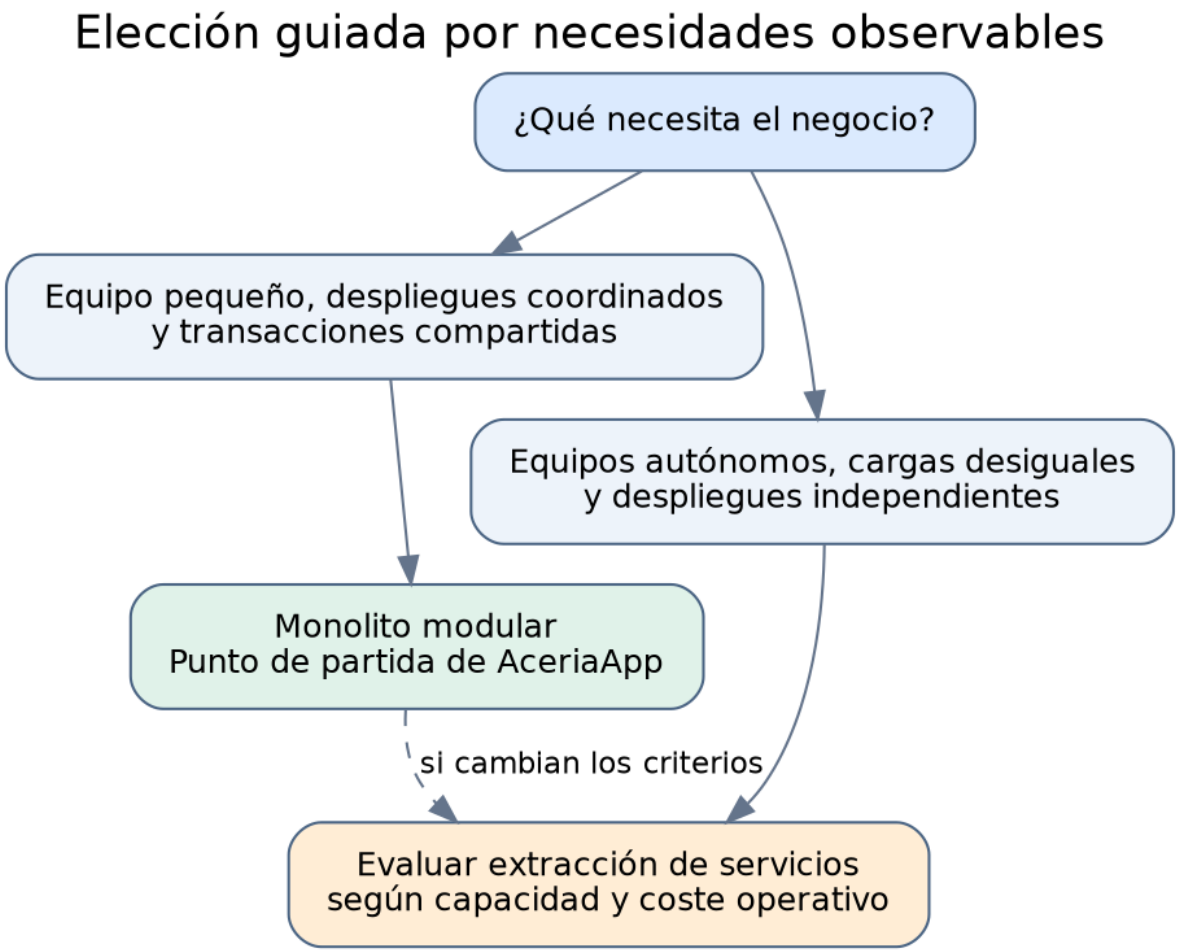


Figura. Criterios que orientan la decisión entre monolito modular y microservicios.

La elección inicial del curso responde a un contexto concreto: el dominio se conoce progresivamente, el equipo es pequeño y las primeras operaciones necesitan poder comprobarse con HTTP, EF Core y SQL Server sin depender de una red de servicios. El monolito modular ofrece una referencia clara desde la que medir consecuencias antes de extraer capacidades. En otra organización, una necesidad fuerte de despliegues independientes o de aislamiento de cargas podría justificar microservicios desde antes. Por eso las decisiones arquitectónicas deben dejar constancia de sus supuestos y de los acontecimientos que obligarían a revisarlas. La aplicación didáctica utilizará índices sencillos de acoplamiento para explorar tendencias, pero no confundirá una puntuación de clase con una recomendación automática de diseño. Lo decisivo será combinar métricas con propiedad de datos, reglas de negocio y capacidad operativa.

```
Criterios aplicados al proyecto del curso:

Dominio mediano              → monolito modular.
Equipo de 2 personas         → monolito modular.
```

```
Despliegue semanal            → monolito modular.
Tolerancia al fallo baja      → monolito modular.
Consistencia fuerte requerida → monolito modular.
Madurez operativa inicial     → monolito modular.
```

El cuadro aplica seis restricciones hipotéticas y muestra por qué, en ese contexto, resulta razonable un monolito modular. La exigencia de consistencia o un equipo de dos personas son pistas, no prohibiciones de utilizar microservicios. Para reutilizar el razonamiento en otro proyecto deben revisarse los supuestos de despliegue, datos y madurez operativa. El objetivo de la decisión es hacer explícitas esas condiciones.

La conclusión es que el monolito modular es la elección correcta para el punto de partida del curso. La evolución hacia microservicios se justificará en módulos posteriores cuando alguno de los seis criterios cambie. El proyecto no adopta microservicios por moda, sino que los adopta cuando un criterio objetivo lo justifique. Este enfoque evita el coste prematuro de la distribución y permite medir el impacto de cada decisión arquitectónica.

Los seis criterios de decisión necesitan pesos acordados por quienes operan el producto y por quienes asumen sus riesgos. Para un equipo pequeño con despliegue semanal y reglas transaccionales exigentes, el monolito modular evita introducir prematuramente red, mensajería y coordinación de fallos. Esa preferencia no impide reservar puntos de extensión: contratos internos, independencia lógica de módulos y documentación de decisiones. Las clases MetricasAcoplamiento y MetricasServicio de este punto representan un ejercicio de instrumentación, no un detector automático de microservicios óptimos. Sus índices son convenciones docentes que requieren validar qué se entiende por proyecto, referencia y tabla compartida. En particular, el denominador de IndiceComparticionDeDatos es NumeroDeProyectos y se debe comprobar la división por cero sobre esa propiedad. Las pruebas cubren las fórmulas, pero no validan por sí solas una decisión arquitectónica.

Caso de análisis — Comparar dos despliegues con los mismos requisitos

Supongamos un escenario hipotético, creado para aplicar el marco de decisión, con tres capacidades: Órdenes, Inventario y Clientes. Durante un ensayo, Órdenes concentra la mayoría de peticiones y sus operaciones consumen más CPU; Inventario recibe consultas periódicas y Clientes se usa de forma esporádica. La necesidad inicial consiste en mantener un nivel de respuesta aceptable para Órdenes sin incrementar innecesariamente el coste del resto. Para evaluar una extracción se definen dos alternativas: tres réplicas del monolito completo o tres réplicas de Órdenes más una de Inventario y una de Clientes. Los números de procesos son conocidos; el ahorro económico, en cambio, no puede deducirse de ellos sin conocer memoria, CPU, red y operación.

La primera fase consiste en fijar condiciones idénticas: volumen de solicitudes, tamaño de la base, nivel de disponibilidad, duración del ensayo y perfil de uso. Después se mide el consumo en reposo, el consumo bajo carga, la latencia por operación y el número de errores. No resulta válido ejecutar una arquitectura en una máquina descargada y la otra con los logs de depuración activados y concluir que la diferencia se debe al estilo arquitectónico. También deben medirse los pasos extra de comunicación remota: obtener información de Inventario desde Órdenes no cuesta lo mismo en una llamada local que en una solicitud HTTP serializada.

Indicador	Qué medir	Interpretación
Recursos	CPU y memoria totales de todos los procesos	Coste directo de operación
Respuesta	Latencia p50, p95 y errores por ruta	Calidad de servicio bajo carga
Despliegue	Cambios que requieren publicar componentes no afectados	Independencia de ciclos de vida

Indicador	Qué medir	Interpretación
Coordinación	Componentes modificados por cada regla de negocio	Acoplamiento de evolución
Consistencia	Operaciones que necesitan confirmación atómica	Riesgo al dividir datos
Recuperación	Reversión y restauración ante fallo de dependencia	Complejidad operativa añadida

La comparación se vuelve interesante si el monolito muestra un gasto elevado de memoria que la separación podría ahorrar. Pero la hipótesis puede fallar: cinco procesos separados podrían consumir más memoria base y exigir un Gateway, una plataforma de logs y una infraestructura de secretos que tres réplicas monolíticas. Por tanto, el equipo no debe anotar «microservicios más baratos», sino «microservicios potencialmente más selectivos; ahorro pendiente de medición». Esa formulación es más precisa y puede orientar pruebas posteriores sin decidir de antemano su resultado.

En el punto 1.2 el alumno crea métricas numéricas para representar dependencia entre proyectos y tablas. El índice de acoplamiento se calcula como referencias divididas por número de proyectos; un resultado de 2,0 significa que se han contado dos referencias por proyecto en promedio, no que exista una escala estándar donde 2,0 implique necesariamente mal diseño. El índice de compartición también depende de qué se está contando. Si cada tabla propia se contabiliza como «compartida», el indicador puede resultar engañoso. Es necesario acompañar los datos de un inventario de referencias y propietarios para interpretar el ratio.

La selección automática de candidatos en `MetricasServicio` utiliza un máximo de una dependencia externa y tres tablas. Son umbrales de la actividad didáctica, no reglas universales de industria. Un servicio con cuatro tablas estrechamente relacionadas podría constituir un límite excelente, mientras otro con una tabla y una dependencia crítica podría ser muy difícil de extraer. El alumno debe ser capaz de explicar por qué el validador devuelve `true` o `false` y, a la vez, por qué ese resultado no justifica por sí solo un despliegue real. Así se aprende la diferencia entre medir, clasificar y decidir.

La decisión final para AceriaApp consiste en conservar el monolito modular como referencia y utilizar las métricas para plantear hipótesis de separación. Un ADR adecuado registraría qué evidencia falta, cómo se obtendría y qué condiciones llevarían a revisar la elección. La arquitectura no se justifica por el número de servicios creados, sino por una mejora verificable para el negocio y por un coste operativo asumible.

La decisión final no debe expresarse como un ranking universal entre monolito y microservicios. El tamaño del dominio, la capacidad del equipo y la exigencia de consistencia son restricciones observadas en un momento concreto. Un ADR es útil cuando conserva ese contexto y define qué debería cambiar para reexaminarlo. La motivación de AceriaApp consiste en disponer de un sistema inicial que permita demostrar operaciones HTTP, persistencia relacional y reglas de dominio sin necesitar desde el primer ejercicio mecanismos de comunicación distribuida. Se trata de una elección pedagógica y operativa concreta, no de afirmar que todos los proyectos de tamaño similar deban comenzar igual.

Condiciones que justificarían reabrir la decisión:

- Órdenes necesita desplegarse de manera independiente de Inventario con una frecuencia que afecta claramente al negocio.
- El consumo de recursos de una capacidad obliga a replicar funciones que permanecen prácticamente inactivas.
- Existe un límite de dominio estable, con propietario de datos y un contrato capaz de evolucionar sin coordinación continua.

- La organización puede mantener monitorización, recuperación y gestión de contratos de varios procesos sin descuidar el desarrollo de funcionalidades.
- Se dispone de una estrategia para las operaciones que antes dependían de una sola transacción local.

Un buen ADR contiene alternativas descartadas y sus costes. En este caso, empezar con microservicios obligaría a resolver descubrimiento, fallos remotos y consistencia sin que exista todavía una necesidad que compense esa inversión. Mantener un monolito sin modularizar también tiene un coste: las dependencias internas podrían dificultar la extracción posterior. La alternativa elegida es el punto intermedio, monolito modular con contratos y responsabilidades explícitas.

La práctica registra la decisión en documentación versionada y añade clases que calculan índices sencillos. Es esencial no confundir esos índices con evidencia suficiente para ejecutar una extracción automática: `EsCandidatoAExtraccion` aplica umbrales establecidos para aprender a razonar, pero no analiza latencia, requisitos de seguridad, transacciones ni carga real. El documento debe advertir esta limitación. Así se enseña al alumno a justificar una decisión con información incompleta, mantener trazabilidad y revisar sus hipótesis conforme el proyecto incorpora nuevos hechos.

Síntesis y condiciones para revisar la decisión

En AceriaApp elegimos inicialmente un monolito modular porque permite enseñar el recorrido completo de una petición y validar los cambios con pocas dependencias operativas. Esa elección tiene límites: si las capacidades adquieren cargas, propietarios o ciclos de despliegue diferentes, habrá que reconsiderar el ámbito del proceso. La comparación no se reduce a «más fácil» frente a «más escalable»; comprende tiempo de publicación, consistencia, observabilidad, coordinación del equipo y coste de incidentes. Conviene registrar la decisión y sus supuestos para poder revisarla cuando existan evidencias nuevas, en lugar de sustituirla por otra moda arquitectónica.

Punto 1.3 — Principios de diseño

Módulo 1: Introducción a los microservicios

Objetivos de aprendizaje:

- Comprender los principios de diseño que guían la descomposición de un sistema en servicios autónomos.
- Identificar el principio de responsabilidad única aplicado a servicios.
- Identificar el principio de ocultación de información y su relación con los límites del servicio.
- Identificar el principio de acoplamiento débil y cohesión fuerte.
- Aplicar los principios al dominio de la acería y añadir al proyecto un documento de principios y una clase de validación de límites de servicio.

Dibujar fronteras a partir del negocio

El código de una aplicación no se convierte en una arquitectura por el mero hecho de distribuir sus clases en varios proyectos. En este punto examinaremos qué principios justifican una frontera, cómo se pueden aplicar mientras AceriaApp todavía es una solución monolítica y qué evidencia pediríamos antes de convertir uno de esos límites en un servicio independiente.

Principios de diseño como criterios de descomposición

Descomponer una aplicación no consiste en repartir clases entre carpetas del mismo tamaño. La frontera de un módulo o de un servicio tiene consecuencias sobre sus reglas, los datos que controla y las modificaciones que necesitarán coordinarse con otros componentes. Para encontrar una frontera útil conviene examinar tres cuestiones complementarias: qué capacidad del negocio representa, qué detalles debe mantener privados y de qué otros módulos depende para completar sus operaciones. Son principios que sirven tanto dentro de un monolito como en una aplicación distribuida. Si no conseguimos expresar esas responsabilidades con claridad en una única solución de Visual Studio, separarlas entre procesos probablemente añadirá dificultades sin resolver el problema original. En AceriaApp comenzamos por el significado de una orden de producción y por las operaciones que deberían conservar sus invariantes; después evaluaremos si esa capacidad requiere autonomía operativa.

```
Categorías de principios de diseño:

1. Responsabilidad  → ¿qué hace el servicio?
2. Ocultación       → ¿qué esconde el servicio?
3. Acoplamiento     → ¿cómo se relaciona con otros?
```

Las tres preguntas del esquema se responden sobre el mismo caso de uso. «Responsabilidad» localiza las decisiones que pertenecen a una capacidad; «ocultación» determina qué detalles no deberían filtrarse a otros módulos; y «acoplamiento» examina cómo atraviesa una petición las fronteras. Se complementan: un servicio puede estar bien nombrado y seguir publicando datos internos o depender directamente del almacenamiento de otro.

La primera categoría, la responsabilidad, responde a la pregunta de qué hace el servicio. La segunda, la ocultación, responde a la pregunta de qué esconde. La tercera, el acoplamiento, responde a la pregunta de cómo se relaciona con otros servicios. Las tres categorías se aplican de forma conjunta: un servicio bien diseñado tiene una responsabilidad clara, oculta sus detalles internos y mantiene un acoplamiento débil con el resto. La aplicación conjunta de las tres categorías es lo que distingue un límite de servicio correcto de uno incorrecto. Si una nueva regla de Órdenes exige modificar clases de

Facturación y de Inventario, la separación actual puede estar ocultando dependencias relevantes. El resultado puede registrarse en una decisión arquitectónica con evidencias observables: número de módulos afectados por cada cambio, contratos expuestos y necesidad de coordinación de despliegue.

La ausencia de principios explícitos produce tres síntomas reconocibles. El primero es el servicio "cajón de sastre", que acumula responsabilidades heterogéneas. El segundo es el servicio "transparente", que expone sus tablas y su modelo interno a otros servicios. El tercero es el servicio "acoplado", que necesita coordinarse con otros para cualquier cambio. Los tres síntomas se corrigen aplicando los principios de responsabilidad, ocultación y acoplamiento respectivamente. El curso aplica estos principios al dominio de la acería para que el alumno los vea en un contexto concreto. La responsabilidad, la ocultación y el acoplamiento deben evaluarse conjuntamente: trasladar métodos a dos carpetas reduce el tamaño de los archivos, pero no elimina las dependencias temporales ni el acceso compartido a tablas. En AceriaApp, las clases de validación sirven para practicar los criterios, pero sus umbrales son simplificaciones educativas y no sustituyen una revisión del dominio con especialistas.

Principio de responsabilidad única aplicado a servicios

Una capacidad del negocio puede necesitar numerosos métodos sin dejar de tener una responsabilidad coherente. El principio de responsabilidad única no prescribe una clase por verbo ni un servicio por tabla: pide que un componente cambie principalmente por un conjunto de razones relacionadas. Crear, cancelar y consultar órdenes pueden formar parte de la misma capacidad si comparten reglas, vocabulario y propietario funcional. Facturar una venta, en cambio, puede depender de leyes fiscales, calendarios y responsables diferentes; agruparlo con las reglas de fabricación podría obligar a modificar el mismo componente por motivos desconectados. Esta observación permite detectar fronteras débiles antes de introducir un servidor por servicio. La prueba no consiste en contar métodos, sino en imaginar cambios de negocio y analizar dónde se producirían y con quién habría que negociar sus efectos.

```
// Correcto: una responsabilidad por servicio
public class OrdenServicio
{
    public void CrearOrden(CrearOrdenCommand command) { }
    public void CancelarOrden(Guid id) { }
}

// Incorrecto: dos responsabilidades en el mismo servicio
public class OrdenYFacturacionServicio
{
    public void CrearOrden(CrearOrdenCommand command) { }
    public void EmitirFactura(Guid ordenId) { }
}
```

Las dos clases hacen visible una posible frontera de negocio. `OrdenServicio` reúne acciones que afectan al ciclo de una orden; `OrdenYFacturacionServicio` mezcla esas acciones con la emisión de facturas. El ejemplo no es una regla que prohíba toda coordinación entre órdenes y facturación: es una advertencia contra ubicar reglas de dominios distintos en el mismo componente sin una razón explícita.

El tamaño del servicio no debería deducirse de una cifra de métodos. Para comprobar si Órdenes es un componente cohesivo, revisaremos cambios como introducir un nuevo estado de fabricación o prohibir cancelar una orden ya terminada. Si ambos cambios se resuelven dentro del mismo módulo, hay una señal de responsabilidad localizada. Si una modificación fiscal obliga también a editar la clase de órdenes, conviene investigar la frontera con Facturación; el problema no es contar funciones, sino comprender qué reglas evolucionan por razones distintas.

La responsabilidad única no equivale a limitar una clase o un servicio a un solo método. Un servicio de Órdenes puede crear, cancelar, consultar y validar órdenes si esas operaciones cambian por las mismas razones de negocio y comparten sus invariantes. La extracción de Facturación debe justificarse cuando existan reglas, ritmos de cambio o responsables suficientemente diferentes, no porque dos métodos aparezcan juntos. También debe evitarse que un servicio de dominio dependa del conocimiento técnico de otros servicios para ejecutar una operación cotidiana. En el modelo de acería, conviene anotar qué reglas pertenecen a planificación de producción y cuáles a emisión de facturas, junto con las entidades afectadas. Los nombres elegidos por el negocio son más fiables que separar módulos por controladores, repositorios o tablas. La calidad de la frontera se observa cuando un cambio habitual permanece localizado.

Un servicio no adquiere responsabilidad única por tener una clase o un verbo principal. Una capacidad de negocio contiene numerosas operaciones que evolucionan juntas. El servicio de Órdenes puede crear, cancelar, consultar y reprogramar órdenes; todas son responsabilidades coherentes cuando responden a las mismas reglas de producción y al mismo propietario funcional. En cambio, calcular el precio de una factura según condiciones fiscales y reservar stock disponible responden a reglas diferentes, aunque ambas operaciones se invoquen durante la aceptación de una orden. La frontera se descubre examinando quién decide cada política y qué cambios se producen conjuntamente, no contando métodos o líneas de C#.

En un taller de análisis, resulta útil formular varios cambios hipotéticos: una nueva restricción sobre cantidad mínima de chapa, una norma de retención de facturas y una forma diferente de notificar a clientes. Después se marca qué clase o módulo debería verse afectado. Si los tres cambios obligan a editar una única clase central, probablemente existen responsabilidades mezcladas. Si las modificaciones se concentran en componentes distintos y la comunicación entre ellos usa contratos estables, existe una señal de buena separación. Esta técnica evita dividir prematuramente un servicio por cada entidad, decisión que suele producir operaciones distribuidas innecesarias.

Caso AceriaApp: la entidad `0rden` del punto 1.1 representa un contrato de persistencia y transporte muy sencillo. Todavía no contiene una política compleja de fabricación. Su existencia no autoriza concluir que la futura capacidad Órdenes esté completamente diseñada. El alumno debe identificar qué reglas podrían incorporarse a esa capacidad y cuáles pertenecerían a Inventario o Facturación. El validador de límites que se añade en el punto 1.3 comprueba atributos declarados; no inspecciona automáticamente el contenido semántico de cada clase. Esta distinción es necesaria para interpretar correctamente el test `Validar_LimiteCompleto_DevuelveValido`: demuestra que un objeto cumple los umbrales configurados, no que el negocio real esté correctamente descompuesto.

Principio de ocultación de información aplicado a servicios

Un módulo ofrece ocultación de información cuando sus consumidores pueden utilizarlo sin conocer su almacenamiento, algoritmos o representación interna. En una aplicación de órdenes, otros componentes podrían necesitar la disponibilidad de un material, pero no tendrían por qué conocer tablas, índices ni reglas privadas de Inventario. Es mejor exponer un contrato estable que permitir modificaciones directas sobre objetos internos. Esta decisión tiene consecuencias concretas: permite cambiar EF Core, ajustar un cálculo o dividir el almacenamiento sin modificar a todos los consumidores, siempre que el contrato preserve su significado. Ocultar no equivale a dejar el comportamiento sin documentar; por el contrario, el consumidor debe conocer qué acepta una operación, qué devuelve y cómo informa de los errores. El diseño se empobrece tanto al exponer todo como al convertir cada interacción en una caja negra imposible de razonar.

```
Ocultación en tres niveles:

1. Modelo de datos   → cada servicio tiene su propia base de datos.
2. Lógica interna    → solo se exponen contratos públicos.
3. Topología         → API Gateway centraliza el enrutamiento.
```

El esquema recorre tres ámbitos de ocultación. La propiedad del modelo de datos evita escrituras ajenas; el contrato público permite utilizar una función sin conocer su implementación; y el enrutamiento puede ocultar a los clientes la ubicación de cada proceso. Disponer de un gateway es una opción frecuente, no la condición que convierte una API en un microservicio autónomo.

La violación más común del principio es el acceso directo a la base de datos de otro servicio. Cuando dos servicios comparten una base de datos, cualquier cambio en el esquema afecta a ambos y la independencia de despliegue desaparece. La segunda violación es la exposición del modelo interno a través de la API. Cuando un servicio devuelve su entidad de base de datos tal cual, cualquier cambio en la entidad rompe a los consumidores. La tercera violación es la exposición de la topología interna. Cuando los clientes conocen la ubicación de cada servicio, cualquier cambio de infraestructura afecta a los clientes.

Un DTO público no debe incluir propiedades de navegación del ORM ni detalles como estados de seguimiento de EF Core. Si una entidad incorpora un nuevo atributo interno, el contrato de respuesta puede seguir siendo el mismo mientras los consumidores no necesiten ese dato. Es importante diferenciar ocultación de secreto: un DTO no protege información confidencial por sí solo; el servicio debe autorizar las operaciones y filtrar los campos según el cliente. La base de datos es otro límite de responsabilidad. Compartir el mismo servidor SQL por motivos administrativos puede ser compatible con autonomía si cada servicio posee sus tablas, permisos y migraciones, pero permitir consultas directas entre esquemas destruye esa independencia. En AceriaApp, los ejemplos de DTO y de Gateway anticipan decisiones que se desarrollarán después. En este punto solo se analizan y validan límites hipotéticos, no se implementa todavía esa infraestructura distribuida.

La ocultación de información debe evitar que el consumidor dependa de decisiones privadas del productor. Una entidad de EF Core puede contener propiedades de navegación, configuraciones de seguimiento, campos internos o reglas de actualización que no deberían convertirse en requisitos de una API pública. Cuando un endpoint devuelve directamente una entidad, un refactor del modelo puede modificar accidentalmente el JSON. Un DTO dedicado separa ambas representaciones: la entidad puede cambiar su forma interna mientras la API conserva el contrato que conocen los consumidores. Esa separación requiere mapear explícitamente valores y decidir qué campos se admiten en entrada y salida.

Existe también ocultación de topología. El cliente de una aplicación web no debería conocer cada host interno ni construir direcciones concatenando nombres de contenedor. Un Gateway puede resolver rutas públicas hacia servicios internos, pero no debe trasladar el modelo privado de un servicio ni asumir sus reglas de dominio. Los nombres DNS, puertos y réplicas pertenecen a la configuración operativa y pueden cambiar sin modificar el contrato semántico. En un monolito inicial esta separación puede parecer innecesaria, aunque prepara la migración hacia servicios independientes cuando las condiciones la justifiquen.

La prueba decisiva para evaluar la ocultación es un cambio controlado. Se puede renombrar internamente una propiedad, introducir una representación de persistencia diferente o modificar el proveedor de datos y comprobar que el mismo cliente sigue utilizando la API sin cambios. Si un consumidor necesita recompilar su código por una decisión privada del productor, la frontera no está funcionando. En AceriaApp, la clase `LimiteServicio` recoge `ExponeEntidadesDeDominio` y `ComparteBaseDeDatos` como indicadores de riesgo. Esos booleanos facilitan una explicación didáctica, pero no sustituyen pruebas de integración ni inspecciones de dependencias reales. La validación debe interpretarse como una herramienta para formular preguntas arquitectónicas precisas.

Principio de acoplamiento débil y cohesión fuerte

La cohesión expresa si las operaciones de un componente contribuyen a una misma responsabilidad; el acoplamiento refleja cuánto necesita conocer de otros componentes para funcionar. Ambos principios se relacionan, pero no son simples cifras de métodos o referencias. Un servicio puede tener

pocas dependencias y estar mal diseñado si agrupa reglas comerciales que cambian por motivos distintos. También puede tener varias dependencias legítimas si su caso de uso orquesta contratos bien definidos sin conocer implementaciones privadas. Lo que interesa es evaluar la **dirección y el significado** de esas dependencias: ¿la aplicación conoce una abstracción estable o accede directamente a una tabla ajena? En AceriaApp, una operación de planificación puede solicitar información a Inventario, pero no debería apropiarse de su política de cálculo de existencias. Una frontera adecuada hace localizables las reglas y limita los cambios en cascada.

```
Cohesión alta:  un servicio maneja un solo concepto.
Cohesión baja:  un servicio maneja varios conceptos.

Acoplamiento bajo:  un servicio depende de pocos servicios.
Acoplamiento alto:  un servicio depende de muchos servicios.
```

Los términos alta y baja califican relaciones, no tamaños absolutos. Un servicio que implementa varias operaciones de la misma capacidad puede ser cohesivo; un servicio pequeño con reglas de dominios inconexos puede no serlo. De igual modo, importa más una dependencia estable y explícita que un recuento ciego del número de servicios invocados. Para evaluar el diseño hay que estudiar ejemplos de cambios y seguir sus consecuencias.

La cohesión se pone a prueba examinando qué operaciones deben cambiar juntas y qué conocimiento necesita cada una de otros módulos. Una consulta de stock puede ser una dependencia legítima de un caso de uso de planificación; no por ello debería introducir en Producción las reglas privadas de Inventario. La calidad del límite se aprecia cuando un cambio cotidiano permanece localizado y los demás componentes solo conocen un contrato estable. Un indicador numérico de referencias puede orientar una revisión, pero jamás sustituir ese análisis funcional.

La cohesión no puede reducirse objetivamente al número de entidades o conceptos: tres conceptos fuertemente relacionados pueden formar un conjunto coherente, mientras dos conceptos sin relación pueden indicar una frontera incorrecta. Del mismo modo, contar dependencias salientes no expresa su criticidad. Una dependencia a un servicio disponible de manera asíncrona no tiene el mismo coste que una llamada síncrona imprescindible para confirmar cada operación. El validador del ejercicio utiliza umbrales de tres conceptos y tres dependencias porque necesita reglas comprobables, pero deben presentarse como heurísticas docentes. En un análisis real se documenta para cada dependencia su frecuencia, dirección, latencia, semántica de fallo y necesidad de consistencia. En AceriaApp, separar Órdenes e Inventario puede aumentar cohesión funcional y, al mismo tiempo, crear un flujo distribuido de reserva de materiales. Ambas consecuencias deben valorarse en la misma decisión.

Un número pequeño de dependencias salientes suele facilitar la evolución, pero no todas las dependencias tienen el mismo coste. Consumir un contrato estable para consultar un dato de referencia no equivale a necesitar que tres servicios confirmen simultáneamente cada operación de negocio. La primera dependencia puede introducir latencia y disponibilidad remota; la segunda, además, puede obligar a coordinar estados, compensaciones y versiones. De modo similar, medir cohesión por número de conceptos es una aproximación pedagógica: dos conceptos pueden ser inseparables dentro de una misma capacidad y cuatro conceptos pueden pertenecer a una regla de negocio perfectamente coherente.

Para detectar acoplamiento problemático deben observarse los motivos de cambio y los flujos de datos. Una dependencia es especialmente costosa cuando obliga a desplegar dos componentes simultáneamente, cuando impide operar si una capacidad secundaria está caída o cuando comparte tablas que ambos lados pueden modificar. En lugar de contar solo llamadas HTTP, conviene clasificar si son lecturas, comandos, eventos y consultas transaccionales. Cada categoría presenta riesgos distintos. En AceriaApp la organización por carpetas ayuda a identificar responsabilidades, mientras que los índices de `MetricasServicio` y `LimiteServicio` permiten practicar decisiones basadas en reglas simples.

Indicio	Interpretación prudente
Muchos endpoints en una capacidad	Puede haber una API rica y coherente; revisar reglas, no solo cantidad
Pocas llamadas externas	Puede indicar autonomía o simplemente una responsabilidad muy estrecha
Varios módulos editan una tabla	Hay riesgo de propiedad compartida y coordinación de cambios
Dos capacidades cambian siempre juntas	Revisar si la frontera coincide con el negocio

El método `Validar` del punto 1.3 marca como no válidos los límites que superan umbrales concretos. Esta automatización es útil para aprender a interpretar un informe de reglas: identifica candidatos a revisión, no emite sentencias definitivas. Antes de dividir una capacidad, el equipo debe contrastar el resultado con el lenguaje del dominio, las transacciones necesarias y las exigencias de operación. De esa forma, las métricas complementan el juicio arquitectónico en vez de sustituirlo.

Principio de límite del contexto



Figura. Separación de contextos delimitados en una acería.

El lenguaje del negocio ayuda a descubrir límites que una taxonomía técnica no revelaría. En una acería, la palabra «orden» puede significar una orden de fabricación o una orden de compra; comparten una palabra, pero no necesariamente estados, responsables ni reglas. DDD denomina **contexto delimitado** al ámbito en el que un modelo y sus términos poseen significado consistente. Esa frontera orienta la separación funcional y puede sugerir la propiedad de un servicio, sin obligar a desplegar uno por cada sustantivo del negocio. Lo importante es localizar qué hechos necesita conocer otro contexto y definir cómo los expresa. Cuando Producción necesita que Inventario confirme existencias, ambas capacidades deberían acordar un mensaje o consulta y evitar intercambiar el modelo interno completo de cualquiera de ellas. Este análisis precede a la creación de controllers, repositorios o contenedores.

```
Contextos delimitados en la acería:

Contexto de producción  → "orden" = orden de fabricación.
Contexto de aprovisionamiento → "orden" = orden de compra.
Contexto de clientes    → "cliente" = empresa compradora.
```

Los tres ejemplos de «orden» muestran por qué un nombre aislado no identifica un modelo. La orden de producción pertenece a reglas de fabricación, mientras que la orden de compra se relaciona con aprovisionamiento; ambas pueden necesitar intercambiar información, pero sus estados y responsables no son necesariamente los mismos. El vocabulario compartido debe fijarse dentro de cada contexto y traducirse explícitamente en sus límites.

Un contexto delimitado mantiene un significado coherente para sus conceptos. La orden de fabricación no tiene por qué compartir estados ni reglas con una orden de compra, aunque sus

nombres coincidan. Esa diferencia justifica modelos separados, pero el paso a procesos distintos es otra decisión: puede existir más de un contexto en una solución monolítica, y algunos servicios abarcar un conjunto coherente de responsabilidades. La frontera operativa debe justificarse con necesidades de despliegue, propiedad de datos y comunicación, además de las reglas del negocio.

Un contexto delimitado se define por el significado y las reglas que comparten sus participantes, no solo por el nombre de un objeto. En Producción, una orden puede representar trabajo planificado, rutas de proceso y fecha de fabricación; en Aprovisionamiento puede significar el pedido enviado a un proveedor. Aunque ambas utilicen el texto “orden”, sus estados, propietarios y ciclos de vida no tienen por qué coincidir. Intentar imponer una única entidad universal genera acoplamiento semántico. La traducción entre contextos debe realizarse mediante contratos explícitos y transformaciones conscientes, no compartiendo directamente clases del dominio. Durante el diseño conviene enumerar los eventos que cruzan la frontera: orden aprobada, material reservado o recepción confirmada. El catálogo de límites de AceriaApp incorpora el campo ContextoDelimitado para practicar esta identificación, pero no demuestra automáticamente que los significados sean consistentes.

El concepto *orden* tiene una forma distinta según el contexto. En Producción, una orden determina qué producto fabricar, en qué cantidad y bajo qué restricciones; en Compras, una orden se dirige a un proveedor y representa compromisos de aprovisionamiento. El hecho de compartir la palabra no significa que ambas operaciones deban emplear la misma clase o tabla. Si se crea un único tipo `Orden` con todas las propiedades posibles, cada capacidad termina dependiendo de campos que no tienen sentido para su negocio. Un contexto delimitado permite que cada modelo use sus propios términos y valide sus propias invariantes.

Un mapa de contextos indica también cómo se intercambia la información entre capacidades. Producción puede consultar disponibilidad a Inventario mediante un contrato expresado en cantidades de material; Inventario no necesita conocer las propiedades internas de una orden de fabricación. Facturación puede recibir la confirmación de una operación comercial y construir su documento fiscal a partir de datos acordados. Las traducciones entre modelos cumplen una función importante: impiden que un cambio semántico en un contexto se propague sin control a los demás. Un adaptador anticorrupción, estudiado posteriormente, formaliza ese cambio de representación.

Ejemplo de límite correcto: Producción conoce la referencia de material y la cantidad necesaria para fabricar; Inventario conoce existencias, ubicación y movimientos. Órdenes envía una solicitud de reserva con un identificador, no modifica directamente la tabla de movimientos. Si Inventario rechaza la operación, Producción conserva un estado de negocio reconocible y no inventa que la reserva se completó. Este escenario no exige desarrollar todavía mensajería ni un servicio remoto; sirve para explicar por qué la frontera se define por conocimiento y propiedad.

En la práctica del punto 1.3 se describen límites con atributos `CapacidadDeNegocio`, `ContextoDelimitado` y `NumeroDeDependenciasSalientes`. Cada atributo debe asociarse a una decisión de modelado que el alumno pueda verbalizar y justificar. Un test que marca un límite como válido no reemplaza el mapa conceptual: solo comprueba que las propiedades proporcionadas cumplen reglas explícitas. El resultado docente es comprender el razonamiento que permite llenar esas propiedades con información real.

Principio de autonomía del servicio

La autonomía de un servicio suele explicarse en términos de despliegue, datos y tolerancia a fallos, aunque cada dimensión necesita una comprobación distinta. Un proceso puede publicarse solo y, sin embargo, depender de la base de datos de otro para casi todas sus operaciones. Del mismo modo, poseer un `DbContext` independiente no prueba que exista propiedad de datos si varios servicios escriben las mismas tablas. La autonomía se consigue al delimitar qué invariantes mantiene cada capacidad y qué intercambios están permitidos entre ellas. Algunos flujos necesitarán coordinación y consistencia eventual; otros pueden responder con información parcial cuando una dependencia falle.

En el módulo inicial solo se prepara una estructura en la que estas decisiones puedan identificarse; la operación distribuida real se incorpora mucho después, cuando el dominio y los contratos sean suficientemente claros.

```
// Autonomía de datos: cada servicio tiene su propio DbContext
public class OrdenesDbContext : DbContext
{
    public DbSet<Orden> Ordenes => Set<Orden>();
}

public class InventarioDbContext : DbContext
{
    public DbSet<Material> Materiales => Set<Material>();
}
```

Los dos contextos EF Core sirven para distinguir modelos persistidos por capacidades diferentes. Que existan `OrdenesDbContext` e `InventarioDbContext` no garantiza, por sí mismo, autonomía de datos: habría que revisar qué tablas modifican, qué credenciales utilizan y si un componente puede alterar registros pertenecientes al otro.

La propiedad de los datos significa que una capacidad decide cómo almacenar y modificar su información. No exige necesariamente una máquina SQL distinta para cada servicio: bases o esquemas separados en un mismo motor pueden mantener propiedad funcional si se evitan accesos de escritura cruzados y se gobiernan permisos y cambios. Tener varios `DbContext` tampoco basta si todos editan las mismas tablas. En AceriaApp se estudiará la diferencia entre propiedad lógica, separación de esquemas y aislamiento físico cuando aparezcan Producción e Inventario.

La autonomía de datos exige que un servicio sea responsable de sus migraciones, respaldos y políticas de acceso. Dos DbContext diferentes conectados al mismo esquema y escribiendo las mismas tablas no cumplen el principio solo por tener nombres distintos. La autonomía de fallo también necesita diseñarse por caso de uso: una consulta histórica de órdenes puede continuar si Inventario no responde, mientras una operación que reserva material quizá deba suspenderse. En la autonomía de despliegue importa la compatibilidad del contrato; actualizar Órdenes de forma aislada carece de valor si Inventario debe desplegarse al mismo tiempo por una modificación compartida. Estas tres formas se miden de forma distinta y ninguna garantiza automáticamente las demás. En AceriaApp, el monolito de partida usa un DbContext común; la separación física se estudiará posteriormente cuando existan procesos verdaderamente independientes.

Principio de contrato explícito

Un contrato explícito es la parte del comportamiento que el consumidor puede utilizar con seguridad. En HTTP incluye rutas, verbos, estructura de las peticiones y respuestas, códigos de estado y reglas de compatibilidad. En eventos comprende además el significado del hecho publicado, su versión y la posibilidad de recibirlo más de una vez. Al separar estas superficies del modelo interno, un servicio puede modificar sus clases sin obligar a los clientes a recompilarse. Para AceriaApp, un DTO de orden representa datos compartidos con consumidores, mientras la entidad de dominio debería conservar invariantes que no se exponen como propiedades editables. La claridad de ese límite es más importante que elegir prematuramente un formato sofisticado. El contrato sirve a sus consumidores: debe explicar qué pueden esperar de él, especialmente cuando la operación no puede completarse.

```
public record OrdenDto(
    Guid Id,
    string Cliente,
    string Material,
    int Cantidad,
    DateTime FechaCreacion);
```

El `record OrdenDto` expresa una representación que puede circular por un contrato HTTP; agrupa identidad, cliente, material, cantidad y fecha. Es distinto de la entidad que protege reglas internas y decide transiciones. Su utilidad está en publicar solo los datos acordados, y su evolución debe evaluarse desde los consumidores que lo utilizan.

Exponer un DTO permite estabilizar la superficie pública sin incluir cada detalle del modelo de dominio. La protección no se obtiene automáticamente por utilizar `record` o nombrar una clase `Dto`: exige decidir qué propiedades se publican, cómo se validan y qué significan para el consumidor. Si la entidad `Orden` incorpora después nuevas reglas, el contrato `OrdenDto` podrá mantenerse siempre que su semántica siga siendo compatible. El diseño de DTOs facilita la evolución, pero no la garantiza por sí solo.

El contrato no es solo una estructura de datos. También define códigos de respuesta, restricciones, errores, paginación y significado de los campos. Un OrdenDto con propiedades idénticas a una entidad sigue siendo útil si establece una frontera estable entre la API y el modelo de persistencia. Las fechas deben expresar con claridad la zona horaria y los identificadores deben conservar su significado entre versiones. Cambiar el nombre o el tipo de un campo puede romper consumidores generados automáticamente; añadir campos tampoco es siempre inocuo cuando el cliente realiza validación estricta. En AceriaApp, el DTO presentado es un ejemplo teórico para el módulo de APIs REST, no una clase añadida al proyecto en el punto 1.3. El validador de límites comprueba el indicador ExponeEntidadesDeDominio declarado por el autor, pero no inspecciona endpoints ni serialización de respuestas.

Un contrato público debe describir una promesa que los consumidores puedan comprender y verificar. Un DTO es una pieza de esa promesa, pero no la agota. El contrato incluye la ruta, el método HTTP, los campos obligatorios, las restricciones de validación, los posibles códigos de error y el significado de la respuesta. Cambiar un nombre en el JSON, por ejemplo `Cliente` por `Comprador`, puede romper aplicaciones externas incluso cuando el tipo C# se recompila correctamente. Por el contrario, modificar una entidad interna sin variar el DTO ni el comportamiento HTTP no tiene por qué afectar a ningún consumidor.

Conviene distinguir compatibilidad de código y compatibilidad de contrato. Dos servicios pueden utilizar el mismo ensamblado compartido y compilar en la misma solución, pero estar temporalmente acoplados: una actualización del paquete común puede exigir que ambos se desplieguen simultáneamente. En una arquitectura distribuida resulta preferible definir contratos públicos suficientemente estables y realizar cambios compatibles cuando sea posible. La coexistencia de versiones puede facilitar una migración, pero supone una obligación adicional de mantenimiento, observabilidad y retirada controlada de consumidores antiguos.

Escenario de AceriaApp: la versión v1 de una orden expone identificador, cliente, material, cantidad y fecha. Una futura versión interna añade una clasificación de prioridad para planificar fabricación. Se puede conservar v1 sin publicar inmediatamente la prioridad, o diseñar una adición opcional si los consumidores toleran campos desconocidos. No se debe asumir compatibilidad de forma automática: generadores de clientes, validadores estrictos y reglas de negocio pueden interpretar una ampliación como un cambio significativo. La prueba adecuada utiliza consumidores representativos y verifica deserialización, semántica y códigos de respuesta.

La teoría prepara así la práctica de buenas prácticas del punto 1.5, donde el catálogo registra la adopción de contratos. Ese catálogo no constituye por sí mismo una prueba de compatibilidad: la afirmación `EstaAplicada = true` es un dato proporcionado al validador. Para demostrar compatibilidad real deben ejecutarse contratos en un escenario que reproduzca las expectativas del consumidor. Separar declaración, implementación y evidencia evita convertir una norma de diseño en una simple casilla marcada.

Principio de tolerancia al fallo

Un servicio tolerante a fallos no promete responder correctamente siempre, sino conservar un comportamiento **definido** cuando una dependencia deja de estar disponible. Una consulta informativa de existencias podría devolver un error temporal o una respuesta degradada claramente identificada; una orden que exige confirmación de stock no debería inventar disponibilidad. La elección depende de las reglas del negocio y del riesgo de utilizar información incompleta. Timeouts, reintentos y circuit breakers protegen recursos, pero cada uno resuelve un problema diferente y también puede introducir efectos secundarios si se configura sin límites. En AceriaApp se estudiarán esas políticas con escenarios de caída reales y controlados. Aquí la idea esencial es que un fallo remoto forma parte del contrato operativo y debe ser probado del mismo modo que el camino satisfactorio.

```
var policy = Policy
    .Handle<HttpRequestException>()
    .CircuitBreakerAsync(
        exceptionsAllowedBeforeBreaking: 3,
        durationOfBreak: TimeSpan.FromSeconds(30));
```

El ejemplo muestra una política de corte ante excepciones HTTP repetidas y un intervalo antes de intentar recuperar la dependencia. No basta con declararla para demostrar resiliencia: hay que provocar fallos, contar intentos y verificar la recuperación. Es una formulación conceptual de Polly; las APIs concretas de resiliencia usadas más adelante en el curso pueden diferir.

Una dependencia remota debe tratarse como potencialmente indisponible, aunque no todas las llamadas requieran la misma política. Un timeout es necesario para evitar esperas sin límite; un reintento solo tiene sentido si el fallo es recuperable y la operación puede repetirse con seguridad; un circuit breaker puede evitar saturar un destino claramente degradado. En una planificación de producción sería incorrecto devolver éxito con datos de inventario inventados. La respuesta frente al fallo debe cumplir el contrato del negocio, incluso cuando signifique rechazar temporalmente la operación.

La tolerancia al fallo debe distinguir los errores transitorios de los definitivos. Un 400 por validación de la solicitud no se soluciona repitiendo la llamada; un timeout puede indicar sobrecarga temporal, pero un reintento sin límite agrava esa condición. El circuit breaker reduce la presión sobre un destino que presenta fallos repetidos y debe configurarse junto con presupuestos de tiempo y un mecanismo para observar su estado. También es necesario definir qué funcionalidad degradada es aceptable: ofrecer la última disponibilidad conocida puede ser útil para una pantalla informativa, pero peligroso para confirmar una reserva física. En el ejemplo de Polly, el umbral de tres excepciones y la ventana de treinta segundos son valores ilustrativos, no universales. Las pruebas de resiliencia tienen que provocar fallos controlados y confirmar que el sistema no pierde datos ni promete resultados que no puede garantizar.

Principio de evolución independiente

La evolución independiente implica que una capacidad pueda modificar su modelo interno sin exigir una actualización simultánea de todos sus consumidores. Para ello los contratos tienen que mantener significado estable, y los cambios incompatibles necesitan una estrategia de transición. Introducir una versión nueva de una ruta no resuelve por sí solo la compatibilidad: hay que mantener durante el periodo acordado la anterior, probar clientes relevantes y controlar qué datos pueden desconocer. Algunas ampliaciones aditivas son compatibles, otras afectan a las reglas de validación aun cuando conservan el formato. El principio también depende de la organización del trabajo: los equipos deben conocer quién puede modificar el contrato y cómo se comunica una deprecación. En el curso aprenderemos a registrar esas decisiones antes de cambiar una ruta pública que ya tenga consumidores.

```
Versionado de contratos:

/api/v1/ordenes  → contrato estable, sin cambios.
/api/v2/ordenes  → contrato nuevo, con campos adicionales.
```

Las rutas `/api/v1/ordenes` y `/api/v2/ordenes` representan una transición explícita entre contratos. Mantener dos rutas permite migrar consumidores gradualmente, pero exige documentar qué comportamiento conserva cada versión y durante cuánto tiempo permanecerá disponible la anterior.

Para evolucionar un contrato debemos distinguir cambios compatibles de cambios incompatibles. Un productor puede añadir campos opcionales conservando su ruta actual si los consumidores antiguos los toleran; una modificación que altera el significado de un dato puede requerir un nuevo contrato y un periodo de migración. Versionar mediante `/api/v1` y `/api/v2` es una estrategia posible, no una obligación universal. La independencia se logra cuando el equipo puede comprobar que los clientes antiguos siguen funcionando, comunicar deprecaciones y retirar versiones solo tras conocer su utilización.

Un contrato versionado permite coexistencia temporal, pero mantener versiones antiguas indefinidamente introduce costes y riesgos de seguridad. Para retirar una versión, el productor necesita saber qué consumidores la utilizan, comunicar una fecha de eliminación y comprobar el tráfico residual. No todos los cambios requieren crear /v2: añadir un campo opcional puede ser compatible, mientras cambiar la semántica de una propiedad existente suele exigir contrato nuevo. El versionado por URL es una opción clara para formación, aunque existen alternativas basadas en cabeceras o tipos de contenido. La evolución de la base de datos debe coordinarse también con despliegues graduales, siguiendo cambios expansivos antes de retirar columnas antiguas. En AceriaApp, los ejemplos /api/v1 y /api/v2 son propuestas de contrato; las prácticas de este punto validan límites, no implementan todavía una segunda versión HTTP.

Aplicación de los principios al proyecto del curso

Los principios anteriores deben poder comprobarse en la solución y no quedarse en una declaración general de buenas intenciones. La responsabilidad se observa cuando una regla de producción permanece localizada; la ocultación, cuando un consumidor no necesita tipos de infraestructura ajena; la autonomía, cuando el despliegue y los datos pertenecen a un propietario identificable; y la tolerancia a fallos, cuando la aplicación responde conforme a un contrato aun si una dependencia falla. El punto 1.3 incorpora un pequeño validador y documentación de límites como herramienta de aprendizaje. Esos tests muestran que los datos de entrada cumplen determinados criterios, pero no pueden certificar automáticamente que la descomposición del negocio sea correcta. Para afirmar esto último hay que analizar casos reales y comprobar si las decisiones reducen o multiplican la coordinación del equipo.

```
Aplicación de los principios al proyecto:

Responsabilidad única       → módulos por capacidad de negocio.
Ocultación                  → cada módulo con su propia carpeta.
Acoplamiento débil          → interfaces entre módulos.
Límite del contexto         → contextos delimitados.
Autonomía                   → evolución hacia microservicios.
Contrato explícito          → DTOs públicos.
Tolerancia al fallo         → Polly.
Evolución independiente     → versionado de contratos.
```

La correspondencia final vincula cada principio con una decisión de la evolución del curso. El mapa sirve para localizar qué deberíamos poder observar o probar cuando lleguen los módulos siguientes: contratos DTO, límites entre capacidades, políticas de resiliencia y versiones públicas. No convierte por sí solo esas características en funcionalidades ya implementadas en M01.

La aplicación de los principios no es un ejercicio teórico: cada principio se materializa en una decisión concreta del proyecto. El alumno puede rastrear cada decisión hasta el principio que la justifica. Esta trazabilidad es lo que distingue una arquitectura razonada de una arquitectura por acumulación. El punto 1.3 cierra el bloque de fundamentos y prepara el terreno para el punto 1.4, donde se estudiará la organización concreta de una solución basada en microservicios.

La aplicación de los principios se beneficia de una matriz explícita entre decisión, evidencia y riesgo. Para responsabilidad única se registra la capacidad de negocio; para ocultación, los contratos visibles; para acoplamiento, la lista de dependencias; para autonomía, la propiedad de los datos; para tolerancia, la respuesta ante fallos. En el código del punto 1.3, LimiteServicio es una descripción declarada, ValidadorLimiteServicio examina atributos de esa descripción y CatalogoLimites filtra resultados. Una prueba que marque ComparteBaseDeDatos como verdadero demuestra que el validador emite un error; no demuestra que la aplicación carezca de accesos compartidos. Esta distinción evita confundir pruebas del algoritmo educativo con auditorías automáticas de arquitectura real. En módulos posteriores pueden añadirse pruebas estructurales que analicen referencias, contratos y rutas de persistencia con evidencias directas.

Caso de análisis — Corregir un límite de servicio mal definido

Consideremos una propuesta denominada `GestionGeneral`, responsable de crear órdenes, consultar existencias, actualizar fichas de clientes y generar facturas. Desde el punto de vista de la organización del código parece cómoda: la API ofrece un único lugar desde el cual ejecutar todo. Sin embargo, reúne reglas que cambian por razones distintas. El área de producción puede modificar la secuencia de fabricación, almacén puede cambiar su modelo de ubicaciones, el equipo comercial puede alterar los datos de cliente y administración puede revisar normas de facturación. Aunque esas cuatro capacidades utilicen la palabra «pedido» en distintos momentos, no comparten necesariamente el mismo modelo ni la misma cadencia de evolución.

Para identificar los límites se redactan primero las responsabilidades en lenguaje del negocio. Después se asocia cada responsabilidad a sus datos propietarios y a las decisiones que toma. La confirmación de una orden puede necesitar conocer la disponibilidad de material, pero conocer no significa poseer. Producción debería utilizar un contrato de Inventario y tratar sus respuestas como hechos externos. Una respuesta de «material insuficiente» pertenece a Inventario; la decisión sobre si una orden queda pendiente o rechazada pertenece a Producción. La frontera es más clara cuando se separan hechos observados de decisiones tomadas.

Capacidad	Conocimiento propio	Información que recibe de otras capacidades
Producción	Reglas de fabricación y estado de órdenes	Disponibilidad o reserva de materiales
Inventario	Existencias, ubicaciones y movimientos	Solicitudes de reserva identificadas
Clientes	Identidad y datos comerciales autorizados	Referencias de operaciones relacionadas
Facturación	Emisión y estado de documentos fiscales	Datos necesarios de operaciones confirmadas

La siguiente revisión examina datos compartidos. Si `GestionGeneral` dispone de una sola entidad `Orden` que representa simultáneamente orden de compra y orden de fabricación, el problema no se resuelve cambiando el nombre de una clase. Hace falta reconocer contextos donde el término tiene un significado diferente y definir una traducción explícita entre representaciones. Producción podría utilizar `OrdenFabricacion` y Compras `OrdenCompra`. No se trata de escribir más clases por costumbre, sino de evitar que las reglas específicas de una actividad contaminen la otra.

El validador del punto 1.3 ayuda a recorrer este razonamiento mediante propiedades declarativas. `CapacidadDeNegocio` y `ContextoDelimitado` obligan a dar nombre al propósito; `NumeroDeConceptos` y `NumeroDeDependenciasSalientes` introducen señales de cohesión y acoplamiento; `ExponeEntidadesDeDominio` y `ComparteBaseDeDatos` representan posibles filtraciones del límite. La implementación devuelve mensajes diferenciados para cada regla vulnerada. El valor pedagógico consiste en leer esos mensajes y relacionarlos con decisiones reales, no en memorizar el número tres de un umbral.

Es igualmente importante identificar falsos positivos y falsos negativos. Una propuesta puede superar todas las reglas numéricas y continuar siendo inadecuada porque su capacidad de negocio está mal descrita. También puede superar un umbral por una razón perfectamente legítima y requerir únicamente documentación, no una división adicional. El método `Validar` no interpreta el lenguaje de una organización ni conoce el significado de una reserva; solo procesa atributos. La validación automatizada es una ayuda para revisar, no un sustituto del modelado de dominio.

La evidencia que enlaza teoría y práctica es doble. Los tests verifican que el validador detecta los casos previstos y que `CatalogoLimites` separa elementos válidos e inválidos. La justificación escrita permite explicar por qué se aceptó o rechazó un límite en el contexto de la acería. Ambas formas de evidencia son necesarias para que la enseñanza de responsabilidad única y contexto delimitado no quede reducida a un conjunto de booleanos.

Síntesis: el límite que podemos justificar

Un buen límite se reconoce menos por su nombre que por los cambios que permite contener. En AceriaApp es razonable pensar en Producción e Inventario como capacidades diferenciadas, pero habrá que comprobar sus reglas, propietarios de datos y necesidades de comunicación antes de dar por concluida la división. Las abstracciones, los DTO, la tolerancia a fallos y el versionado son mecanismos que ayudan a preservar esas fronteras; ninguno sustituye el análisis del negocio. Las comprobaciones del laboratorio inicial enseñan a formular criterios y detectar dependencias evidentes, no a demostrar de forma automática una arquitectura distribuida completa.

Punto 1.4 — Organización de una solución basada en microservicios

Módulo 1: Introducción a los microservicios

Objetivos de aprendizaje:

- Comprender cómo se organiza una solución basada en microservicios.
- Diferenciar entre repositorio único (monorepo) y repositorio por servicio (polyrepo).
- Identificar las convenciones de nombres de proyectos, carpetas y ensamblados.
- Comprender la separación entre código de producción, tests y documentación.
- Añadir al proyecto AceriaApp un documento de organización de solución y una clase de verificación de convenciones de nombres.

Hacer visible la arquitectura en una solución de trabajo

Después de identificar los límites funcionales, necesitamos una organización que permita mantenerlos. Una estructura adecuada debe guiar a un alumno hasta el código, las pruebas y las decisiones de cada estado sin confundir la presentación de los materiales docentes con los límites de despliegue de un sistema real. Examinaremos repositorios, proyectos, nombres y documentación siguiendo esa distinción.

Organización de soluciones y su impacto en la evolución

La organización de una solución condiciona cuánto tarda un desarrollador en encontrar una regla, ejecutar sus pruebas y comprender el efecto de un cambio. Por eso el árbol de carpetas es más que una decisión estética: muestra qué código se mantiene junto, qué pruebas lo acompañan y dónde quedan registradas las razones de una decisión. Para AceriaApp necesitamos una organización que permita enseñar estados progresivos sin que una práctica destruya la anterior. Esa finalidad docente puede coexistir con buenas convenciones profesionales, aunque no debe confundirse con la estructura obligatoria de un sistema empresarial. Tendremos que elegir cómo agrupar los repositorios, cómo distribuir el código dentro de cada estado y qué nombres ayudan a distinguir responsabilidades. Un proyecto bien organizado no garantiza buenas reglas de negocio, pero reduce la fricción para identificarlas y modificarlas sin efectos inesperados.

```
Tres decisiones de organización:

1. Estructura de repositorios  → monorepo o polyrepo.
2. Estructura de carpetas      → src, tests, docs.
3. Convenciones de nombres     → proyectos, ensamblados, espacios de nombres.
```

La organización se examina en tres escalas. La elección de repositorios determina dónde se versionan los cambios; las carpetas muestran dónde buscar código, pruebas y decisiones; los nombres ayudan a reconocer la finalidad de cada proyecto. No hay que utilizar las tres como listas de reglas independientes: deben describir una misma división de responsabilidades comprensible para todo el equipo.

La organización no es una cuestión estética: tiene consecuencias medibles sobre el tiempo de incorporación de nuevos desarrolladores, sobre la facilidad para extraer servicios y sobre la capacidad de automatizar tareas. Un proyecto mal organizado tarda semanas en incorporar a un nuevo desarrollador. Un proyecto bien organizado tarda días. La diferencia no está en la complejidad técnica

del código, sino en la previsibilidad de su estructura. El curso aplica estas tres decisiones al proyecto AceriaApp de forma explícita y trazable.

Un repositorio organizado debe permitir encontrar el código correcto sin conocer previamente a su autor. Esta propiedad se comprueba con tareas concretas: localizar un endpoint, identificar su test y encontrar el ADR que explica su existencia. La estructura solo aporta valor cuando se mantiene de forma constante; si cada equipo inventa un nombre distinto para las mismas responsabilidades, el coste de navegación aumenta con cada servicio. En AceriaApp la unidad pedagógica es el punto, y cada punto conserva una solución independiente con src, tests y docs. Esa organización no es idéntica a la estructura de un producto que mantiene una única solución evolutiva, pero permite repetir experimentos y comparar estados sin reemplazar entregas anteriores. Debe evitarse mezclar herramientas internas de verificación dentro de carpetas que el alumno interpreta como código o material lectivo.

Monorepo frente a polyrepo

Un repositorio único facilita consultar conjuntamente las interfaces, sus implementaciones y las pruebas que las protegen. Esto resulta especialmente cómodo cuando una aplicación está comenzando o cuando varias capacidades comparten todavía el mismo ritmo de desarrollo. La alternativa de varios repositorios da a los equipos control independiente sobre permisos, políticas, versiones y pipelines, pero añade tareas de coordinación si un cambio afecta a contratos comunes. No hay una regla universal que haga mejor un monorepo o un polyrepo. Lo importante es que la ubicación del código no contradiga las fronteras arquitectónicas: un monorepo puede contener servicios desplegables por separado, y varios repositorios pueden esconder un despliegue inseparable. AceriaApp conserva un repositorio docente y etapas de referencia independientes; eso favorece comparar cambios sin presentar las carpetas como equivalentes a microservicios.

```
Monorepo:

AceriaApp/
├── modulo-1/
│   ├── 1.1-monolito-vs-microservicios/
│   ├── 1.2-ventajas-inconvenientes/
│   └── ...

Polyrepo:

AceriaApp.Ordenes/     → repositorio independiente.
AceriaApp.Inventario/  → repositorio independiente.
AceriaApp.Clientes/    → repositorio independiente.
```

El esquema compara un repositorio que agrupa las etapas del curso con varios repositorios hipotéticos por capacidad. La primera alternativa favorece la consulta de toda la evolución; la segunda ofrece políticas de mantenimiento potencialmente independientes. Ninguna distribución demuestra por sí sola cuántos procesos se ejecutan. La estrategia de Git y la estrategia de despliegue son decisiones distintas que deben coordinarse.

El monorepo es la elección del curso porque el equipo es pequeño y los servicios cambian juntos durante la fase de aprendizaje. Un equipo de dos personas que trabaja sobre tres servicios no se beneficia de la independencia de permisos del polyrepo, pero sí se beneficia de la búsqueda global y de la refactorización entre servicios del monorepo. La decisión se documenta en un ADR y se revisa cuando el equipo crece.

Un monorepo puede tener integración continua selectiva: modificar únicamente el servicio de Órdenes no obliga necesariamente a volver a publicar Inventario. La independencia se consigue mediante detección de cambios, límites de paquetes y contratos estables; el número de repositorios no la garantiza por sí solo. Un polyrepo introduce otro tipo de trabajo, como sincronizar dependencias

comunes, revisar varias solicitudes de cambio y gestionar permisos de acceso separados. En un curso con un equipo pequeño y servicios que evolucionan conjuntamente, mantener las versiones por punto en un único repositorio es una decisión práctica. La elección debe revisarse si los equipos necesitan permisos diferenciados o cadencias de despliegue propias. Conviene documentar quién puede modificar los contratos compartidos y cómo se verifican, sea cual sea la topología de repositorios elegida.

El número de repositorios no determina cuántos servicios existen ni cuántos pueden desplegarse por separado. Un monorepo puede albergar decenas de servicios autónomos si las compilaciones y publicaciones están correctamente acotadas por dependencias. Un polyrepo, por el contrario, puede contener servicios tan acoplados que necesiten actualizarse al mismo tiempo. La diferencia se encuentra en la gestión del código, los permisos y los flujos de cambio. El monorepo facilita editar varios componentes en una misma operación y validar un contrato común; el polyrepo simplifica algunas fronteras de permisos y puede reducir el volumen de trabajo visible para cada equipo.

Para AceriaApp, el monorepo ofrece una ventaja didáctica concreta: cada punto conserva una solución completa, independiente de las demás, de modo que el alumno puede abrir el estado 1.1 y compararlo con 1.5. Esa duplicación intencional sirve para enseñar la progresión. En un proyecto de producción no tendría por qué conservarse una copia íntegra del código por cada cambio: normalmente se utilizaría el historial de Git, etiquetas y releases. Es importante reconocer la diferencia entre una estructura pensada para aprender y una estructura pensada para desplegar. Confundirlas lleva a creer que el mantenimiento habitual exige duplicar proyectos constantemente.

Criterio	Monorepo	Polyrepo
Cambios que afectan a varios servicios	Más sencillos de revisar juntos	Requieren coordinar repositorios
Gestión de permisos	Puede requerir reglas por carpetas	Fronteras naturales por repositorio
Visibilidad de dependencias	Puede analizarse centralmente	Precisa inventario entre repositorios
Publicación independiente	Posible con pipelines selectivos	Natural, aunque no garantizada
Curso AceriaApp	Facilita comparar estados por punto	Añadiría coordinación sin valor formativo inicial

La elección de repositorio debe reflejarse en los pipelines. En un monorepo no tiene sentido volver a desplegar Inventario por un cambio exclusivo en documentación de Clientes, salvo que exista una razón explícita. En la práctica del módulo 1 se utiliza una solución independiente por punto para comprobar el aprendizaje. Los workflows de prueba pueden verificar esas soluciones sin alterar la estructura de trabajo del estudiante. Así se distingue una convención del repositorio de una propiedad arquitectónica del sistema desplegado.

Estructura de carpetas dentro de un repositorio

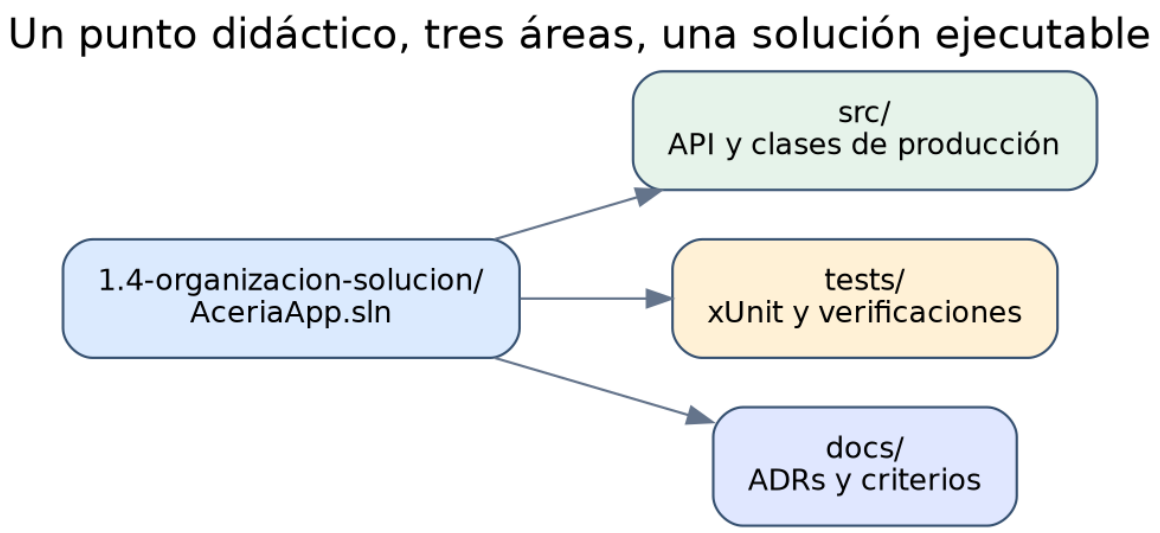


Figura. Separación del código, las pruebas y la documentación.

Dentro de una etapa, separar `src/`, `tests/` y `docs/` ayuda a reconocer tres tipos de responsabilidad. El primero contiene aquello que se distribuye como aplicación o biblioteca; el segundo reúne comprobaciones que deben poder ejecutarse y fallar cuando una regla se rompe; el tercero guarda decisiones que requieren contexto humano, como alternativas descartadas o límites de un módulo. Esta estructura solo funciona si las relaciones son claras. Por ejemplo, un test de unidad puede explicar el comportamiento esperado de una orden, pero no sustituye un documento que justifica por qué Producción debería tener propiedad sobre esa regla. Tampoco conviene duplicar sin necesidad documentos extensos en cada proyecto. En el curso, las carpetas por punto mantienen una fotografía completa para que el alumnado pueda recuperar un estado anterior y contrastarlo con el siguiente.

```
Estructura de un punto del curso:

1.1-monolito-vs-microservicios/
├── README.md
├── AceriaApp.sln
├── src/
│   └── AceriaApp.Monolito.Api/
├── tests/
│   ├── AceriaApp.Monolito.Tests/
│   ├── AceriaApp.Metricas.Tests/
│   └── AceriaApp.Diseno.Tests/
└── docs/
    ├── ADR-001-monolito-vs-microservicios.md
    ├── ADR-002-metricas-acoplamiento.md
    └── ADR-003-principios-diseno.md
```

La etapa contiene `AceriaApp.sln`, proyectos en `src/`, pruebas en `tests/` y documentos en `docs/`. Esa proximidad permite reconstruir qué se añadió en un momento del curso. Los nombres de las pruebas y documentos del dibujo son referencias para estudiar la organización; si un archivo no existe en el estado concreto que estás abriendo, debe seguirse el README de esa etapa y no crear archivos únicamente para imitar el dibujo.

La separación en tres carpetas de primer nivel es una convención, no una obligación técnica. Su valor está en la previsibilidad: cualquier desarrollador que abra el repositorio sabe dónde buscar el código, dónde buscar los tests y dónde buscar la documentación. Esta previsibilidad reduce el tiempo de incorporación y facilita la automatización de tareas de compilación y despliegue.

La convención src/tests/docs distingue tipos de artefactos, pero no sustituye la separación lógica entre capacidades de negocio. Dentro de src se pueden agrupar los proyectos por servicio, y dentro de tests se colocan pruebas unitarias, de integración y de contrato según las necesidades. Los ADR explican decisiones y sus alternativas; no deben usarse como sustitutos de ejemplos que el alumno necesita ejecutar. En AceriaApp, cada punto representa un estado reproducible y la separación evita que pruebas de organización se confundan con archivos de infraestructura. Cuando el alumno abre la solución de 1.3 encuentra los módulos de los puntos anteriores ya activos y las clases nuevas comentadas. Esa diferencia entre código heredado y código por activar debe constar tanto en el README como en las instrucciones de prácticas. Sin esa correspondencia, la estructura parece correcta pero el itinerario falla.

Una estructura consistente reduce el coste de encontrar información, pero también ayuda a detectar errores de dependencia. En `src` deben encontrarse los componentes de producción: proyectos ejecutables y bibliotecas que intervienen en la aplicación. En `tests` deben residir las pruebas automáticas que usan y verifican esos componentes. En `docs` conviene guardar decisiones, límites de contexto y procedimientos que explican por qué el sistema está construido de una manera determinada. Los archivos `.sln` y `.csproj` expresan relaciones técnicas; los ADR documentan razones. Ninguno de esos elementos sustituye al otro.

Una práctica habitual que genera confusión consiste en mezclar código de verificación con ejemplos para ejecutar manualmente. Si un test automatizado necesita un `DbContext` real, debe preparar sus dependencias y comprobar el resultado dentro del proyecto de pruebas correspondiente. Si se trata de una guía de uso, debe documentar el comando, la ruta y el comportamiento esperado. En el curso AceriaApp, cada punto contiene artefactos preparados con bloques comentados para que el alumno los active progresivamente. El lugar correcto para explicar esa activación es el documento de prácticas; el código puede contener marcadores didácticos, pero los informes internos de CI no deben aparecer mezclados con el material que se entrega al estudiante.

La carpeta `docs` es particularmente importante porque conserva el contexto que el código no puede expresar con facilidad. Un `ProjectReference` indica que un proyecto necesita otro; no explica si esa dependencia es aceptable dentro del dominio ni qué alternativa se evaluó. Un ADR sí puede describir el problema y las consecuencias. Para revisar una solución debe recorrerse el conjunto completo: localizar el requisito, encontrar la clase responsable, identificar los tests que verifican su comportamiento y leer la decisión documentada. Esta revisión detecta mejor los huecos que una comprobación limitada a la existencia de carpetas.

La estructura didáctica de cinco soluciones completas es, por tanto, un mecanismo de comparación. El alumno debe observar no solo los archivos añadidos, sino también qué responsabilidades no han cambiado desde el punto anterior. Si las nuevas clases de métricas o buenas prácticas obligaran a reescribir el modelo de `Orden` sin justificación, la ampliación habría creado un acoplamiento inesperado. La separación `src/tests/docs` facilita esta inspección, pero la evidencia procede de revisar contenido y dependencias, no del nombre de las carpetas.

Convenciones de nombres de proyectos

Los nombres de proyectos sirven para comunicar de qué capacidad forman parte y qué función cumplen, no para codificar toda la arquitectura en una cadena interminable. `AceriaApp.Ordenes.Api` sugiere un host de HTTP relacionado con Órdenes; `AceriaApp.Ordenes.Domain` indica clases del modelo; `AceriaApp.Ordenes.Tests` identifica las comprobaciones asociadas. El sufijo aporta una pista útil, pero no demuestra que el código esté en la capa correcta: una biblioteca llamada `Domain` puede seguir referenciando infraestructura indebidamente. El valor de la convención se comprueba al recorrer una solución nueva: otros miembros del equipo deben localizar una responsabilidad sin necesidad de memorizar la historia del proyecto. Los nombres deben ser estables, comprensibles en los contextos del negocio y suficientemente concretos para no confundirse entre servicios.

```
Convención de nombres:

AceriaApp.Ordenes.Api        → API del servicio de órdenes.
AceriaApp.Ordenes.Domain     → dominio del servicio de órdenes.
AceriaApp.Ordenes.Tests      → tests del servicio de órdenes.
AceriaApp.Inventario.Api     → API del servicio de inventario.
AceriaApp.Monolito.Api       → API del monolito de referencia.
```

Los nombres distinguen capacidad y tipo de componente. `Ordenes.Api` e `Inventario.Api` indican hosts posibles; `Ordenes.Domain` y `Ordenes.Tests` señalan modelo y pruebas. La convención es informativa: el contenido y las referencias de cada proyecto deben comprobarse por separado, especialmente cuando una biblioteca hereda dependencias de etapas anteriores.

La convención facilita la búsqueda por nombre, la configuración de pipelines y la organización de la solución en el panel Solution Explorer. Un proyecto mal nombrado rompe la uniformidad y obliga a los desarrolladores a recordar excepciones. La convención se aplica también a los espacios de nombres: el espacio de nombres raíz de cada proyecto coincide con el nombre del proyecto. Esta coherencia entre nombre de proyecto y espacio de nombres facilita la resolución de tipos y la lectura del código.

El patrón de nombre AceriaApp.Componente.Tipo se puede evaluar de forma automática, aunque la validación de una cadena no demuestra que el contenido del proyecto corresponda a su etiqueta. Un proyecto llamado Ordenes.Domain podría contener acceso directo a SQL Server, lo que violaría su responsabilidad pese a cumplir la convención. Las reglas son útiles para evitar errores accidentales y favorecer herramientas de construcción, pero necesitan acompañarse de revisiones de dependencias. También hay decisiones legítimas que requieren nombres con más de tres segmentos, como adaptadores específicos; una regla rígida debe documentar sus excepciones. En el ejercicio, ValidadorConvencionNombres rechaza nombres que no tienen tres segmentos y tipos fuera de una lista cerrada. Sus tests comprueban ese contrato docente. La decisión de utilizar esa lista se registra en el ADR para que sea revisable si cambian los tipos admitidos.

Una convención de nombres permite inferir una función a partir de un identificador, pero su valor depende de que el patrón refleje la estructura real. El formato `AceriaApp.Componente.Tipo` es sencillo de aplicar a `AceriaApp.Ordenes.Api`; también obliga a decidir cómo se representan componentes que contienen varias palabras y qué tipos se permiten. Un validador debe tratar esos casos de forma explícita. Si acepta una cadena con tres partes pero permite que `Componente` esté vacío, podría aceptar `AceriaApp..Api`, que no comunica ninguna responsabilidad. Del mismo modo, el hecho de empezar por `AceriaApp.` no garantiza que exista un nombre válido detrás del prefijo.

Los namespaces merecen una revisión independiente. En C#, una carpeta no obliga técnicamente a declarar un namespace específico: la convención se mantiene mediante disciplina y herramientas de análisis. Por ello la correspondencia entre ruta de archivo y namespace es una recomendación de organización, no una restricción automática del compilador. Un nombre válido tampoco implica que las referencias entre proyectos respeten la arquitectura. Una API puede llamarse correctamente `AceriaApp.Inventario.Api` y depender de una clase privada de Órdenes, reproduciendo el problema que se quería evitar.

Casos de frontera para un validador de convenciones:

- `AceriaApp.Ordenes.Api`: patrón permitido y componente explícito.
- `AceriaApp.Ordenes.Controlador`: tipo fuera de la lista acordada.
- `AceriaApp.Ordenes`: falta el tipo del proyecto.
- `AceriaApp..Api`: contiene un componente vacío; requiere una regla adicional para rechazarse.
- `OtraApp.Ordenes.Api`: no pertenece al prefijo de la solución.

En el punto 1.4 se construye un `ValidadorConvencionNombres` con reglas deliberadamente acotadas: comprueba prefijo, número de partes y tipo permitido. Los tests incluidos demuestran esas comprobaciones; no debe afirmarse que cubren todos los nombres inválidos posibles. Esta limitación

es formativa porque enseña a definir una especificación verificable y después ampliarla mediante casos de borde. La calidad de una convención se evalúa por la claridad de la estructura que produce, no por la cantidad de reglas que se agreguen a un validador.

Convenciones de nombres de espacios de nombres

Los espacios de nombres son una herramienta de organización del código C# y colaboran con la lectura, la navegación y la resolución de tipos. Cuando su jerarquía refleja el proyecto y sus áreas funcionales, resulta más fácil reconocer de dónde procede una clase y detectar dependencias que atraviesan límites no deseados. Sin embargo, un `namespace` no crea aislamiento de procesos ni impide por sí solo referencias incorrectas: esas restricciones deben expresarse mediante proyectos, visibilidad y pruebas de arquitectura. En AceriaApp se utilizan nombres que relacionan dominio, API e infraestructura de forma reconocible. El alumnado debe distinguir la convención de su mecanismo técnico: el compilador acepta namespaces que no coinciden con la carpeta, aunque una organización consistente reduzca confusiones. La consistencia que buscamos es semántica, no una obediencia ciega a la estructura de directorios.

```
namespace AceriaApp.Monolito.Api.Dominio;

public class Orden
{
    public Guid Id { get; set; }
}
```

El ejemplo utiliza `AceriaApp.Monolito.Api.Dominio` para agrupar una clase `Orden`. El namespace permite referenciar el tipo desde otros archivos, pero no protege sus reglas ni fuerza una dependencia correcta. El objetivo es reconocer la diferencia entre clasificación del código y separación de responsabilidades, que necesitará proyectos y límites verificables.

La convención de espacios de nombres debe ser consistente con la estructura de carpetas del proyecto. Un archivo en la carpeta Dominio debe declarar el espacio de nombres ...Dominio. Un archivo en la carpeta Infraestructura debe declarar el espacio de nombres ...Infraestructura. Esta correspondencia facilita la navegación y evita conflictos de nombres. La convención se aplica también a los tests: cada proyecto de tests declara un espacio de nombres que termina en Tests.

La correspondencia entre carpeta y espacio de nombres facilita navegación y búsqueda, pero el compilador de C# no exige que coincidan. Esa diferencia es importante: una incoherencia puede compilar correctamente y seguir siendo difícil de mantener. La verificación debe inspeccionar el namespace real del archivo y compararlo con el prefijo del proyecto, no asumir que el nombre de carpeta lo determina. En AceriaApp, Orden pertenece a Dominio, AceriaDbContext a Infraestructura y las clases de métricas a Metricas, todas dentro del mismo proyecto monolítico. Esta estructura es apropiada como referencia inicial; más adelante pueden desplazarse algunas clases a bibliotecas separadas sin cambiar necesariamente sus contratos. Al renombrar una carpeta, conviene revisar las referencias y los usings para evitar confusiones. Los analizadores del IDE pueden ayudar a mantener una convención uniforme.

Separación entre código de producción y tests

Una solución fácil de mantener permite ejecutar pruebas relacionadas con un comportamiento sin modificar las dependencias de producción. Situar los proyectos xUnit en `tests/` contribuye a esa separación y permite expresar con claridad qué se está comprobando: una regla de dominio, una integración con SQL Server o una ruta HTTP completa. Sus diferencias importan. Un test con un doble puede mostrar que el caso de uso reaccionó correctamente a una respuesta, mientras que una prueba con EF Core y SQL Server real demuestra el comportamiento de persistencia del proveedor. Ninguna de las dos sustituye a la otra. Los nombres de los proyectos ayudan a encontrarlos, pero la garantía

nace de las aserciones y del entorno ejecutado. El alumnado no debería limitarse a obtener una compilación correcta: debe entender qué comportamiento protege cada batería de pruebas.

```
Correspondencia entre producción y tests:

AceriaApp.Monolito.Api     → AceriaApp.Monolito.Tests
AceriaApp.Metricas         → AceriaApp.Metricas.Tests
AceriaApp.Diseno           → AceriaApp.Diseno.Tests
```

La correspondencia entre aplicación y tests facilita saber dónde buscar la prueba de una regla o de un servicio. La relación real se establece con referencias de proyecto y aserciones, no con una coincidencia textual perfecta de nombres. En particular, las pruebas que necesitan SQL Server deben preparar explícitamente el motor antes de afirmar que han demostrado persistencia.

La correspondencia uno a uno entre producción y tests no es obligatoria pero sí recomendable. Un proyecto de producción sin proyecto de tests correspondiente es una señal de deuda técnica. Un proyecto de tests sin proyecto de producción correspondiente es una señal de código obsoleto. La convención facilita la detección de ambas señales. La separación entre src/ y tests/ también facilita la configuración de la solución para que los proyectos de tests no se incluyan en el artefacto publicado.

La relación entre código de producción y tests debe responder a los límites funcionales y a la velocidad de las pruebas, no a una obligación aritmética de tener exactamente un proyecto de tests por biblioteca. Un servicio puede contar con pruebas unitarias, de integración y de contrato en proyectos diferentes. En el módulo 1 se crean proyectos especializados para métricas, límites, organización y buenas prácticas aunque sus clases de producción estén en la misma API monolítica. Esa elección hace visible el aprendizaje progresivo, pero no debe presentarse como una correspondencia perfecta uno a uno. Al ejecutar una solución acumulativa se esperan los tests anteriores más los nuevos, sin duplicar métodos de prueba. Es necesario distinguir los tests básicos de los retos opcionales: los recuentos 1, 9, 13, 18 y 23 son del itinerario básico. Las pruebas opcionales aumentan el número únicamente si el alumno las activa.

Documentación como parte del repositorio

Las decisiones arquitectónicas necesitan memoria. Cuando un equipo decide mantener una API monolítica o separar un módulo, la explicación puede perderse si solo se conserva el resultado en carpetas. Un ADR registra el problema, las alternativas consideradas, la decisión tomada y las consecuencias conocidas. Con el tiempo, esa información permite reconocer si las condiciones han cambiado y si conviene revisar la elección. La documentación también tiene límites: una guía que describe una ruta inexistente o una base no utilizada deja de ayudar. Por eso debe mantenerse junto al código y comprobarse cuando cambian contratos o procedimientos. En AceriaApp los documentos de decisiones acompañan las etapas, pero no sustituyen el código ni su validación. Una buena documentación permite que el alumno comprenda por qué se incorporó una pieza y cómo contrastar su función.

```
Contenido de docs/:

docs/
├── ADR-001-monolito-vs-microservicios.md
├── ADR-002-metricas-acoplamiento.md
├── ADR-003-principios-diseno.md
├── analisis-ventajas-inconvenientes.md
└── principios-diseno.md
```

Los ADR conservan decisiones y sus contextos; los documentos de análisis registran criterios de comparación. Al recorrer la carpeta, el alumno debería poder explicar qué problema resuelve cada archivo. La documentación no debe acumularse por cumplir una cuota: una nota útil que permita verificar una decisión vale más que varias páginas de formulaciones repetidas.

La documentación se organiza por tipo: los ADRs con prefijo ADR-NNN-, los análisis con nombre descriptivo, las guías con prefijo guia-. Esta organización permite localizar rápidamente el documento relevante. La documentación no duplica el código: describe las decisiones y las razones, no la implementación. Un documento que describe línea por línea lo que hace el código es un documento que queda obsoleto en el primer cambio. Un documento que describe por qué se tomó una decisión es un documento que permanece válido aunque el código cambie.

Un ADR es útil cuando explica por qué se eligió una decisión frente a alternativas reales y bajo qué condiciones debería revisarse. Una lista de clases o comandos pertenece al README o a la práctica, mientras que la historia de una decisión permanece en el ADR. La documentación viva debe evitar dos extremos: copiar cada línea de código, que se vuelve obsoleto enseguida, o redactar afirmaciones vagas que nadie puede contrastar. En AceriaApp, ADR-001 registra el inicio monolítico y las siguientes decisiones explican métricas, límites, organización y buenas prácticas. El alumnado debe comprobar que los ADR heredados no cambian al pasar de un punto al siguiente. La verificación mediante hashes o una comparación textual ofrece una evidencia más precisa que confiar únicamente en el nombre del archivo. Esa disciplina conserva el historial de decisiones mientras el código crece.

No toda la documentación debe generarse desde el código. OpenAPI puede describir rutas, parámetros y tipos de respuesta porque esos elementos forman parte del contrato implementado. En cambio, el motivo por el que se eligió un monolito modular en lugar de microservicios no puede inferirse de los archivos `.cs`. Un ADR conserva esa justificación, los supuestos que se aceptaron y los riesgos conocidos. El valor del ADR no reside en repetir el contenido de los endpoints, sino en permitir que un desarrollador futuro comprenda qué decisión está revisando y por qué se tomó originalmente.

Un buen historial de decisiones debe conservar cambios de criterio. Supongamos que, en una etapa posterior, AceriaApp necesita extraer Inventario para escalarlo de forma independiente. No conviene borrar el ADR-001 y sustituirlo silenciosamente por la nueva elección. La documentación debería registrar una nueva decisión que explique qué condiciones cambiaron, qué alternativas se consideraron y qué consecuencias tiene el nuevo diseño. De esa manera, el histórico permite comprender la evolución del sistema. El documento anterior sigue siendo válido como evidencia de lo que se sabía en el momento inicial.

En las prácticas, los archivos `ADR-001` a `ADR-005` permiten identificar decisiones diferentes: arquitectura inicial, métricas, principios de límites, organización y buenas prácticas. No son equivalentes a un manual de comandos. Al consultar el repositorio, el alumno debe poder distinguir una instrucción operativa para ejecutar `dotnet test` de una razón arquitectónica que condicionará módulos posteriores. Esta separación mejora la claridad y evita que una decisión acabe enterrada entre listados de pasos. También permite comprobar qué documentos permanecen iguales cuando se avanza de una solución a la siguiente.

La documentación viva exige mantenimiento, pero no significa reescribir todo en cada cambio. Si la estructura de proyectos permanece estable, el ADR puede conservarse. Si el contrato HTTP cambia, la descripción OpenAPI debe actualizarse y las pruebas de compatibilidad deben indicar qué clientes se ven afectados. El criterio de actualización deriva de la clase de conocimiento documentado. Esta distinción ofrece una regla práctica para mantener `docs` relevante sin duplicar el código ni llenar el repositorio de textos que se vuelven obsoletos inmediatamente.

Convenciones de ramas y commits

Una rama permite aislar cambios en desarrollo, y un commit proporciona un punto concreto que puede inspeccionarse y compararse. Las convenciones de nombres facilitan identificar si se está corrigiendo un error, incorporando una funcionalidad o documentando una decisión, pero no obligan a un estilo arquitectónico ni garantizan calidad. Lo importante es que los cambios tengan un alcance comprensible y que sus pruebas puedan relacionarse con ese alcance. Para un alumno, las etapas acumulativas de AceriaApp son estados de referencia: puede comparar su trabajo con la etapa

anterior sin necesidad de fabricar commits vacíos o reconstruir el historial del profesor. En un equipo real, la granularidad y nomenclatura de commits se acuerdan según la estrategia de integración. La trazabilidad se consigue comparando contenidos efectivos y decisiones, no únicamente escribiendo un título formal.

```
Convenciones de ramas:

feature/nueva-funcionalidad
fix/correccion-error
docs/actualizacion-adr
refactor/extraccion-servicio

Convenciones de commits:

Punto 1.1: estructura inicial del proyecto AceriaApp
Punto 1.2: análisis de ventajas e inconvenientes y métricas de acoplamiento
Punto 1.3: principios de diseño y validación de límites de servicio
Punto 1.4: organización de la solución y convenciones de nombres
```

Las ramas de ejemplo expresan tipos de trabajo habituales; los commits enumeran hitos del aprendizaje. Para comparar cambios debe utilizarse un historial real, por ejemplo `git diff` entre dos commits existentes. Es incorrecto asumir que los textos del dibujo son etiquetas Git disponibles en cualquier clon del curso. Los directorios de etapas sí permiten comparar sus archivos aunque no haya commits didácticos creados por cada alumno.

La convención de commits permite comparar el estado del proyecto entre dos puntos consecutivos con git diff Punto1.1 Punto1.2. Esta capacidad es útil para el alumno que quiere ver exactamente qué se añadió en cada punto. La convención de ramas permite aislar el trabajo en curso sin afectar a la rama principal. Ambas convenciones son recomendaciones, no obligaciones: su valor está en la trazabilidad que proporcionan.

Los nombres de ramas y commits son convenciones de comunicación, no mecanismos que garanticen calidad del código. Un commit identificado con un punto permite localizar un conjunto de cambios, pero solo una comparación real del árbol muestra qué archivos se añadieron o modificaron. En un repositorio previamente publicado, clonar una versión final puede dejar el árbol limpio antes de iniciar las prácticas: ejecutar git commit sin cambios no creará un commit nuevo. El alumno debe interpretar ese resultado correctamente, sin modificar archivos innecesariamente para forzar una confirmación. Los puntos separados dentro de PROYECTO proporcionan fotografías acumulativas; los commits docentes de cada punto pueden crearse cuando la sesión se reconstruye desde un repositorio de inicio o una rama de trabajo. Los materiales no deben confundir los commits históricos de publicación del instructor con los que produce el alumno.

Estructura de una solución con varios servicios

Cuando varias capacidades se despliegan por separado, es útil que cada una mantenga el código y las pruebas de sus reglas más próximas a su responsabilidad. Órdenes puede reunir un host HTTP, un dominio y una adaptación de persistencia; Inventario puede hacer lo mismo, sin obligar a que sus entidades compartan una biblioteca genérica. Así los cambios permanecen localizados y cada equipo conoce qué pruebas ejecutar antes de publicar. Se debe evitar, sin embargo, multiplicar proyectos solo por reproducir un diagrama. Un contrato compartido mínimo puede estar justificado; una enorme biblioteca común con reglas de todos los servicios suele reintroducir el acoplamiento que se intentaba eliminar. AceriaApp evolucionará hacia esta organización de forma gradual, y cada paso debería conservar una relación clara entre propiedad de reglas, dependencias y pruebas.

```
Estructura con varios servicios:

src/
├── AceriaApp.Ordenes.Api/
```

```
├── AceriaApp.Ordenes.Domain/
├── AceriaApp.Ordenes.Infrastructure/
├── AceriaApp.Inventario.Api/
├── AceriaApp.Inventario.Domain/
└── AceriaApp.Inventario.Infrastructure/

tests/
├── AceriaApp.Ordenes.Tests/
└── AceriaApp.Inventario.Tests/
```

El árbol futuro separa dominios y adaptadores de Órdenes e Inventario. Sus pruebas aparecen por capacidad y deberían utilizar la infraestructura real solo cuando su nivel de comprobación lo necesite. El esquema no propone compartir una base de datos por el mero hecho de compartir repositorio, ni crear bibliotecas duplicadas que no aporten una frontera verificable.

La estructura con varios servicios mantiene la separación entre src/ y tests/ y añade la separación entre servicios. Cada servicio tiene su propio conjunto de proyectos y su propio conjunto de tests. Esta estructura permite compilar y desplegar un servicio de forma independiente aunque todos los servicios vivan en el mismo repositorio. La solución se convierte en el punto de entrada común para compilar y probar todos los servicios.

La separación por servicio aumenta el número de proyectos, y por ello conviene evitar referencias cruzadas indiscriminadas. Ordenes.Api puede depender de una capa Application que expresa casos de uso, pero no debe acceder al proyecto Infrastructure de Inventario para leer su base de datos. La solución común de Visual Studio es una agrupación de compilación y navegación; su existencia no obliga a ejecutar todos los proyectos en el mismo proceso. Para desplegar de manera autónoma, cada servicio debe poder publicarse con las dependencias necesarias y una configuración propia. En AceriaApp la estructura futura con Ordenes e Inventario es ilustrativa. El código del módulo 1 todavía ejecuta un único host monolítico, y el objetivo de la práctica de organización es validar convenciones sobre nombres, no crear efectivamente seis servicios distribuidos. Esa precisión evita que el alumno espere proyectos inexistentes.

Una solución con varios servicios puede organizarse en proyectos de API, dominio e infraestructura, pero debe evitar una biblioteca compartida que concentre todos los modelos de negocio. Si Órdenes e Inventario referencian el mismo ensamblado con entidades mutables, un cambio interno de uno puede imponer una actualización al otro. Esa dependencia no desaparece porque los proyectos estén en carpetas separadas. Es preferible compartir convenciones muy estables y bibliotecas transversales pequeñas, mientras cada servicio mantiene su modelo de dominio bajo su propia responsabilidad. El contrato de integración se comparte como especificación, no necesariamente como una entidad de persistencia común.

La dirección de las referencias también importa. La API puede conocer la capa de aplicación para invocar casos de uso; la infraestructura puede implementar interfaces definidas por una capa interna. Un modelo de dominio que depende directamente de ASP.NET Core, SQL Server y un cliente HTTP concreto tendría dificultades para evolucionar de forma autónoma. La arquitectura hexagonal que se estudiará después formaliza esta separación mediante puertos y adaptadores, pero el punto de partida es reconocer qué conocimiento pertenece al negocio y cuál al mecanismo de entrada o salida.

Lectura de la estructura futura de AceriaApp: si existen `AceriaApp.Ordenes.Domain` y `AceriaApp.Inventario.Domain`, deben corresponder a conceptos diferentes aunque ambos utilicen tipos `Guid` o `DateTime`. Cada servicio puede tener proyectos de tests dedicados, pero todos esos tests no deben depender de que los demás servicios estén activos. Las pruebas de integración remota se ejecutarán aparte, con una frontera HTTP o de mensajería claramente definida. Esta separación mejora el diagnóstico: un test de dominio fallido identifica una regla, mientras que un escenario E2E fallido puede implicar base de datos, red o configuración.

Los diagramas de dependencias ayudan a detectar ciclos antes de que se conviertan en un impedimento. La existencia de un ciclo de `ProjectReference` impide compilar o fuerza soluciones

artificiales; en un sistema distribuido, un ciclo de llamadas síncronas puede compilar pero producir una dependencia operativa igual de difícil de sostener. Por ello la organización del repositorio solo es la superficie visible de una estructura más profunda: la propiedad de las reglas, los contratos y los datos. La transición hacia servicios independientes tiene éxito cuando estas responsabilidades ya están claras.

Aplicación de la organización al proyecto del curso

La organización utilizada por el curso responde a dos necesidades que conviene distinguir. Como material de formación, cada punto dispone de una solución de referencia independiente, para que el alumno pueda reconstruir el resultado y consultar estados anteriores. Como aplicación .NET, cada estado separa código de producción, pruebas y documentación, y utiliza nombres que permiten reconocer responsabilidades. Esta estructura ayuda a verificar el progreso, pero no implica que los equipos profesionales deban almacenar obligatoriamente una copia completa por ejercicio. Lo transferible al trabajo real es la disciplina: cambios localizables, convenciones comprensibles y tests que permitan comprobar el estado del sistema. La transición futura a varios servicios se hará cuando los límites de negocio y las dependencias lo justifiquen, no porque el árbol de carpetas pueda reproducirse fácilmente.

```
Aplicación al proyecto:

Monorepo                  → AceriaApp/ con módulos y puntos.
src/, tests/, docs/       → separación dentro de cada punto.
AceriaApp.Componente.Tipo → convención de nombres.
Namespace = carpeta       → convención de espacios de nombres.
Punto N.M: descripción    → convención de commits.
```

La tabla de correspondencias resume decisiones para orientar la lectura de las soluciones. El patrón `AceriaApp.Componente.Tipo` y las carpetas `src/`, `tests/` y `docs/` son convenciones de legibilidad. Lo que acredita una separación efectiva es poder localizar, cambiar y probar una responsabilidad sin reescribir el resto del proyecto.

La aplicación de estas convenciones prepara el proyecto para la extracción de microservicios en módulos posteriores. Cuando un módulo se extraiga como servicio, mantendrá la misma estructura de carpetas y las mismas convenciones de nombres. La coherencia entre el monolito modular y los microservicios facilita la transición y reduce el coste de la migración. El punto 1.4 cierra el bloque de fundamentos del módulo 1 y prepara el terreno para el punto 1.5, donde se estudiarán las buenas prácticas de desarrollo de microservicios.

La convención debe sobrevivir a la evolución del curso sin convertirse en un obstáculo. Al añadir un Gateway, un servicio de eventos o adaptadores de infraestructura pueden aparecer tipos de proyecto que no estén previstos en una lista inicial de cinco valores. El validador debe recibir esa lista como configuración, y sus tests han de mostrar tanto nombres aceptados como nombres rechazados. CatalogoProyectos clasifica cadenas de nombres; no examina el sistema de archivos ni infiere automáticamente la composición de una solución. La práctica debe comunicar esta limitación y explicar qué parte se automatizó realmente. También debe reservar un lugar claro a la documentación de dependencias: el nombre de un proyecto ofrece una pista sobre su función, pero la dirección de sus referencias indica qué módulos conoce en realidad. La organización correcta combina convenciones, dependencias explícitas y contratos versionados.

Caso de análisis — Seguir la evolución estructural de AceriaApp

Las soluciones independientes de los puntos del módulo 1 permiten observar cómo crece un proyecto sin perder el estado anterior. En el punto 1.1 existen un proyecto de API y un proyecto de tests. En el 1.2 se añade el proyecto de tests de métricas; en el 1.3 se añade el de diseño; en el 1.4 se incorpora

el de organización; y en el 1.5 se añade el de buenas prácticas. Las clases de cada ampliación se sitúan dentro del proyecto de API existente en carpetas especializadas. Por tanto, el número de proyectos crece a razón de uno por punto después del primero y alcanza seis en la última solución, no siete. Es necesario distinguir este recuento de la cantidad de carpetas y de la cantidad de clases, que siguen otro patrón.

Estado de la solución	Proyectos .NET acumulados	Nueva responsabilidad de estudio
Punto 1.1	2	API mínima y tests de `Orden`
Punto 1.2	3	Métricas de acoplamiento
Punto 1.3	4	Validación de límites de servicio
Punto 1.4	5	Validación de convenciones de nombres
Punto 1.5	6	Catálogo y validación de buenas prácticas

Un alumno que abre la solución 1.4 antes de realizar sus pasos no debería encontrarse con seis proyectos. Encontrará los heredados del estado anterior y un archivo preparado para la nueva etapa, que puede estar comentado. La documentación tiene que distinguir «estructura disponible al comenzar» y «estructura obtenida después de activar los elementos del punto». Esta precisión evita un error de secuencia: si la guía ordena crear un proyecto que ya existe o verificar uno que todavía no se ha incorporado, el estudiante queda bloqueado aun cuando el código final compile sin problemas.

La existencia de una carpeta por punto es una estrategia docente para comparar versiones; no representa necesariamente una arquitectura de producción apropiada. En un repositorio profesional el historial Git podría conservar los estados intermedios sin replicar por completo la solución. Aquí la duplicación facilita abrir y ejecutar cada etapa de manera aislada. La contraparte es el riesgo de inconsistencias: un archivo heredado del punto anterior no debería cambiar en el siguiente sin que exista una razón didáctica explícita. La comparación por hashes permite comprobar conservación exacta y detectar cambios inadvertidos.

También conviene separar distintos niveles de validación. El script que activa bloques comentados y ejecuta `dotnet build` confirma que el código de la etapa puede compilar. La suite xUnit comprueba reglas particulares. Los escenarios E2E con SQL Server Express y HTTP ejercitan integración. La revisión manual del alumno confirma que sabe localizar el archivo, identificar el bloque correcto y comprender la consecuencia del cambio. Ninguna de esas pruebas sustituye por completo a las demás, aunque juntas proporcionan una evidencia mucho más sólida que limitarse a abrir Visual Studio y observar la ausencia de subrayados rojos.

El reto de organización introduce otra lección: una convención puede ser fácil de expresar y presentar excepciones. Un nombre con tipo no permitido es rechazado por el validador, pero algunos nombres mal formados podrían pasar si no se comprueba que el componente no está vacío. Este comportamiento no invalida el aprendizaje inicial; muestra cómo debe crecer una especificación automatizada mediante casos límite claros. En una evolución posterior, podría introducirse un analizador más completo sin sustituir el principio básico de usar nombres consistentes y comprensibles.

El resultado de este caso de análisis es poder recorrer cualquier estado del módulo, decir qué estaba disponible al comenzar y qué se incorpora en él, justificar las convenciones y señalar qué pruebas demuestran los resultados. La organización se aprende como un mecanismo de control de cambios y de responsabilidades, no como una mera decoración del árbol de carpetas.

Síntesis: organizar para poder cambiar

La organización de AceriaApp debe ayudar a entender su evolución y a detectar cambios que atraviesan límites no deseados. Un monorepo con carpetas por etapa es útil para la enseñanza y puede alojar proyectos bien separados; un polyrepo ofrece otras posibilidades de gobierno sin garantizar autonomía funcional. Los nombres, los namespaces y los ADR hacen visible la intención del diseño, mientras que los tests muestran qué comportamientos están efectivamente protegidos. En los próximos módulos estas convenciones seguirán siendo una ayuda, pero ninguna sustituirá la revisión de dependencias y contratos reales.

Punto 1.5 — Buenas prácticas

Módulo 1: Introducción a los microservicios

Objetivos de aprendizaje:

- Identificar las buenas prácticas aplicables al desarrollo de microservicios.
- Comprender la idempotencia como requisito de las operaciones expuestas.
- Comprender el versionado de contratos públicos y su relación con la evolución independiente.
- Comprender la gestión explícita de errores mediante códigos de respuesta y problem details.
- Añadir al proyecto AceriaApp un documento de buenas prácticas, una clase de validación de buenas prácticas y un cierre del módulo 1 con el resumen del estado del proyecto.

Elegir mecanismos que protegen un comportamiento real

Ya sabemos distinguir responsabilidades y organizar sus componentes. Ahora necesitamos evitar que esa estructura se degrade cuando el sistema acepta peticiones, recibe repeticiones, responde a errores o evoluciona. Para cada mecanismo estudiaremos el riesgo al que responde, su expresión en código y la comprobación que faltaría para considerarlo realmente implantado.

Buenas prácticas como marco de calidad

Una buena práctica no es una fórmula que pueda agregarse al final de cualquier clase: responde a un riesgo concreto que queremos controlar. En una aplicación que recibe órdenes, persiste datos y llega a hablar con otros servicios, esos riesgos aparecen en la superficie pública del contrato, en la ejecución de las operaciones, en la comunicación remota y en la evolución de las versiones. Cada ámbito requiere decisiones distintas. Publicar únicamente los campos necesarios evita filtrar el modelo interno; definir respuestas de error facilita que los consumidores actúen correctamente; gestionar peticiones repetidas protege los efectos de negocio; y mantener contratos compatibles reduce cambios simultáneos. A lo largo de este punto veremos ejemplos concretos y distinguiremos cuáles son conceptos que todavía no están programados en el monolito inicial. Una práctica se considera aplicada cuando su comportamiento puede observarse, no porque exista una clase con un nombre prometedor.

```
Categorías de buenas prácticas:

1. Contrato     → qué se expone y cómo.
2. Operación    → cómo se ejecuta y se recupera.
3. Comunicación → cómo se habla con otros servicios.
4. Evolución    → cómo se cambia sin romper.
```

Las cuatro categorías son útiles para identificar la clase de riesgo de cada decisión. «Contrato» define lo que otros pueden conocer; «operación» protege la integridad y la respuesta del servicio; «comunicación» contempla fallos de dependencia; y «evolución» intenta conservar compatibilidad durante los cambios. No son cajones aislados: una petición repetida, por ejemplo, afecta a la vez al contrato y a la consistencia de la operación.

La primera categoría, contrato, agrupa las prácticas relacionadas con lo que el servicio expone. La segunda, operación, agrupa las prácticas relacionadas con la ejecución. La tercera, comunicación, agrupa las prácticas relacionadas con las llamadas a otros servicios. La cuarta, evolución, agrupa las prácticas relacionadas con el cambio. Las cuatro categorías se aplican de forma conjunta: un servicio con buen contrato pero mala operación falla en producción. Un servicio con buena operación pero mal contrato rompe a los consumidores. La aplicación conjunta es lo que produce un servicio de calidad.

Por ejemplo, “usar idempotencia” solo se considera implantada cuando una misma clave enviada dos veces no crea dos órdenes y el resultado se conserva según un contrato definido. Esta distinción es esencial para interpretar los tests: una prueba del catálogo demuestra que filtra registros, pero no garantiza la presencia de timeouts o versionado.

La ausencia de buenas prácticas produce cuatro síntomas reconocibles. El primero es el contrato inestable, que cambia sin versionado y rompe a los consumidores. El segundo es la operación frágil, que no tolera fallos ni recupera el estado. El tercero es la comunicación caótica, sin timeouts ni reintentos controlados. El cuarto es la evolución bloqueada, donde ningún servicio puede cambiar sin coordinación global. Los cuatro síntomas se corrigen aplicando las buenas prácticas de contrato, operación, comunicación y evolución respectivamente. El curso aplica estas buenas prácticas al dominio de la acería de forma explícita y trazable. Marcar una propiedad EstaAplicada en un objeto no verifica automáticamente ese comportamiento: únicamente registra una declaración. En AceriaApp, estas clases constituyen un inventario educativo; las prácticas técnicas se van materializando en módulos posteriores y necesitan tests específicos de API, persistencia o resiliencia.

Buena práctica de contrato: exposición de DTOs

Una entidad de dominio y la representación de una API cumplen funciones diferentes. La entidad debe proteger reglas y mantener estado válido según las decisiones del negocio; el DTO expresa los datos que un consumidor está autorizado a enviar o recibir. Si devolvemos directamente una entidad EF Core, un cambio en el modelo de persistencia puede modificar accidentalmente el contrato HTTP, y una propiedad interna podría quedar expuesta o resultar editable sin intención. En AceriaApp, `OrdenDto` sirve para separar lo que la API publica de lo que `Orden` utiliza internamente. Esta separación también abre la posibilidad de utilizar DTOs distintos para crear una orden y para consultarla: no todos los campos de salida deberían aceptarse en la entrada. La transformación entre ambos modelos debe ser explícita y verificable, especialmente cuando se incorporen nuevas reglas y propiedades.

```
// Entidad de dominio: no se expone.
public class Orden
{
    public Guid Id { get; set; }
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public DateTime FechaCreacion { get; set; }
    public EstadoOrden Estado { get; set; }
}

// DTO público: se expone.
public record OrdenDto(
    Guid Id,
    string Cliente,
    string Material,
    int Cantidad,
    DateTime FechaCreacion);
```

El ejemplo presenta una entidad `Orden` con propiedades del modelo interno y un `OrdenDto` que deja fuera `Estado`. Esa diferencia permite decidir qué información debe publicarse y cuál se utiliza para controlar transiciones. La decisión real debería pasar por DTOs de entrada y salida adecuados: devolver `Estado` puede ser correcto en una consulta, aunque no debería significar que un cliente pueda asignarlo libremente al crear una orden.

La diferencia entre la entidad y el DTO no es solo de forma: la entidad puede cambiar sin romper el contrato, mientras que el DTO es el contrato. Si mañana la entidad Orden añade un campo Prioridad, el DTO sigue igual y los consumidores no se ven afectados. Si el DTO necesita el campo Prioridad, se

crea un DTO nuevo o se versiona el contrato. La separación entre entidad y DTO es una inversión inicial que se recupera en el primer cambio de la entidad.

Separar DTOs de entidades obliga a decidir cómo se realiza el mapeo entre representación pública y modelo interno. Esa transformación debe definir qué campos son de lectura, qué datos puede enviar el cliente y cuáles genera siempre el servidor. En Orden, el identificador y la fecha de creación no deberían quedar sujetos a valores arbitrarios del POST si representan decisiones del sistema. En una API pública, exponer directamente la entidad también puede generar problemas de sobrepublicación de propiedades futuras, referencias circulares o acoplamiento a atributos del ORM. Un DTO debe validarse como contrato estable y versionarse cuando cambie su significado. El fragmento de este bloque es conceptual: en el código fuente del punto 1.5 todavía no se crea OrdenDto ni se cambia el endpoint para usarlo. Esa implementación corresponde a una etapa posterior y no debe afirmarse como resultado de esta práctica.

Separar la entidad de dominio del DTO resulta útil incluso cuando ambos contienen inicialmente campos idénticos. La entidad representa el estado y las reglas que la aplicación controla; el DTO representa información que el consumidor está autorizado a enviar o recibir. En una operación de creación de órdenes no debería ser el cliente quien determine el identificador final o la fecha de creación, porque esas propiedades pertenecen al servidor. Un `CrearOrdenRequest` podría contener cliente, material y cantidad, mientras `OrdenResponse` incluiría también el identificador y el estado asignados tras persistir. Esta distinción reduce sobreescrituras involuntarias y permite evolucionar la entidad sin cambiar inmediatamente el contrato.

También debe decidirse qué campos son obligatorios, cómo se comunican los errores y cómo se representa la ausencia de un valor. Un atributo nullable en C# no define automáticamente toda la semántica de un contrato JSON: la validación de entrada, las anotaciones y la documentación OpenAPI deben ser coherentes. En un sistema con varios clientes, cambiar un campo de número a cadena o pasar de opcional a obligatorio puede ser incompatible aunque el servidor acepte la nueva compilación. Las pruebas de contrato deben comprobar el JSON y la respuesta HTTP observables, no solamente la existencia de una clase `record`.

Elemento	Responsabilidad
`Orden`	Estado interno y reglas del dominio
DTO de creación	Datos que un cliente puede solicitar al crear
DTO de respuesta	Datos que el servidor se compromete a publicar
Validación HTTP	Rechazar datos incorrectos con respuesta explícita
Mapeo	Traducir entre transporte y modelo interno

El primer estado de AceriaApp recibe una entidad `Orden` directamente en el endpoint POST. Esto es suficiente para introducir Minimal APIs y persistencia, pero no significa que el proyecto ya aplique la buena práctica explicada en este bloque. La teoría describe una mejora arquitectónica para etapas posteriores. En la práctica del punto 1.5, el catálogo registra declaraciones sobre buenas prácticas; no sustituye la creación y verificación de DTOs reales. Esta distinción entre estado actual y diseño objetivo evita que el alumno confunda conocer un principio con haberlo implementado.

Buena práctica de contrato: versionado en la URL

La versión de un contrato comunica una promesa a los consumidores: ciertas rutas, datos y reglas se mantendrán durante un periodo determinado. Situar v1 en la URL es una estrategia sencilla de hacer visible esa promesa, aunque no es la única forma de versionar HTTP. Cuando aparezca un cambio incompatible, podrá ofrecerse una ruta v2 mientras los clientes migran; eso implica mantener y probar ambas variantes el tiempo acordado. La dificultad no está en escribir un número en la ruta, sino en

saber qué diferencia de comportamiento justifica una nueva versión y cómo detectaremos consumidores antiguos. Añadir un campo opcional suele ser menos costoso que cambiar el significado de un identificador, pero incluso los cambios aparentemente aditivos necesitan pruebas. En AceriaApp empezaremos por `/api/v1`, y el módulo de APIs REST desarrollará con más detalle su semántica.

```
Endpoints versionados:

GET  /api/v1/ordenes
GET  /api/v1/ordenes/{id}
POST /api/v1/ordenes
GET  /api/v2/ordenes   ← versión futura, convive con v1
```

Las rutas muestran tres operaciones de la versión 1 y una futura versión 2 del recurso. v1 y v2 pueden convivir durante una migración, pero publicar una nueva ruta no sustituye un plan de compatibilidad ni pruebas que aseguren que los clientes antiguos siguen funcionando. El esquema no declara una implementación de v2 en M01.

El versionado en la URL tiene dos consecuencias prácticas. La primera es que el enrutamiento se resuelve con un prefijo explícito, sin lógica condicional. La segunda es que el consumidor declara explícitamente qué versión usa. La combinación de las dos consecuencias facilita la detección de consumidores obsoletos: el productor puede contar cuántas peticiones recibe cada versión y decidir cuándo retirar la antigua. El versionado en la URL es la práctica más adoptada en el ecosistema .NET y la que el curso aplica.

El prefijo /api/v1 ofrece una convención visible, pero no permite automáticamente mantener dos versiones. Para que /api/v2 conviva con la anterior deben existir rutas, transformaciones de datos y pruebas que comprueben el contrato de cada versión. También es necesario definir qué cambios admiten evolución compatible sin incrementar la versión mayor; añadir propiedades opcionales puede bastar si los consumidores toleran los campos desconocidos. Cuando un campo modifica su significado, mantener el nombre puede ser más peligroso que introducir una versión explícita. En AceriaApp el endpoint inicial ya incorpora /api/v1, pero todavía no existe una segunda versión. El material debe distinguir la práctica ya visible en el proyecto del diseño recomendado para el futuro. Las estadísticas de uso por versión permiten planificar la retirada de contratos antiguos con evidencias en lugar de suposiciones.

Buena práctica de operación: idempotencia



Figura. Flujo de dos solicitudes con la misma clave de idempotencia.

Los reintentos son inevitables en sistemas donde una respuesta puede perderse aunque la operación se haya ejecutado. Si un cliente envía una orden y agota su espera, quizá intente crearla otra vez; un servidor que no reconoce la repetición puede terminar almacenando dos órdenes. La **idempotencia** describe que repetir una operación no multiplica su efecto sobre el estado observable. HTTP ya define métodos con distintas propiedades de idempotencia, pero crear recursos mediante POST requiere

normalmente un mecanismo adicional: por ejemplo, una clave de operación generada por el cliente y almacenada junto al resultado. El reto es que la protección funcione incluso cuando lleguen dos peticiones simultáneas o existan varias réplicas de la API. El ejemplo del apartado permite reconocer la idea, pero no representa todavía una implementación completa ni forma parte del código ejecutable de la primera etapa de AceriaApp.

```
app.MapPost("/api/v1/ordenes", async (
    AceriaDbContext db,
    Orden orden,
    [FromHeader(Name = "Idempotency-Key")] string? idempotencyKey) =>
{
    if (!string.IsNullOrWhiteSpace(idempotencyKey))
    {
        var existente = await db.Ordenes
            .FirstOrDefaultAsync(o => o.IdempotencyKey == idempotencyKey);
        if (existente is not null)
        {
            return Results.Ok(existente);
        }
    }

    orden.Id = Guid.NewGuid();
    orden.IdempotencyKey = idempotencyKey;
    orden.FechaCreacion = DateTime.UtcNow;
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.Created($"/api/v1/ordenes/{orden.Id}", orden);
});
```

El endpoint consulta una clave de idempotencia antes de insertar una orden. Esa comprobación explica la intención de devolver el recurso ya creado cuando la clave se repite. No obstante, una consulta seguida de inserción **no es segura ante concurrencia**: necesita un índice único, tratamiento transaccional del conflicto y una política sobre reutilización de claves con cuerpos diferentes. Además, la entidad `Orden` usada en las primeras prácticas no incorpora `IdempotencyKey`, por lo que este código no se debe pegar sin completar el modelo.

La idempotencia es una propiedad de la operación, no un mecanismo reservado al POST. Según la semántica HTTP, GET y PUT son idempotentes, y DELETE lo es respecto al estado que deja la eliminación; las respuestas a sucesivas llamadas no tienen por qué ser idénticas. Un POST de creación requiere una protección adicional cuando deben evitarse efectos duplicados. La idempotencia de las operaciones de creación es la más crítica porque un duplicado produce un recurso duplicado. La clave de idempotencia se genera en el cliente y se envía en la cabecera. El servidor la comprueba y decide si ejecuta o devuelve el resultado anterior.

La idempotencia de un POST exige protección frente a concurrencia. Si dos solicitudes con la misma clave llegan al mismo tiempo, consultar primero y luego insertar puede permitir dos filas cuando ambas consultas observan que todavía no existe el registro. El almacenamiento necesita una restricción única sobre la clave dentro de su ámbito de validez y una gestión transaccional del conflicto. También debe definirse qué ocurre cuando una misma clave acompaña cargas de contenido distintas: devolver un error de conflicto suele ser más seguro que reutilizar silenciosamente una respuesta no relacionada. La repetición debe preservar la semántica acordada del resultado, incluida la identidad del recurso creado. En el módulo 1 el ejemplo presenta el algoritmo básico; la entidad Orden del proyecto no tiene IdempotencyKey ni se añade este endpoint idempotente. Por ello sería incorrecto afirmarlo como capacidad implementada al cerrar 1.5.

Consultar primero si existe una orden con una clave de idempotencia es un buen punto de partida conceptual, pero no garantiza por sí solo ausencia de duplicados. Dos peticiones concurrentes pueden ejecutar la consulta cuando la clave todavía no está registrada y ambas intentar insertar. Si la base de datos no aplica una restricción de unicidad, las dos operaciones pueden confirmarse. Por ello una implementación robusta necesita combinar un identificador estable de operación con una garantía de

persistencia: índice único sobre la clave dentro de su ámbito, transacción adecuada y tratamiento del conflicto de inserción. Además debe definirse durante cuánto tiempo se conserva el resultado de una operación y qué ocurre si se reutiliza la clave con un cuerpo de petición distinto.

La respuesta reutilizada también debe formar parte del contrato. Si una creación inicial devuelve 201 con identificador y ubicación, el reintento con la misma clave puede devolver una representación equivalente o la respuesta previamente almacenada según la política definida. Lo esencial es evitar ejecutar de nuevo el efecto de negocio y mantener una semántica documentada. En operaciones con pago o reserva de stock, el servidor puede necesitar guardar tanto el identificador como un resumen del cuerpo y el resultado de la primera ejecución. En cambio, usar simplemente un diccionario en memoria no resulta suficiente cuando existen varias réplicas: cada proceso tendría su propio registro de claves.

Secuencia de comprobación que debería superar una API idempotente:

1. Enviar una petición de creación con una clave nueva y comprobar la persistencia de una sola orden.
2. Repetir exactamente la petición con la misma clave y verificar que no aparece una segunda orden.
3. Enviar dos peticiones concurrentes con esa clave y verificar que la restricción de almacenamiento evita duplicados.
4. Reutilizar la clave con datos diferentes y comprobar el tratamiento definido para el conflicto.
5. Repetir la prueba con dos instancias de la API para descartar un control limitado a memoria local.

El bloque de código original ilustra una búsqueda por clave, pero la entidad `Orden` de la práctica inicial no incluye aún `IdempotencyKey` ni una restricción única. Es, por tanto, un ejemplo de diseño, no un endpoint copiable sobre la solución 1.1 sin modificaciones. Este aviso es esencial: la validación del catálogo de buenas prácticas de 1.5 no prueba la ausencia real de duplicados. La propiedad se demostrará cuando los escenarios HTTP y SQL anteriores formen parte de la implementación.

Buena práctica de operación: gestión explícita de errores

Una respuesta HTTP tiene significado para quien consume la API. Un `201 Created` informa de que se creó un recurso y puede indicar su ubicación; un `400 Bad Request` identifica una solicitud inválida; un `409 Conflict` puede expresar una transición incompatible con el estado actual. Si se devuelve siempre un 200 con un texto libre o un 500 ante cualquier situación, el cliente no puede reaccionar con seguridad. `ProblemDetails` ofrece una estructura conocida para describir fallos, pero su utilidad depende de elegir códigos coherentes y mensajes que no revelen datos privados. Una validación de cliente vacío corresponde a un error de entrada, no a un fallo inesperado de infraestructura. En AceriaApp se utilizará esta distinción para que el alumno pueda interpretar las pruebas HTTP y no limitarse a comprobar que el servidor respondió de alguna forma.

```
app.MapPost("/api/v1/ordenes", async (AceriaDbContext db, Orden orden) =>
{
    if (string.IsNullOrWhiteSpace(orden.Cliente))
    {
        return Results.Problem(
            title: "Cliente requerido",
            detail: "El campo Cliente no puede estar vacío.",
            statusCode: StatusCodes.Status400BadRequest);
    }

    orden.Id = Guid.NewGuid();
    orden.FechaCreacion = DateTime.UtcNow;
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
```

```
    return Results.Created($"/api/v1/ordenes/{orden.Id}", orden);
});
```

El fragmento valida que `Cliente` no esté vacío y devuelve un `ProblemDetails` con el estado 400 cuando falla. Si la entrada es válida, guarda la orden y responde 201 con una ubicación. Este contraste muestra cómo una condición de negocio puede traducirse al contrato HTTP; para utilizarlo en una API real deben añadirse las demás validaciones y el tratamiento de excepciones no previstas.

La gestión explícita de errores tiene dos consecuencias prácticas. La primera es que el cliente sabe exactamente qué ha fallado. La segunda es que el soporte puede reproducir el error con el código y el detalle. La adopción del formato problem details evita la duplicación de estructuras de error y facilita la integración con herramientas de monitorización. El curso aplica este formato en todos los endpoints que devuelven errores.

Los códigos HTTP deben distinguir situaciones diferentes del negocio. Un 400 indica que la solicitud no puede aceptarse por sus datos o formato; un 404 que el recurso solicitado no existe; un 409 que la operación entra en conflicto con el estado actual; un 500 que se produjo un fallo interno no esperado. El cuerpo Problem Details debe ofrecer información útil sin filtrar detalles internos, rutas sensibles ni credenciales. La especificación vigente RFC 9457 actualiza RFC 7807; ambos nombres pueden encontrarse en materiales anteriores. El ejemplo de Results.Problem ilustra una respuesta estructurada, pero no prueba que todos los endpoints manejen errores de esa manera. Una prueba de integración adecuada envía una petición inválida y comprueba el status code, el tipo de contenido y los campos devueltos. El validador declarativo de buenas prácticas no sustituye esta prueba.

El formato *Problem Details* permite que clientes distintos interpreten los errores de forma uniforme. No basta con devolver un JSON que contenga un mensaje libre: el código HTTP, el título y el detalle deben representar una situación real y estable. Un recurso inexistente se asocia normalmente con 404, una entrada inválida con 400 y un conflicto de estado con 409. Un fallo inesperado del servidor no debe exponerse al cliente mediante detalles sensibles de la excepción interna. El mensaje público describe el problema que el consumidor puede entender; el log interno registra la excepción, la correlación y los datos necesarios para diagnóstico seguro.

En operaciones de creación, la validación debe realizarse antes de persistir. Para `Orden`, podría verificarse que `Cliente` no esté vacío, que el material corresponda a una referencia válida y que `Cantidad` sea positiva. Son reglas diferentes: unas pertenecen al formato y otras al negocio. La API debe comunicar cuáles han fallado y evitar guardar un estado inválido. El uso de *Problem Details* no implica que todos los errores compartan el mismo código. La especificación HTTP y la semántica del dominio son las que orientan la elección; el cuerpo uniforme facilita al cliente procesarla.

Situación en AceriaApp	Respuesta orientativa	Razón
Cliente vacío en solicitud	400	Entrada que no supera validación
Orden consultada inexistente	404	El recurso solicitado no está disponible
Clave idempotente reutilizada indebidamente	409	Conflicto con una operación previa
Excepción de infraestructura no controlada	500	Fallo interno que debe registrarse

Una prueba HTTP comprueba algo distinto de un test unitario que devuelve `null` al consultar la base de datos. Para verificar 404 debe realizarse una petición al endpoint, observar el código de estado y comprobar que la respuesta no revela información no prevista. En el reto de 1.1 aparecen ambos conceptos: el código del endpoint contempla `Results.NotFound()`, mientras el test inicial comprueba únicamente una consulta vacía. La batería E2E completa la evidencia HTTP, pero la teoría debe mostrar por qué ambos niveles son necesarios. Con esa distinción, los alumnos entienden qué

requisito verifica cada prueba y evitan certificar un contrato mediante una simple comprobación interna.

Buena práctica de comunicación: timeouts y reintentos

Cuando Producción consulte Inventario, la duración de esa llamada debe estar acotada. Un timeout limita cuánto puede esperar un intento; un presupuesto de tiempo del caso de uso debe contemplar también reintentos y el trabajo anterior y posterior a la consulta. Repetir indiscriminadamente una operación solo porque una excepción parece transitoria puede amplificar la carga del servicio remoto y empeorar una caída. Para consultas idempotentes es posible establecer un número pequeño de intentos con una espera creciente; para escrituras hay que comprobar antes que no se duplicarán efectos. El ejemplo de Polly ilustra la mecánica de una política, pero no incorpora todas las decisiones necesarias en producción. Más adelante el curso empleará clientes HTTP configurados y políticas de resiliencia que puedan probarse contra fallos simulados y dependencias reales.

```
var policy = Policy
    .Handle<HttpRequestException>()
    .WaitAndRetryAsync(
        retryCount: 3,
        sleepDurationProvider: attempt => TimeSpan.FromMilliseconds(200 * Math.Pow(2, attempt)),
        onRetry: (exception, delay, attempt, _) =>
        {
            Console.WriteLine($"Reintento {attempt} tras {delay.TotalMilliseconds} ms");
        });

var httpClient = new HttpClient { Timeout = TimeSpan.FromSeconds(5) };
var response = await policy.ExecuteAsync(() =>
    httpClient.GetAsync("http://acería-inventario/api/v1/materiales"));
```

La política introduce hasta tres reintentos con intervalos exponenciales calculados a partir del número de intento. `HttpClient.Timeout` establece un límite para cada llamada, pero no garantiza por sí solo que el presupuesto total de varios intentos sea aceptable. Tampoco contempla todas las respuestas HTTP de error. En ASP.NET Core real conviene configurar clientes mediante `IHttpClientFactory` y definir explícitamente qué operaciones permiten reintentos.

La configuración de timeouts y reintentos no es opcional en sistemas distribuidos. La latencia de red es variable y los errores transitorios son frecuentes. Un servicio sin timeouts puede bloquearse indefinidamente. Un servicio sin reintentos propaga errores transitorios como fallos definitivos. La combinación de ambos reduce la tasa de fallo observada sin añadir complejidad al código. La política se aplica en un solo lugar y se reutiliza en todas las llamadas externas del servicio.

Una estrategia de reintentos debe controlar el presupuesto global de latencia. Tres intentos con timeout de cinco segundos pueden mantener al cliente esperando mucho más de cinco segundos si el límite se aplica a cada intento por separado. El intervalo exponencial presentado crece según las potencias de dos, pero debe completarse con una pequeña variación aleatoria cuando muchos clientes puedan reintentar simultáneamente. Es importante decidir qué estados HTTP ameritan reintento; una validación fallida no es transitoria, mientras un 503 puede reflejar sobrecarga recuperable. El cliente HTTP también debe crearse y reutilizarse mediante IHttpClientFactory cuando corresponda, en lugar de instanciar uno por petición. Los fragmentos Polly de este bloque son ejemplos de diseño. El proyecto del módulo 1 no registra un HttpClient externo ni prueba políticas de comunicación entre microservicios.

Timeout y reintento resuelven problemas diferentes. El timeout establece cuánto puede esperar una operación antes de abandonar la llamada; el reintento determina si vale la pena iniciar una petición nueva tras un fallo elegible. Si se configura un timeout de cinco segundos y se permiten tres reintentos completos, la duración máxima observable puede exceder ampliamente esos cinco segundos porque el límite se aplica a cada intento y además existen esperas entre ellos. Una política

responsable define un presupuesto global de la operación y comprueba que los reintentos caben dentro de él. Ese presupuesto debe ser coherente con el límite de espera del consumidor final.

El retroceso exponencial aumenta la pausa entre intentos para reducir la presión sobre una dependencia degradada. Por ejemplo, una política ilustrativa con esperas de 200, 400 y 800 milisegundos introduce pausas crecientes; un cálculo lineal de `200 * intento` produciría 200, 400 y 600 milisegundos y no sería exponencial. Se recomienda incorporar variación aleatoria controlada cuando muchos clientes podrían reintentar al mismo tiempo, evitando que todos vuelvan a saturar el destino simultáneamente. Sin embargo, la política nunca debe aplicarse mecánicamente a operaciones que producen efectos no idempotentes.

Antes de permitir reintentar un POST de creación de órdenes hay que conocer su semántica. Si el servicio ha confirmado la orden pero la respuesta se pierde, el cliente no sabe si una segunda petición creará un duplicado. La solución puede incluir una clave de idempotencia persistida y una forma de consultar el resultado anterior. Sin esa garantía, el reintento automático de una escritura es una fuente de errores de negocio. Para una consulta GET la situación es más sencilla, pero siguen siendo necesarios límites de frecuencia y clasificación de fallos transitorios.

En la teoría de AceriaApp, los fragmentos con Polly muestran la sintaxis de las políticas, no una configuración ya incorporada a los proyectos de módulo 1. La evidencia de resiliencia futura debe demostrar cuántas llamadas se realizaron, cuánto duró el escenario y qué hizo la aplicación ante un fallo prolongado. Un test que solo construye un objeto `Policy` no acredita esa conducta. Esta explicación ayuda a relacionar los ejemplos del módulo con las prácticas E2E que serán necesarias cuando aparezcan comunicaciones entre servicios.

Buena práctica de comunicación: circuit breaker

Un circuit breaker protege al consumidor cuando una dependencia empieza a fallar repetidamente. Mientras el circuito está abierto, nuevas llamadas no intentan acceder al destino, sino que reciben inmediatamente una señal de indisponibilidad; después se permite comprobar si se ha recuperado. Esto evita que cada nueva petición consuma conexiones y tiempo esperando a un servicio del que ya existen suficientes señales de fallo. No se trata de reparar mágicamente Inventario ni de fabricar una respuesta válida. En una operación de consulta podría existir un comportamiento alternativo claramente identificado; para una planificación que necesita confirmar una regla de negocio, quizá debamos rechazar la solicitud. El umbral de apertura, la duración y el ámbito compartido del circuito afectan al comportamiento real. El fragmento muestra una configuración conceptual que tendrá que contrastarse con pruebas cuando el sistema incorpore comunicación entre procesos.

```
var circuitBreaker = Policy
    .Handle<HttpRequestException>()
    .CircuitBreakerAsync(
        exceptionsAllowedBeforeBreaking: 3,
        durationOfBreak: TimeSpan.FromSeconds(30),
        onBreak: (exception, timespan) =>
        {
            Console.WriteLine($"Circuito abierto durante {timespan.TotalSeconds} s");
        },
        onReset: () =>
        {
            Console.WriteLine("Circuito cerrado de nuevo");
        });
```

El breaker se abre después de alcanzar el umbral de excepciones configuradas y mantiene temporalmente interrumpidas las llamadas protegidas. `onBreak` y `onReset` muestran transiciones del estado del circuito, no el restablecimiento de la dependencia remota. Las pruebas posteriores deberán comprobar tanto la protección del consumidor como la recuperación cuando Inventario vuelva a responder.

El circuit breaker se combina con el reintento y con el timeout para formar una política completa de resiliencia. El reintento recupera errores transitorios. El timeout evita bloqueos indefinidos. El circuit breaker evita la saturación cuando el destino falla de forma sostenida. Las tres políticas se aplican en capas: primero el timeout, después el reintento, después el circuit breaker. El orden de aplicación determina el comportamiento observado. El curso introduce el circuit breaker en el módulo 7 con detalle; en el módulo 1 se presenta como buena práctica.

Un circuit breaker puede tener estados cerrado, abierto y de prueba de recuperación. Mientras está abierto, las solicitudes no deben continuar golpeando una dependencia que probablemente fallará; al cumplirse el periodo de espera se admiten intentos limitados para comprobar su recuperación. El número de errores consecutivos y los treinta segundos del ejemplo son parámetros didácticos que requieren medición en sistemas reales. El circuito no sustituye el timeout ni el reintento, y una composición incorrecta de políticas puede aumentar la carga sobre el servicio remoto. Debe supervisarse el número de aperturas y la proporción de solicitudes rechazadas rápidamente. En AceriaApp todavía no se añade este circuito al host monolítico; se estudiará en el módulo de resiliencia. La práctica de 1.5 clasifica una declaración de su adopción, no simula fallos ni valida el funcionamiento de Polly.

Buena práctica de evolución: contratos compatibles

Un productor de API debe pensar en sus consumidores cuando modifica un contrato. Añadir un campo opcional suele permitir que clientes antiguos continúen funcionando, pero no siempre: un cliente que deserializa de forma estricta o un valor nuevo de un enum no previsto pueden provocar errores. Eliminar un campo obligatorio, cambiar un tipo o reinterpretar su significado suele requerir una nueva versión o una migración coordinada. La compatibilidad también puede romperse sin cambiar el JSON, por ejemplo al endurecer una validación o alterar el orden de operaciones de un flujo. Para AceriaApp resulta útil escribir pruebas de contrato que simulen a un consumidor anterior y comprueben que sigue comprendiendo las respuestas. La evolución independiente empieza por diseñar cambios que no obliguen a publicar todos los componentes a la vez.

```
Cambios compatibles:

+ Añadir campo opcional al DTO.
+ Añadir nuevo endpoint.
+ Añadir nuevo valor a un enum existente.

Cambios incompatibles:

- Eliminar campo del DTO.
- Cambiar tipo de campo del DTO.
- Cambiar el significado de un campo existente.
- Eliminar un endpoint.
```

La clasificación muestra cambios habitualmente compatibles y otros que suelen romper consumidores. Hay una excepción importante: añadir un valor a un enum puede ser incompatible con clientes que tratan exhaustivamente todos los valores; de forma semejante, un campo aparentemente opcional puede alterar validaciones o decisiones del cliente. Por eso la lista es una guía para preparar pruebas, no una garantía de compatibilidad.

La compatibilidad de contratos se verifica con tests que consumen la versión antigua y comprueban que siguen funcionando. Estos tests se ejecutan en el pipeline antes de publicar una nueva versión. Si un test falla, el cambio es incompatible y requiere una nueva versión del contrato. La verificación automática evita que un cambio incompatible llegue a producción. El curso introduce los tests de compatibilidad en el módulo 2 con detalle; en el módulo 1 se presenta como buena práctica.

La compatibilidad hacia atrás depende de cómo interpreten los clientes los cambios. Añadir un valor a un enum puede ser incompatible para consumidores que utilicen una selección exhaustiva de casos o

que rechacen valores desconocidos, de modo que no debe clasificarse universalmente como seguro. Añadir un campo opcional suele ser menos arriesgado, siempre que no modifique el significado de los campos existentes. Un test de contrato debería construir peticiones con un cliente antiguo contra una versión nueva del productor y comparar los resultados relevantes. Los cambios de esquema de datos también requieren compatibilidad durante el despliegue gradual de varias instancias. En AceriaApp el módulo 1 explica estas reglas, pero no aporta aún una suite de pruebas de compatibilidad HTTP. La declaración de buenas prácticas no debe confundirse con evidencia de compatibilidad real; esa verificación se abordará al desarrollar las APIs.

Añadir un campo opcional a una respuesta suele ser más seguro que eliminar uno, pero la compatibilidad depende del consumidor. Algunos clientes ignoran campos desconocidos; otros usan validadores estrictos o generan clases que rechazan diferencias. Añadir un valor a un enumerado también puede afectar a un cliente que asume conocer todas las alternativas y falla al recibir un valor nuevo. Por tanto, la compatibilidad hacia atrás debe definirse respecto a consumidores concretos y comprobarse con pruebas reales. Una tabla orientativa de cambios ayuda a razonar, pero no reemplaza la verificación de comportamientos en cada integración.

Un cambio debe evaluarse por sintaxis y semántica. Mantener el nombre `Cantidad` pero cambiar su unidad de kilogramos a toneladas rompe el significado del contrato aunque el JSON siga conteniendo el mismo número. Del mismo modo, conservar una respuesta 200 cuando la operación debería devolver un conflicto puede inducir errores en los clientes. Las versiones de URL permiten mantener dos contratos cuando aparece una incompatibilidad, pero también requieren gobernanza: saber qué clientes utilizan v1, comunicar la transición y establecer criterios para retirar una versión antigua.

En AceriaApp, la ruta `/api/v1/ordenes` sirve como primera referencia. Una futura ruta v2 podría publicar una representación nueva, pero no debería introducirse únicamente por añadir una propiedad interna a la entidad. El cambio se decide cuando la interfaz observable realmente lo exige. Un inventario de consumidores ayuda a evitar versiones eternas o retiradas prematuras. Los registros deben permitir determinar qué versiones se utilizan sin exponer datos sensibles.

La prueba de compatibilidad debe ejercitar un cliente que conozca el contrato antiguo, invocar el productor actualizado y comprobar que conserva la semántica esperada. Un test unitario del DTO nuevo no demuestra esa propiedad. La documentación OpenAPI, los tests de consumidor y el versionado explícito forman un conjunto: la especificación declara el contrato, la ejecución lo cumple y las pruebas detectan desviaciones. Esta relación es una de las razones por las que la práctica de buenas prácticas no debe limitarse a registrar un booleano que afirma que «el contrato es compatible». La declaración es útil para auditar intenciones; la evidencia de integración es la que permite aprobar la compatibilidad.

Buena práctica de evolución: documentación viva

La documentación es útil cuando coincide con la aplicación que se está ejecutando. OpenAPI permite describir rutas, parámetros, esquemas y respuestas de una API ASP.NET Core; Swagger UI facilita explorarlos y realizar peticiones durante el desarrollo. Pero publicar una especificación no demuestra por sí solo que el comportamiento se cumpla: una ruta puede figurar en el documento y devolver errores inesperados o no persistir los datos. La documentación viva requiere mantener sincronizados contrato y ejecución mediante generación, pruebas y revisión. En AceriaApp seguiremos el recorrido desde una definición de endpoint hasta una petición real y su resultado; el alumno tendrá que reconocer qué parte del contrato queda documentada automáticamente y cuáles decisiones semánticas necesitan explicarse por escrito.

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
```

```
        Title = "AceriaApp.Ordenes.Api",
        Version = "v1",
        Description = "API de gestión de órdenes de producción."
    });
});

app.UseSwagger();
app.UseSwaggerUI(options =>
{
    options.SwaggerEndpoint("/swagger/v1/swagger.json", "AceriaApp.Ordenes.Api v1");
});
```

El registro de `AddEndpointsApiExplorer` y `AddSwaggerGen` permite construir un documento OpenAPI de la API; `UseSwagger` y `UseSwaggerUI` lo exponen para consulta durante el desarrollo. Esa configuración debe ajustarse al entorno y a la tecnología de documentación elegida. El documento describe la superficie HTTP, mientras que las pruebas siguen siendo necesarias para comprobar el comportamiento real de las operaciones.

La documentación viva tiene dos consecuencias prácticas. La primera es que el contrato público está siempre actualizado porque se genera a partir del código. La segunda es que los consumidores pueden probar los endpoints directamente desde la interfaz gráfica. La combinación de ambas reduce el tiempo de integración entre servicios. La documentación se versiona con el código y se publica con el servicio. El curso usa OpenAPI con Swagger en todos los servicios.

OpenAPI se genera a partir de la configuración y los metadatos del servicio, por lo que su calidad depende de que estén declaradas rutas, tipos de respuesta y posibles errores. Swagger UI facilita la exploración, pero no garantiza por sí sola que los ejemplos se correspondan con las validaciones reales del endpoint. El documento debe publicarse de forma coherente con la configuración de seguridad y con el entorno de despliegue. En producción puede decidirse restringir la interfaz de pruebas mientras se mantiene una especificación consumible por herramientas autorizadas. El bloque utiliza OpenApiInfo, que requiere el espacio de nombres y paquete adecuados cuando se compila como ejemplo. En el proyecto 1.1 se registra Swagger para GET y POST básicos; en el 1.5 no se amplía con el catálogo de contratos futuros. La documentación del código publicado debe reflejar esa diferencia.

La documentación OpenAPI no se mantiene actualizada por el mero hecho de registrar `AddSwaggerGen`. El resultado depende de los endpoints, sus tipos de respuesta declarados y los metadatos que permitan al generador describir el contrato. Un endpoint que devuelve indistintamente varios tipos sin documentar los códigos posibles puede producir una especificación incompleta. En Minimal APIs conviene declarar de forma explícita los resultados relevantes, la validación y los ejemplos necesarios para que el consumidor comprenda la operación. El objetivo no es decorar Swagger UI, sino publicar una descripción verificable de las entradas y salidas que soporta el servicio.

El contrato generado puede inspeccionarse automáticamente. Una prueba E2E arranca la API, obtiene `/swagger/v1/swagger.json` y comprueba que existen rutas, verbos y esquemas esperados. Esa comprobación es más fuerte que mirar únicamente el registro de servicios en `Program.cs`, pero tampoco es suficiente para garantizar toda la semántica: un endpoint puede aparecer en el documento y devolver 500 cuando se ejecuta. Por ello es útil complementar el contrato con peticiones reales que ejerciten éxito y error, incluyendo persistencia con SQL Server cuando el comportamiento depende de la base de datos.

Evidencias mínimas para la API de Órdenes:

- El documento OpenAPI incluye las rutas previstas y sus métodos HTTP.
- La petición POST con datos válidos devuelve una respuesta de creación y persiste un identificador de orden.
- La consulta GET recupera la orden creada desde el almacenamiento configurado.
- La petición a una orden inexistente produce 404 en el endpoint de detalle.
- La API conserva el contrato esperado tras los cambios de implementación del punto.

El proyecto inicial ya ofrece una API mínima documentada y un escenario de persistencia que puede comprobarse. No debe afirmarse, sin embargo, que implemente idempotencia, DTOs separados o todas las políticas de resiliencia solo porque esos asuntos aparecen en la teoría. La documentación viva refleja lo que existe y permite detectar lo que falta. Esta disciplina cierra el módulo con una idea transferible: cada afirmación técnica debería vincularse a un artefacto observable y a una prueba adecuada al riesgo que se pretende controlar.

Aplicación de las buenas prácticas al proyecto del curso

El módulo se cierra conectando los principios con la evolución real del proyecto. Los DTO separarán los contratos públicos de las entidades; las rutas versionadas permitirán introducir cambios controlados; los errores HTTP expresarán estados de negocio y fallos técnicos; y las políticas de resiliencia protegerán la comunicación cuando aparezcan servicios separados. No todo ese comportamiento existe todavía en la API monolítica de referencia. La práctica inicial introduce validadores y ejemplos que ayudan a reconocer riesgos; la capacidad funcional solo quedará demostrada cuando existan endpoints, persistencia, mensajes y pruebas específicas que la ejerzan. Este criterio evita convertir una lista de buenas prácticas en una falsa certificación de producción. La pregunta de cierre para cada mecanismo será qué error previene, qué comportamiento obliga a implementar y cómo comprobaremos que realmente lo hace.

```
Aplicación al proyecto del curso:

DTOs separados          → OrdenDto en lugar de Orden.
Versionado en URL       → /api/v1/ordenes.
Idempotencia            → cabecera Idempotency-Key.
Errores explícitos      → problem details.
Timeouts y reintentos   → Polly.
Circuit breaker         → Polly.
Contratos compatibles   → tests de compatibilidad.
Documentación viva      → OpenAPI.
```

El cuadro reúne prácticas que aparecerán en distintos módulos. En M01 varias son propuestas conceptuales; su implementación evoluciona al incorporar Minimal APIs, DDD, mensajería, Polly y YARP. La distinción evita atribuir a un validador de catálogo la capacidad de impedir duplicados, recuperar una caída remota o proteger contratos que todavía no se consumen.

La aplicación de las buenas prácticas prepara el proyecto para la evolución hacia microservicios. Cuando un módulo se extraiga como servicio, mantendrá las mismas buenas prácticas. La coherencia entre el monolito modular y los microservicios facilita la transición. El punto 1.5 cierra el módulo 1 y prepara el terreno para el módulo 2, donde se estudiará el desarrollo de APIs REST con ASP.NET Core.

El cierre del módulo debe distinguir resultados construidos de prácticas únicamente estudiadas. El proyecto ejecutable contiene un monolito Minimal API, EF Core y una migración inicial, además de clases educativas para métricas, límites, convenciones y catálogo de buenas prácticas. Los tests básicos verifican esas clases y suman veintitrés en la última solución cuando se activan los bloques de los puntos sucesivos. No se incluyen todavía un servicio de Inventario separado, contratos DTO públicos, garantías de idempotencia POST, Saga, Outbox o Polly integrado en llamadas reales. El balance es útil precisamente porque registra ese límite. El siguiente módulo puede construir APIs y pruebas de respuesta HTTP que hagan efectivas varias recomendaciones teóricas. La trazabilidad del aprendizaje exige que cada capacidad anunciada como realizada tenga una ruta de código y una prueba observables.

Caso de análisis — Del contrato declarado a la evidencia de una API fiable

El punto 1.5 reúne prácticas que pueden parecer similares porque todas pretenden mejorar la calidad, pero protegen riesgos diferentes. Usar un DTO evita que el cliente dependa del modelo interno.

Versionar una ruta permite administrar cambios incompatibles. Validar una petición y responder con *Problem Details* hace explícito por qué no se ha aceptado. La idempotencia evita efectos duplicados cuando una operación se repite. Los timeouts y los reintentos limitan el efecto de fallos de red, y un circuit breaker contiene periodos prolongados de indisponibilidad. La documentación OpenAPI informa del contrato; las pruebas verifican que la implementación lo respeta. Ninguna de estas propiedades se deduce automáticamente de las demás.

Tomemos una orden de fabricación enviada dos veces con la misma intención de negocio. La primera petición llega al servidor, inserta la orden y devuelve 201. Por un problema de red, el cliente no recibe la respuesta y repite el POST. Si el servicio genera siempre un GUID nuevo y guarda sin comprobar una clave de operación, quedarán dos órdenes. El cliente podría creer que «el retry ha solucionado el error», cuando en realidad se ha duplicado el efecto. La solución idempotente define una clave estable, comprueba su uso, impide inserciones concurrentes duplicadas mediante una restricción en el almacenamiento y devuelve un resultado documentado para las repeticiones. Esta secuencia muestra que idempotencia no significa simplemente ejecutar el mismo método dos veces.

Escenario	Comportamiento que debe probarse	Nivel de evidencia
POST válido	Se crea una única orden	HTTP + SQL Server
POST sin cliente	Se rechaza con error de validación	Contrato HTTP
GET de identificador inexistente	Respuesta 404	HTTP real
Repetición con clave de idempotencia	No aparece una segunda orden	HTTP concurrente + restricción SQL
Dependencia externa lenta	Se respeta el presupuesto global de espera	E2E con fallo simulado
Cambio en DTO interno	El consumidor antiguo sigue funcionando	Test de compatibilidad

La prueba del GET inexistente ilustra cómo se distinguen niveles. Consultar un `DbSet` y obtener `null` acredita que la consulta no encontró datos; no prueba el código HTTP que finalmente ve un cliente. Para probar 404 debe invocarse la ruta real. A su vez, comprobar que el endpoint devuelve 404 no demuestra que una transacción distribuida de inventario sea consistente. Cada afirmación exige una prueba que atraviese las fronteras relevantes. Esta disciplina permite usar las pruebas E2E como instrumento de aprendizaje y no solo como un resultado verde en el pipeline.

El módulo 1 aún no incorpora todas las prácticas descritas al comportamiento de la API. En particular, el `ValidadorBuenasPracticas` recibe una lista de objetos `Practica`, revisa categorías permitidas y detecta declaraciones pendientes. Sus tests demuestran esa lógica, pero no inspeccionan automáticamente endpoints, contratos ni políticas Polly. Por eso sería incorrecto marcar que la API es idempotente solamente porque existe una entrada con `Codigo = "O-01"` y `EstaAplicada = true`. El catálogo es un instrumento de seguimiento, no un certificado técnico de cumplimiento de los mecanismos declarados.

El cierre razonado del módulo distingue tres niveles de madurez. Primero, el alumno conoce el principio y puede explicar el riesgo que protege. Segundo, dispone de un componente de ejemplo cuya lógica se prueba mediante xUnit. Tercero, cuando el tema se implemente realmente en la API, se exigirá un escenario E2E que verifique el efecto sobre HTTP, persistencia y fallos concurrentes. En AceriaApp, la API inicial está operativa y probada para creación y consulta; las políticas distribuidas y los contratos avanzados se introducen de manera progresiva en módulos posteriores. Esta separación entre conocimiento, prototipo de regla y funcionalidad plenamente integrada es la clave para evaluar honestamente los resultados del curso.

Síntesis: de la recomendación al comportamiento verificado

El criterio que conecta todas las buenas prácticas es que cada una debe proteger un contrato o una regla de negocio identificable. Separar DTOs, versionar rutas, describir errores y gestionar idempotencia disminuye las sorpresas de los consumidores; acotar esperas y abrir circuitos puede proteger la aplicación ante fallos remotos; documentar y probar compatibilidad facilita la evolución. No basta con disponer de ejemplos de C#: antes de afirmar que una capacidad está implantada debemos observar su efecto en peticiones, datos y escenarios de fallo. Esa exigencia acompañará la evolución de AceriaApp a lo largo de los módulos siguientes.